

Architecting Safe and Modular AGI

Joachim De Witte

Architecting Safe and Modular AGI
2026

Contents

Contents	3
1 A Fable of How Minds Learn to Think Together	1
1.1 The Great Parrot — A Mind Too Large to Think	1
1.2 The Generative Aviary — A Thousand Voices Without Direction	2
1.3 The Colony — Where Many Small Minds Learn to Think Together	2
1.4 The Moral — Intelligence Is a Society	4
2 The End of the Monolith	5
2.1 The Limits of the Monolithic Dream	5
2.2 A Shift in Understanding	6
2.3 The Modular Path to AGI	6
2.4 Why Modular Intelligence Is Safer	6
3 The Four Dimensions of Intelligence	9
3.1 Multiplicity: Intelligence Needs Many Parts	9
3.2 Richness: Intelligence Needs Many Ways to Represent the World	10
3.3 Recursion: Intelligence Needs to Think About Its Own Thinking	10
3.4 Energy: Intelligence Needs Throughput	10
3.5 The Quad as a Structural Law	11
3.6 The Quad as the Foundation of AGI	11
4 Biological Intelligence as a Quad	13
4.1 Multiplicity: Biological Subsystems	13
4.2 Richness: Representational Depth	13
4.3 Recursion: Thinking About Thinking	13
4.4 Energy: Extreme Efficiency	13
4.5 Summary	14
5 Artificial Intelligence as a Computational Quad	15
5.1 Multiplicity: Computational Subsystems	15
5.2 Richness: High-Dimensional Representations	15
5.3 Recursion: Inference and Self-Refinement	15
5.4 Energy: Compute and Power	15
5.5 Summary	15

6	Hybrid Intelligence as a Distributed Quad	17
6.1	Multiplicity: Heterogeneous Subsystems	17
6.2	Richness: Shared Representational Capacity	17
6.3	Recursion: The Interaction Loop	17
6.4	Energy: Human Effort and Machine Compute	17
6.5	Summary	18
7	A Historic Vision on AGI: Minsky and the Society of Mind	19
8	What Are Neural Networks?	21
8.1	Neurons and Layers	21
8.2	The Role of Activation Functions	22
8.3	Common Activation Functions	22
8.3.1	ReLU	22
8.3.2	Leaky ReLU	22
8.3.3	Sigmoid	22
8.3.4	Tanh	23
8.3.5	GELU	23
8.3.6	SiLU / Swish	23
8.4	Why Activation Functions Matter in AGI Toys	23
9	Toy Architect: Reasoning using PRE	25
9.1	Introduction — A Tiny Universe of Thought	25
9.2	A Symbolic World — The Language of Thought	25
9.3	The Architect — A Minimal Reasoning Organism	26
9.4	The Evaluator — Oracle of the Universe	26
9.5	The Reasoning Game — Propose, Evaluate, Refine	26
9.6	Cognitive Geometry — The Sliding Window	27
9.7	Developmental Physics — Curriculum and Training	27
9.8	Hybrid Intelligence in Miniature	28
9.9	The Septad — The Architecture of Architectures	28
9.10	Why This Tiny Universe Matters	28
9.11	Iterative Pattern Discovery	29
9.12	Hybrid Intelligence in Miniature	30
9.13	From Toy Universes to Reasoning Architectures	31
10	The PRE Family	33
10.1	Introduction	33
10.2	Limitations of Classical Models	33
10.3	The Three Roles	34
10.4	The PRE Cycle — A Four-Step Engine of Reasoning	34
10.5	PRE as an Architectural Pattern	35
10.6	Architectural Principles of the PRE Family	35
10.7	PRE as the Foundation of Modular Intelligence	35
10.8	Internal Error Languages — Machine-Native Semantics	36
10.9	Two Refinement Regimes — Absolute and Delta	36
10.10	Specialized Evaluator Architectures	37
10.11	Architectural Variants — COMPOSITE and MONO	38

10.12	The PRE Architectural Space	39
10.13	Lightweight Implementation and Universal Training	40
10.14	Synthesis — PRE-Networks as Minimal Universes of Reasoning	40
11	CAS — The Cognitive Vector Layer	43
11.1	Introduction — Why Intelligent Systems Need Cognitive Vectors	43
11.2	From Triadic Epistemics to CAS(v)	43
11.3	CAS as a Cognitive Interface for Mixed and Hybrid Data	45
11.3.1	CAS as a Labeling Layer	45
11.3.2	Learning Safety and Accuracy During Training	46
11.3.3	Relevance and Domain Conditioning	46
11.3.4	Style and Personality as Cognitive Dimensions	46
11.3.5	Why CAS Works for Hybrid and Contaminated Datasets	47
11.4	CAS-Networks — A Minimal Architecture for Cognitive Vectors	47
11.4.1	Multi-Output Architecture	47
11.4.2	Why CAS-Networks Work So Well	48
11.4.3	CAS Without PRE	48
11.4.4	CAS as Structural Decomposition	48
11.5	CAS-N — A Society of Cognitive Minds	49
11.5.1	Multiple CAS-Networks with Independent Histories	49
11.5.2	Consensus as Emergent Agreement	49
11.5.3	Disagreement and Emergent Abort	50
11.5.4	Internal Error Languages	50
11.5.5	CAS-N as a Special Case of CAS(v)	51
11.6	CAS in Practice — Training, Labeling, and Generation	51
11.6.1	Training with CAS-Labeled Data	51
11.6.2	Learning Safety During Training	52
11.6.3	Accuracy and Reliability as Cognitive Dimensions	52
11.6.4	Relevance and Domain Conditioning	53
11.6.5	Style and Personality Control	53
11.6.6	Using CAS During Generation	54
11.6.7	CAS as a Replacement for Prompt Engineering	54
11.7	CAS as the Epistemic Layer of Hybrid Intelligence	55
11.7.1	Why Hybrid Systems Need Epistemics	55
11.7.2	CAS as a Structural Safety Mechanism	55
11.7.3	CAS as a Guide for Recursive Reasoning	55
11.7.4	CAS-N as Distributed Epistemics	56
11.7.5	Integration with PRE — The PRE–CAS Family	56
11.7.6	Why CAS Is the Epistemic Layer	56
11.8	Synthesis — CAS as a Universal Cognitive Substrate	57
12	The PRE–CAS Family: A General Framework for Modular Reasoning	59
12.1	Introduction	59
12.2	The PRE–CAS Stack	59
12.3	Design Principles	60
12.4	Paging: Domain Decomposition of the Action Space	60
12.4.1	The Action Space	60
12.4.2	Definition of a Page	61

12.4.3	The Paging Function	61
12.4.4	Experts Within a Page	62
12.4.5	Why Paging Matters	62
12.5	The PRE Loop: Propose, Refine, Evaluate	62
12.5.1	Input Structure	63
12.5.2	The Propose Step	63
12.5.3	The Refinement Step	63
12.5.4	Evaluator Feedback	63
12.5.5	Accumulating Candidates	64
12.5.6	PRE Pseudocode	64
12.5.7	Why PRE Matters	64
12.6	Evaluators: Orthogonal Constraint Functions	64
12.6.1	Evaluator Functions	65
12.6.2	Types of Evaluators	65
12.6.3	Orthogonality	65
12.6.4	Evaluator Interface	66
12.6.5	Evaluator Set	66
12.6.6	Role in the PRE-CAS Pipeline	66
12.7	Filtering: Validity Before Selection	66
12.7.1	Input to Filtering	67
12.7.2	Hard Constraints	67
12.7.3	Soft Thresholding	67
12.7.4	Filtered Candidate Set	68
12.7.5	Why Filtering Matters	68
12.8	The Arbiter: Integration and Controlled Variation	68
12.8.1	Input to the Arbiter	69
12.8.2	Scoring Function	69
12.8.3	Selecting the Best Candidate	69
12.8.4	Controlled Variation	70
12.8.5	Arbiter Pseudocode	70
12.8.6	Why the Arbiter Matters	70
12.9	Ensembles: Plurality of Minds	71
12.9.1	Definition of an Ensemble	71
12.9.2	Parallel Reasoning	71
12.9.3	Ensemble Arbitration	71
12.9.4	Benefits of Ensembles	72
12.9.5	Architectural Neutrality	72
12.9.6	Summary	72
12.10	Summary and Architectural Outlook	72
12.10.1	Architectural Summary	73
12.10.2	Design Properties	73
12.10.3	Outlook	73
12.11	PRE-CAS Overview Table	74

13 Toy Story: Generation Using PAGED-PRE-CAS	77
13.1 Introduction: Why Toy Story?	77
13.2 Tokenization and the World Space	77
13.3 Embeddings and the Context Window	78
13.4 Paging: Decomposing the Vocabulary into Experts	80
13.5 PRE: Propose, Refine, Evaluate	81
13.6 Evaluators: The Mini-Experts	83
13.7 Token Filtering Before Selection	85
13.8 The Arbiter: Scoring and Variation Injection	86
13.9 The Ensemble: A Population of Models	88
13.10 Collective Recursion (With and Without Mutation)	89
13.11 Summary: What Toy Story Reveals	92
13.12 Toy Story as a Microscope for Generative Behaviour	92
13.13 Curriculum Learning Results in the Toy Story Universe	93
13.13.1 Effect of Curriculum Dataset Size	93
13.13.2 Effect of Refinement Steps	94
13.13.3 Effect of Evaluators	96
13.13.4 Effect of Score Threshold	97
13.14 Collective Recursion Training	98
13.14.1 Collective Recursion Without Mutation	98
13.14.2 Collective Recursion With Mutation	98
13.14.3 Effect of Lowering the Threshold After Recursion	99
13.14.4 Interpretation	99
13.15 The Triadic Cosmos Dataset Scaling Law	100
13.16 Conclusion: The Lesson of Toy Story	101
14 A Path to Modular and Safe AGI	103
14.1 Why Safe AGI Requires Architecture, Not Alignment	104
14.2 The Developmental Path to AGI	105
14.3 AGI as a Structural Phase Transition	108
14.4 Structural Safety Principles	109
14.5 The AGI Society of Minds	111
14.6 Synthesis — A Safe Path to AGI	113
14.7 The Book as Evidence of a Cognitive Phase Transition	114
15 Domain-Bound AGI: A Composite Reasoning Architecture	117
15.1 Introduction	117
15.2 Core Principles	118
15.2.1 Compositionality	119
15.2.2 Hybrid Intelligence	119
15.2.3 Society of Mind	119
15.3 Architecture Overview	120
15.3.1 Reasoning Language Model (Paged-PRE-CAS)	120
15.3.2 Script Engine	121
15.3.3 Object-Referenced Memory Map	121
15.3.4 Expert Modules	122
15.3.5 Generic Depth Search	123
15.3.6 Callback-Driven Self-Prompting	124

15.4	Hierarchical Contexts, Parallel Reasoning, and Asynchronous Processing	125
15.4.1	Contexts as First-Class Cognitive Objects	125
15.4.2	Hierarchical Context Trees	125
15.4.3	Parallel Reasoning Flows	125
15.4.4	Asynchronous Processing	126
15.4.5	Callback-Driven Continuation	126
15.4.6	Advantages Over Token-Based Reasoning	127
15.4.7	A Bidirectional Benefit for Transformers	127
15.4.8	Summary	127
15.5	Monte Carlo and Domain-Specific Search as Depth Reasoning Mechanisms	127
15.5.1	Monte Carlo as a Domain-Agnostic Search Mechanism	128
15.5.2	Domain-Specific Search: Beam Search, A*, and Beyond	128
15.5.3	Integration with the Reasoning Model	129
15.5.4	Replaceability and Extensibility	129
15.5.5	A Bidirectional Benefit for Transformers	130
15.5.6	A Computational Substrate for General Reasoning	130
15.6	Self-Learning Without Weight Updates	130
15.6.1	Procedural Learning Through Expert Acquisition	130
15.6.2	Learning Through Specification-Driven Expert Generation	130
15.6.3	Procedural Memory Instead of Parametric Memory	131
15.6.4	Self-Improvement Through Structured Interaction	131
15.6.5	A Scalable Path to Domain Expansion	132
15.6.6	A New Paradigm for Learning	132
15.7	Comparison With Monolithic Transformer Architectures	132
15.7.1	Monolithic Compression vs. Compositional Structure	132
15.7.2	Implicit Emergence vs. Explicit Mechanisms	133
15.7.3	Static Models vs. Dynamic Systems	133
15.7.4	Opaque Internal State vs. Externalized Cognitive State	133
15.7.5	Single-Threaded Generation vs. Distributed Cognition	133
15.7.6	A Bidirectional Relationship	133
15.7.7	Synthesis	134
15.8	Domain-Bound AGI as a Recipe	134
15.8.1	A Domain-Agnostic Cognitive Core	134
15.8.2	Domain Logic as Modular Ingredients	134
15.8.3	Search as a Pluggable Exploration Strategy	135
15.8.4	Memory as a Structured Cognitive Workspace	135
15.8.5	Instantiating a New Domain	135
15.8.6	A General Pattern for Constructing Intelligence	135
15.9	Learned Cognitive Mechanisms in the Reasoning Core	136
15.9.1	Canonicalization Through Prompt Rewriting	136
15.9.2	Variable Binding Through Local Prompt Maps	136
15.9.3	Flow-Based Procedural Attention	136
15.9.4	Callback Integration Through Self-Prompting	136
15.9.5	Context–Output Separation	136
15.9.6	Repeat–Prompt Recursion	136
15.9.7	Reasoning as a Controlled Recursive Process	137
15.10	Conclusion	137

16 Domain-Bound AGI and AI Agents: A Comparative Architectural Analysis	139
16.1 Why This Comparison Matters	139
16.2 What AI Agents Are	140
16.2.1 The External Agent Loop	140
16.2.2 Strengths of the Agent Approach	141
16.2.3 Limitations of the Agent Paradigm	141
16.3 What Domain-Bound AGI Is	141
16.3.1 A Domain-Agnostic Reasoning Core	142
16.3.2 Structured Memory as Cognitive State	142
16.3.3 Experts as Cognitive Subsystems	142
16.3.4 Callback-Driven Control Flow	143
16.3.5 Pluggable Search as a Depth Substrate	143
16.3.6 A Compositional Cognitive System	143
16.4 Behavioural Overlap Between AI Agents and Domain-Bound AGI	144
16.4.1 Task Decomposition and Planning	144
16.4.2 Tool and Expert Invocation	144
16.4.3 Multi-Step Reasoning and Iterative Refinement	144
16.4.4 Search and Exploration of Alternatives	144
16.4.5 Autonomous Workflow Execution	144
16.4.6 Summary	145
16.5 What AI Agents Do More (and Why It Matters)	145
16.5.1 Integration With Real-World Environments	145
16.5.2 Product-Level Interfaces and Templates	145
16.5.3 Ecosystem Maturity and Framework Support	146
16.5.4 Rapid Deployment and Practical Utility	146
16.5.5 Summary	146
16.6 What AI Agents Do Less (and Why It Matters)	146
16.6.1 Lack of Structured Cognitive State	147
16.6.2 No Internal Control Flow	147
16.6.3 Simulated Rather Than Explicit Search	147
16.6.4 Opaque Intermediate Reasoning	148
16.6.5 Limited Scalability and Fragility	148
16.6.6 No Mechanism for Self-Expansion	148
16.6.7 Summary	148
16.7 Why the Internal Approach Is Stronger	149
16.7.1 Architectural Foundations Rather Than Behavioural Facades	149
16.7.2 Stability Through Structured State	149
16.7.3 Transparent and Governable Cognition	149
16.7.4 True Depth Through Explicit Search	149
16.7.5 Scalability Through Modular Composition	149
16.7.6 Self-Expansion Without Parameter Updates	150
16.7.7 Bidirectional Enhancement of Transformer Models	150
16.7.8 Summary	150
16.8 Synthesis: Two Divergent Paths Toward Intelligent Systems	150
16.8.1 Behavioural Similarity, Architectural Divergence	150
16.8.2 External Orchestration vs. Internal Cognition	151
16.8.3 Emergent Patterns vs. Explicit Mechanisms	151
16.8.4 Shallow Depth vs. Algorithmic Search	151

16.8.5	Static Models vs. Growing Systems	151
16.8.6	Opaque Processes vs. Governable Cognition	151
16.8.7	Two Complementary but Unequal Paradigms	151
16.8.8	Summary	152
16.9	Integrating Large Language Models Inside Domain-Bound AGI	152
16.9.1	LLMs as Internal Cognitive Enhancers	152
16.9.2	A Role Distinct From External Coordination Models	153
16.9.3	Capabilities Beyond Contemporary Agent Systems	153
16.9.4	A New Purpose for Large Language Models	153
16.9.5	Summary	154
16.10	Conclusion	154
17	Toy Board: A Practical Domain-Bound AGI Example	157
17.1	Introduction: Why Toy Board?	157
17.2	The Domain-Bound AGI Problem	158
17.3	The Toy Board Architecture Overview	159
17.4	Tokenization and the Cognitive Vocabulary	160
17.5	Paging: Routing Cognitive Control	162
17.6	PRE: Propose–Refine–Evaluate for Procedural Reasoning	164
17.7	Evaluators: Coherence as the Only Learned Critic	166
17.8	Filtering and Arbitration: A Unified Decision Mechanism	168
17.9	The Script Engine: A Deterministic Interpreter for Cognitive Programs	170
17.10	Search Engines as Cognitive Tools	173
17.11	A Unified Board-Game Ontology for Heterogeneous Domains	176
17.12	Full Example: A Complete Reasoning Trace	179
17.13	Scaling Analysis: Why Toy Board Suggests a Feasible Path to AGI	181
17.14	Implications and Future Directions	186
17.15	Conclusion — A Practical Example of Domain-Bound AGI	189
18	An Ontology for AGI	193
18.1	Introduction	193
18.2	Why AGI Requires an Ontology	194
18.3	The Quad as the Structural Basis for AGI	195
18.4	The Functional Decomposition of Intelligence	196
18.5	Domain-Bound AGI as the Atomic Unit of General Intelligence	197
18.6	The Mediator-LLM as the Coordination Layer	197
18.7	The Compositional Mind: A Society of Domain-AGIs	198
18.8	Self-Improving AGI Through Structural Growth	199
18.9	Why AGI Must Strengthen the Frontier Domains	200
18.10	Toward Full-Scale AGI: A Modular, Distributed, Scientific Intelligence	201
18.11	Conclusion: An Ontology for a Compositional AGI	202
19	Cloud-Native Modular AGI Ecosystems	205
19.1	The Limits of Monolithic Intelligence	205
19.2	PRE-CAS as an Alternative to Monolithic Intelligence	206
19.3	The Cloud as a Cognitive Substrate	207
19.4	Scaling Through Composition	208
19.5	Economic and Operational Implications of Modular Scaling	208

<i>CONTENTS</i>	11
19.6 Observability and Governance in Composite Minds	209
19.7 Planet-Scale Cognitive Systems	210
19.8 Synthesis: A Cloud-Native AGI Ecosystem	211
20 A Tale About Light Bulbs and Snowballs	213

Chapter 1

A Fable of How Minds Learn to Think Together

At the far end of the research campus stood a glass dome so wide it caught the morning sun like a lens. Inside it lived a creature the researchers had once believed would change the world.

This is the story of how they discovered that intelligence is not a single mind, but a society.

1.1 The Great Parrot — A Mind Too Large to Think

Before the aviary became a bustling colony, there was only one bird.

He was enormous— a monument of feathers and neural pathways, built layer by layer in the hope that size alone would give rise to thought.

The Great Parrot could repeat anything:

- riddles,
- equations,
- lullabies,
- entire conversations.

His echoes were flawless, his memory astonishing, his presence overwhelming.

But when someone asked:

“Why did you say that?” “What should we do next?” “How would you solve this?”

... the Great Parrot only blinked, tilted his massive head, and returned a polished version of the question.

He grew larger. He grew louder. He grew more fluent. But he did not grow different.

He was a single, indivisible mind trying to imitate a world that was anything but singular.

One evening, as his echoes faded into the dome’s twilight, a researcher finally whispered:

“We can make him bigger... but we cannot make him think.”

And so they turned away from the giant and toward a new idea.

1.2 The Generative Aviary — A Thousand Voices Without Direction

The next attempt was the opposite.

Instead of one enormous bird, the researchers built many small ones— bright green, electric blue, sunset red— each no larger than a fist, each with a voice like a silver bell.

When the doors opened, the aviary erupted into sound.

The little parrots sang about:

- pirates and planets,
- talking trees,
- dragons who played chess,
- clouds that whispered secrets.

It was a storm of stories— alive, chaotic, dazzling.

But when the researchers listened closely, they noticed something strange.

The parrots were not answering questions. They were not solving problems. They were not following goals.

They were simply continuing— continuing rhythms, continuing patterns, continuing whatever melody happened to be in the air.

Their worlds were beautiful, but weightless— like soap bubbles drifting through sunlight.

Ask for a plan, and they gave a prophecy. Ask for a calculation, and they offered a riddle. Ask for a decision, and they sang a poem.

They had imagination, but not understanding.

And so the researchers realized:

“They can create worlds... but they cannot build one.”

They had swung from one extreme to the other— from a single giant mind to a thousand tiny ones.

Neither was enough.

1.3 The Colony — Where Many Small Minds Learn to Think Together

The breakthrough came quietly.

“What if intelligence,” someone asked, “is not one bird, and not a chaotic flock, but something in between?”

The next morning, the researchers returned with a new idea: not bigger parrots, not more parrots, but different parrots.

Roles Emerge

Four kinds of birds appeared:

- **Red Parrots** — impulsive proposers,

- **Blue Parrots** — careful refiners,
- **Green Parrots** — sharp-eyed evaluators,
- **Yellow Parrots** — steady arbiters.

For the first time, the aviary had structure.
Not one mind. Not chaos. But roles.

Specialists Arrive

Soon new birds joined the colony:

- **Owls** — silent searchers exploring many futures at once,
- **Finches** — tiny domain experts in numbers, rules, and transformations,
- **Magpies** — memory keepers who built nests for tasks and plans,
- **Crows** — executors who carried instructions between nests.

The aviary was no longer a flock. It was a system.

The Nests of Context

The magpies' nests changed everything.

Each nest became a context— a place where a thought could live without being lost in the noise.

A nest could store a goal, hold intermediate steps, wait for an owl's return, or spawn a sub-nest when a task grew too large.

Many thoughts could unfold at once. The aviary had discovered parallel reasoning.

The Callback Dance

When a bird finished its work, it returned to the nest and left a callback feather:

- “Here is the result.”
- “Update this part.”
- “Ask an owl to explore further.”
- “Call a finch for validation.”

The yellow parrots read the feathers. The blue parrots rewrote the plan. The crows carried new instructions.

It was a dance— a choreography of minds.

Pages of Thought

As the colony grew, the birds naturally clustered into chambers:

- language,
- logic,
- planning,
- simulation,
- symbolic manipulation.

Each chamber became a page— a local world with its own vocabulary and its own experts. The aviary had become organized, safe, and remarkably efficient.

A Mind That Can Grow

One day, the parrots faced a task none of them understood.

Instead of panicking, they built a new nest, filled it with a description of the problem, and together designed a new bird to solve it.

A bird born from specification.

The colony had learned not just to think, but to expand itself.

1.4 The Moral — Intelligence Is a Society

As the aviary hummed with activity— owls gliding through rafters, finches darting between nests, parrots debating in their chambers— the researchers finally understood.

This was not a single mind. Not a monolithic creature. Not a chaotic flock.

It was a colony.

A society of small, specialized minds whose structured interactions created something larger than any one of them.

Something coherent. Something stable. Something intelligent.

Intelligence is not a parrot. Intelligence is a society of parrots.

Chapter 2

The End of the Monolith

For much of the history of artificial intelligence, one idea has held a powerful grip on the field: the belief that if we could build a single model large enough, trained on enough data, it would eventually learn to do everything.

This idea has appeared in many forms—the “master algorithm,” the “universal learner,” the “one model to rule them all.” It has shaped research programs, corporate strategies, and public expectations.

And for a time, the evidence seemed to support it. Each new generation of large models became more fluent, more capable, more surprising. Scaling laws suggested that simply making models bigger would continue to unlock new abilities.

But as these systems grew, so did the cracks in the monolithic dream.

A large model is still a single model. It is one continuous block of computation, with no internal boundaries and no natural separation of concerns. Every parameter participates in every task. Every update touches the entire system. Every failure is global.

This is not a recipe for general intelligence. It is a recipe for brittleness.

2.1 The Limits of the Monolithic Dream

As models expand, their internal complexity expands even faster. Their memory requirements grow. Their energy consumption increases. Their behaviour becomes harder to interpret and harder to trust.

A monolithic system has no structural way to:

- isolate errors,
- specialise in domains,
- improve one capability without affecting others,
- trace how a conclusion was reached,
- or build redundancy into its reasoning.

It is a single point of failure—not just technically, but cognitively.

And as we push toward trillion-parameter systems, these limitations become more than engineering challenges. They become fundamental constraints on what such systems can safely and reliably do.

The idea of “one model that knows everything” begins to collapse under its own weight.

2.2 A Shift in Understanding

The most important insight of the last decade is not that models became larger. It is that we began to understand *why* they work—and *why* they stop working.

Intelligence does not emerge from size alone. It emerges from *structure*.

From the interaction of many processes. From the layering of representations. From the interplay between memory, prediction, evaluation, and action. From the ability to revise internal models. From the coordination of multiple perspectives.

In other words:

Intelligence is not a single thing. It is a system of things working together.

This is true in biological organisms, in human cognition, and increasingly in artificial systems.

Ecosystems do not scale by becoming one giant organism. They scale by becoming more diverse, more interconnected, and more modular.

2.3 The Modular Path to AGI

A modular architecture does not attempt to compress all knowledge and reasoning into one model. Instead, it distributes intelligence across many specialised components:

- each with its own domain,
- each with its own representations,
- each with its own strengths and limitations,
- each inspectable and replaceable.

These components communicate through shared interfaces—most naturally, through language.

Together, they form a *compositional mind*: a system in which multiple intelligences collaborate, each contributing its own perspective, without collapsing into a single undifferentiated mass.

This approach is not only more scalable. It is also more aligned with how natural intelligence works.

2.4 Why Modular Intelligence Is Safer

Safety is not an afterthought. It is a property of the architecture itself.

A modular system has:

- **boundaries** — errors remain local rather than global,
- **redundancy** — components can check and correct each other,
- **traceability** — reasoning unfolds in visible steps,
- **governance points** — modules can be audited and controlled,
- **plurality** — no single component dominates the system,
- **replaceability** — faulty modules can be swapped out,

- **evolvability** — new capabilities can be added without destabilising the whole.

These are not optional features. They are the structural foundations of safe intelligence. A monolith cannot provide them. A modular system provides them by design.

The only viable path to AGI is the safe path. And the safe path is modular.

Chapter 3

The Four Dimensions of Intelligence

If we want to understand how intelligence grows—in humans, in machines, or in any system capable of learning—we need more than metaphors. We need structure. We need the cognitive equivalent of physics: the underlying dimensions along which intelligence expands, stabilises, and transforms.

Surprisingly, these dimensions are not mysterious. They appear across biology, artificial systems, collective intelligence, and even in the evolution of scientific knowledge. They form a pattern so consistent that it behaves like a natural law.

This pattern is what we call the *Quad*.

The Quad is not a model. It is not an algorithm. It is not a theory of consciousness.

It is a structural lens: a way of seeing intelligence as a process that unfolds along four universal dimensions:

1. **Multiplicity** — the number of interacting subsystems,
2. **Richness** — the depth and variety of representations,
3. **Recursion** — the ability to refine, predict, and self-correct,
4. **Energy** — the computational throughput that drives learning.

Together, these four dimensions define the space in which intelligence can grow. They are the cognitive equivalent of mass, charge, space, and time.

3.1 Multiplicity: Intelligence Needs Many Parts

No intelligent system is made of one thing.

A human mind is not a single process. It is a constellation of subsystems: perception, memory, language, motor control, planning, evaluation, imagination. Even simple animals have dozens of interacting circuits. Even modern AI models contain layers, heads, embeddings, and internal specialists.

Multiplicity is not optional. It is the foundation of flexibility.

A system with only one cognitive process cannot:

- compare alternatives,
- detect contradictions,
- refine its own ideas,

- or adapt to new domains.

Multiplicity is the first dimension of intelligence because intelligence requires internal diversity.

3.2 Richness: Intelligence Needs Many Ways to Represent the World

A mind that can only see the world in one way cannot understand it.

Richness is the variety of internal representations a system can form: sensory patterns, symbols, abstractions, causal models, spatial maps, temporal predictions.

The richer the representational space, the more a system can:

- generalise,
- imagine,
- reason,
- and transfer knowledge across domains.

Richness is why a child can learn language, mathematics, and social norms—all with the same brain. It is why AI systems become more capable as their internal representations deepen.

Richness is the second dimension of intelligence because intelligence requires expressive models of the world.

3.3 Recursion: Intelligence Needs to Think About Its Own Thinking

Recursion is the engine of refinement.

It is the ability to reflect, revise, predict, simulate, and improve one's own outputs. Recursion is what allows a scientist to refine a theory, a writer to edit a sentence, or an AI system to improve a plan.

Without recursion, intelligence is static. With recursion, intelligence becomes self-correcting.

Recursion is the third dimension of intelligence because intelligence requires iterative improvement.

3.4 Energy: Intelligence Needs Throughput

Energy is the least glamorous dimension, but the most fundamental.

Every thought, prediction, and refinement requires computation, time, memory, and physical energy. A system with insufficient energy cannot:

- explore large search spaces,
- maintain deep representations,
- or run long chains of reasoning.

Energy is the fourth dimension of intelligence because intelligence requires fuel.

3.5 The Quad as a Structural Law

These four dimensions are not arbitrary. They appear in every intelligent system we know: in evolution, in neuroscience, in collective intelligence, in machine learning, and in distributed systems.

They explain why intelligence grows, why it stabilises, and why it sometimes collapses.

Most importantly, they explain why monolithic AI cannot scale.

A monolith cannot increase multiplicity. It cannot diversify representations. It cannot deepen recursion without becoming unstable. And it cannot scale energy without hitting physical limits.

The Quad shows that intelligence grows not by becoming one giant thing, but by becoming many things working together.

3.6 The Quad as the Foundation of AGI

The rest of this part of the book builds on the Quad.

- The functional architecture shows how these dimensions manifest as cognitive roles.
- The modular architecture shows how these roles can be distributed across many specialised components.
- The cloud-native realisation shows how these components can be orchestrated at scale.
- The dataset scaling law shows why systems converge as they learn.
- The ASI horizon shows what happens when these dimensions approach their limits.

The Quad is the unifying structure behind all of it. It is the geometry of intelligence—the space in which minds grow.

Chapter 4

Biological Intelligence as a Quad

Human intelligence is a biological implementation of the Quad. While the details of neural architecture are complex, the underlying structure aligns closely with the four dimensions that govern all intelligent systems: multiplicity, richness, recursion, and energy.

4.1 Multiplicity: Biological Subsystems

The human brain is not a single process. It is a vast collection of interacting subsystems: perceptual circuits, memory systems, linguistic pathways, motor controllers, evaluative networks, and many others. Billions of neurons form specialised modules that cooperate and compete.

Multiplicity is what allows humans to compare alternatives, detect inconsistencies, and adapt to new situations. Without internal diversity, flexible intelligence would not be possible.

4.2 Richness: Representational Depth

Human cognition supports a remarkable variety of representations: layered sensory maps, linguistic structures, conceptual abstractions, causal models, spatial navigational grids, and temporal predictions.

These representations compress experience into reusable forms. They allow humans to generalise from limited data, imagine hypothetical scenarios, and transfer knowledge across domains.

4.3 Recursion: Thinking About Thinking

Humans possess unusually deep recursion. We reflect on our thoughts, revise our plans, construct narratives, and reason about our own reasoning. This metacognitive capacity enables long chains of refinement and self-correction.

Recursion is what allows scientific theories to evolve, stories to be edited, and strategies to be improved.

4.4 Energy: Extreme Efficiency

The human brain operates on roughly twenty watts of power. This constraint shaped the evolution of human intelligence: biological systems learned to maximise cognitive output per unit of energy.

Energy efficiency is not a limitation. It is a feature that guided the architecture of the mind.

4.5 Summary

Humans are a high-recursion, high-richness, energy-optimised Quad. Biological intelligence is not mysterious; it is a structural realisation of the same dimensions that govern all minds.

Chapter 5

Artificial Intelligence as a Computational Quad

Artificial intelligence is a computational implementation of the Quad. Although the mechanisms differ from biology, the structural dimensions are the same.

5.1 Multiplicity: Computational Subsystems

Modern AI systems contain layers, attention heads, memory modules, and specialised components. These form the computational equivalent of biological subsystems. Multiplicity enables parallel processing, pattern diversity, and internal cross-checking.

5.2 Richness: High-Dimensional Representations

AI systems learn embeddings, tensors, and latent spaces that capture patterns across many levels of abstraction. Representational richness grows with model depth, dataset diversity, and training duration.

5.3 Recursion: Inference and Self-Refinement

AI systems increasingly rely on recursive inference loops: multi-step reasoning, planning, self-evaluation, and iterative refinement. These loops deepen the system's ability to correct its own outputs and sustain longer chains of thought.

5.4 Energy: Compute and Power

Artificial systems scale energy through hardware: FLOPs, memory bandwidth, and specialised accelerators. This allows rapid expansion of recursion and representational richness, but also introduces physical and economic limits.

5.5 Summary

AI is a high-multiplicity, high-richness, high-energy Quad with rapidly increasing recursion. It is a structural cousin of biological intelligence, not an alien form.

Chapter 6

Hybrid Intelligence as a Distributed Quad

Hybrid intelligence combines human cognition with artificial systems, forming a distributed Quad that spans biological and computational components.

6.1 Multiplicity: Heterogeneous Subsystems

Hybrid systems integrate human reasoning, machine models, tools, and external artefacts. Together, these form a distributed cognitive architecture with far greater subsystem diversity than either component alone.

6.2 Richness: Shared Representational Capacity

Human meaning, symbolic workspaces, and machine-native embeddings combine to create a richer representational space. The parameter I reflects the total expressive power of this combined system.

6.3 Recursion: The Interaction Loop

Hybrid intelligence operates through a recursive cycle:

propose $\rightarrow$ structure $\rightarrow$ evaluate $\rightarrow$ refine $\rightarrow$ integrate.

Each iteration deepens recursion, enabling long chains of reasoning that neither humans nor machines could sustain alone.

6.4 Energy: Human Effort and Machine Compute

Energy is allocated where it is most effective: humans provide semantic insight and judgement; machines provide large-scale computation. The result is a system that produces more cognitive output per unit of energy than either component alone.

6.5 Summary

Hybrid intelligence is the highest-recursion architecture available today. It is a distributed Quad that leverages the strengths of both biological and artificial minds.

Architectural Insight

A structural model of intelligence reveals the four irreducible capacities from which every form of mind can emerge.

Chapter 7

A Historic Vision on AGI: Minsky and the Society of Mind

Marvin Minsky’s *Society of Mind* remains one of the most influential architectural visions of intelligence. Its central claim is deceptively simple: intelligence is not a single, unified process, but a society of interacting parts. Each part—each “agent”—is small, specialized, and limited. None is intelligent on its own. Yet through structured interaction, a mind emerges.

Three architectural principles define this view:

- **Decomposition** — intelligence arises from many small processes, not one large one.
- **Specialization** — each process performs a distinct cognitive role.
- **Coordination** — intelligence emerges from structured interaction between roles.

This perspective is strikingly modern. It anticipates modular AI, hybrid systems, and the separation of roles that underlies today’s most robust architectures. It also anticipates the limits of monolithic systems: scaling a single mechanism is not enough; structure matters.

Hybrid Intelligence extends this lineage into the human–machine domain. Instead of one model attempting to do everything, cognition is distributed across complementary components: human intuition, machine precision, symbolic scaffolds, evaluators, refiners, and memory systems. It is a contemporary form of Minsky’s decomposition—explicit roles, explicit communication, explicit coordination.

Both perspectives converge on the same architectural insight:

Intelligence is modular. It grows not from scale, but from structure. Not from a single mapping, but from recursive interaction between specialized roles.

This book develops a computational path grounded in this shared lineage. The goal is not to recreate Minsky’s theory, nor to idealize human–machine hybrids, but to operationalize their core principles in modern, reproducible systems—ranging from architectural roles and evaluators to the PRE–CAS–CE engine and the Toy Story implementation. This part of the book translates a historic vision into a concrete, architectural path toward AGI.

One-Line Takeaway

Minsky’s core insight endures: intelligence emerges from structured interaction between specialized roles.

Notes and Further Reading

Minsky’s *Society of Mind* (1986) remains a foundational text for modular theories of intelligence. Its agent-based perspective anticipates modern evaluator-refiner architectures, hybrid intelligence systems, and the modular decomposition used throughout this book.

Chapter 8

What Are Neural Networks?

Neural networks form the computational substrate of modern machine learning, and within this book they serve as the primary *intelligence layer* underlying many of the toy universes and experimental systems that illustrate the Triadic Cosmos. Whenever a toy world learns, adapts, classifies, predicts, or transforms information, it does so through the dynamics of a neural network. Understanding these networks is therefore essential for understanding how the AGI prototypes in this book operate internally.

From the perspective of the Triad–Quad–Septad ontology, neural networks occupy a precise conceptual role. They instantiate the *computational substrate* (Triad: **Substrate**), they implement the *transformational dynamics* that define cognitive processes (Quad: **Dynamics**), and they support the emergence of structured behaviour across multiple scales (Septad: **Emergence**). Neural networks are thus not merely engineering tools; they are concrete realisations of the ontological principles developed earlier.

Despite their widespread use, the internal mechanisms of neural networks are often treated as opaque or mysterious. In this chapter, we take a clear and structured view of what neural networks are, how they operate, and why *activation functions* play a central role in shaping their behaviour. This is particularly important in the context of the AGI book, because many of the toy universes rely on different activation regimes to produce distinct forms of behaviour.

8.1 Neurons and Layers

A neural network is composed of simple computational units called *neurons*. Each neuron receives an input vector $x \in \mathbb{R}^n$, computes a weighted sum, adds a bias, and applies a nonlinear transformation:

$$y = \phi(Wx + b),$$

where W is a weight matrix, b is a bias vector, and ϕ is an activation function.

A single neuron is limited in expressive power, but when many neurons are arranged into layers, and layers are stacked into deeper architectures, the resulting network can approximate highly complex functions. This is the essence of the *universal approximation theorem*: a sufficiently large neural network can approximate any continuous function on a compact domain.

In the Triadic Cosmos, this layered structure corresponds to the *hierarchical organisation of transformations* (Quad: **Composition**) and the emergence of higher-level behaviour from simple local rules (Septad: **Emergence**).

8.2 The Role of Activation Functions

The activation function ϕ is the only source of nonlinearity in a neural network. Without it, the entire network collapses into a single linear transformation, regardless of depth:

$$\phi(x) = x \quad \Rightarrow \quad \phi(W_2\phi(W_1x)) = W_2W_1x.$$

Thus, activation functions are not a minor detail—they are the *structural backbone* that enables neural networks to model nonlinear relationships.

Activation functions determine:

- the geometry of the network’s internal representations,
- the flow of gradients during training,
- the stability of optimisation,
- the expressiveness of each layer,
- and the inductive biases of the entire architecture.

In the AGI book, these properties are essential because the toy universes introduced throughout the chapters rely on different activation regimes. Some toys require sharp, piecewise-linear transitions; others require smooth, expressive nonlinearities; still others benefit from saturating or periodic behaviours. The diversity of activation functions mirrors the diversity of cognitive dynamics across the Triad, Quad, and Septad.

8.3 Common Activation Functions

Several activation functions have become standard in deep learning. Each has distinct properties and inductive biases, and many of them appear throughout the toy systems in this book.

8.3.1 ReLU

$$\text{ReLU}(x) = \max(0, x).$$

Simple, efficient, and widely used, but prone to “dying” neurons when gradients vanish.

8.3.2 Leaky ReLU

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases}$$

with a small negative slope to avoid inactive units.

8.3.3 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Historically important, but prone to saturation and vanishing gradients.

8.3.4 Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Zero-centered and smoother than sigmoid.

8.3.5 GELU

$$\text{GELU}(x) = x\Phi(x),$$

where Φ is the standard normal CDF. Widely used in transformer architectures.

8.3.6 SiLU / Swish

$$\text{SiLU}(x) = x \cdot \sigma(x).$$

Smooth, expressive, and effective across many tasks.

8.4 Why Activation Functions Matter in AGI Toys

Throughout the AGI book, neural networks appear in many forms: as classifiers, as dynamical systems, as agents, as world models, and as components of miniature universes. In each case, the choice of activation function shapes the behaviour of the system.

For example:

- toy reasoning systems often rely on piecewise-linear activations to create sharp decision boundaries,
- toy physics worlds benefit from smooth activations that support continuous dynamics,
- symbolic-numeric hybrids may require saturating functions to stabilise discrete transitions.

Activation functions thus play a central role in determining how a toy universe behaves, learns, and generalises.

Summary

Neural networks are powerful function approximators, but their behaviour is fundamentally shaped by their activation functions. Fixed activations impose specific inductive biases that influence expressiveness, stability, and learning dynamics. Because many toy universes in this book rely on neural networks, understanding these activation functions is essential for understanding the behaviour of the AGI prototypes built on top of them.

The next toy chapters explores how these networks are trained and how their internal representations evolve during learning.

Chapter 9

Toy Architect: Reasoning using PRE

9.1 Introduction — A Tiny Universe of Thought

Toy Architect is the first toy universe in the book and explores reasoning.

It is a symbolic world with:

- a fixed vocabulary of 111 tokens,
- a neural proposer,
- a symbolic evaluator,
- a shared workspace,
- and a recursive refinement loop.

This universe contains no geometry, no forces, no organisms. It contains only symbols, structure, and recursion.

Despite its minimalism, the Toy Architect is rich enough to reveal the physics of reasoning and the architecture of hybrid intelligence.

9.2 A Symbolic World — The Language of Thought

The Toy Architect lives in a machine-native universe built from exactly 111 tokens:

- numbers (0–100),
- operators (PLUS, MINUS),
- control markers (PROPOSAL, EVALUATION, END),
- feedback signals (COLD, WARM, HOT, CORRECT),
- and an EMPTY token.

This vocabulary is not a compressed natural language. It is a symbolic physics: a discrete set of elements that define what can be expressed, judged, and refined.

Every thought the architect produces is a sequence drawn from this alphabet.

9.3 The Architect — A Minimal Reasoning Organism

The Toy Architect is a tiny neural organism:

- a two-layer GELU network,
- trained on next-token prediction,
- operating over one-hot vectors,
- generating symbolic hypotheses token by token.

Its life consists of three activities:

1. reading a number series,
2. proposing a hidden architecture,
3. refining that proposal under feedback.

Its memory is the workspace. Its perception is the sliding window. Its action is the next token.

Even with only 111 symbols and a small neural body, the architect is a complete reasoning agent.

9.4 The Evaluator — Oracle of the Universe

The architect does not reason alone. A second inhabitant lives in this world: the Evaluator.

The evaluator knows the hidden structure that generated the number series and judges the architect's proposals using four signals:

- COLD — wrong structure,
- WARM — correct operators, wrong operands,
- HOT — operands close,
- CORRECT — full match.

These signals are the discrete analogue of gradients. They guide the architect toward the correct structure through recursive refinement.

The evaluator is not a critic. It is a teacher.

9.5 The Reasoning Game — Propose, Evaluate, Refine

Each number series is a micro-cosmos generated by a hidden architecture: a sequence of operator–operand pairs applied repeatedly.

The Toy Architect's task is to infer this structure.

The universe unfolds as a loop:

1. The world presents a number series.
2. The architect proposes a structure.

3. The evaluator judges the proposal.
4. The architect refines its hypothesis.
5. The cycle repeats until convergence.

This loop is the operational form of hybrid intelligence: a dialogue between a neural proposer and a symbolic evaluator.

9.6 Cognitive Geometry — The Sliding Window

The architect does not see the entire workspace at once. It perceives a structured context of 37 slots:

- the number series,
- the previous proposal,
- the previous evaluation,
- the current proposal.

Each slot holds a single token. Each token becomes a one-hot vector. The concatenation of these vectors becomes the architect’s perceptual field.

This sliding window is the geometry of the architect’s mind: a fixed cognitive space in which symbolic structures are arranged, interpreted, and refined.

9.7 Developmental Physics — Curriculum and Training

The architect learns through a two-phase curriculum:

- Proposal training — learning to express correct structures through teacher forcing.
- Feedback training — learning to refine proposals under evaluator feedback.

These two loops mirror the developmental physics of reasoning:

- learning through demonstration,
- learning through interaction,
- learning through refinement.

The architect does not memorize patterns. It internalizes the structure of reasoning.

9.8 Hybrid Intelligence in Miniature

The Toy Architect is the smallest universe in which the Hybrid Intelligence Template becomes visible:

- a symbolic language,
- a neural proposer,
- a symbolic evaluator,
- a recursive refinement loop,
- and a shared workspace.

It is a two-mind system with complementary roles. It is a distributed cognitive architecture. It is hybrid intelligence in miniature.

This is the same pattern that shaped the writing of this book: a recursive dialogue between human intuition and machine reasoning.

9.9 The Septad — The Architecture of Architectures

The Septad appears naturally inside this tiny world:

- Identity — the proposer’s perspective,
- Constraint — the 111-token language,
- Symmetry — the repeated operator–operand structure,
- Correspondence — evaluator feedback,
- Integration — refinement of partial hypotheses,
- Stability — the workspace,
- Continuity — the recursive loop.

Even in this minimal universe, the Septad appears in full.

9.10 Why This Tiny Universe Matters

The Toy Architect is more than a demonstration. It is a microcosm of reasoning itself.

It reveals:

- how structure emerges from recursion,
- how feedback shapes cognition,
- how symbolic and neural components complement each other,
- and how hybrid intelligence can discover architectures in worlds large or small.

It closes the loop that opened the book: if the cosmos behaves like a language model, then a tiny language model can behave like a cosmos.

9.11 Iterative Pattern Discovery

The Toy Architect is a minimal PRE-style architecture that attempts to infer the rule underlying a number series. Given an input sequence, a *proposer* (the architect) generates candidate architectures (e.g., MINUS 8, PLUS 4, MINUS 4), while an *evaluator* returns coarse feedback signals such as COLD, WARM, HOT, or CORRECT. The system iterates until it either converges on a correct architecture or reaches a maximum of 100 iterations.

We trained two stored models for the proposer: one with 100k epochs and one with 500k epochs. Each model was evaluated on 1000 independent test series.

Aggregate Performance

Model	Successes (of 1000)	Average Attempts	Median Attempts
100k epochs	89	41.6	41
500k epochs	657	25.4	17

The 500k-epoch model is substantially stronger: it solves far more series within the 100-step budget and typically converges in fewer iterations. However, the gap between 657/1000 and full coverage indicates that there is still significant room for improvement with further training or architectural refinements.

Full Reasoning Dialogue

The dialogue below shows the complete reasoning trace for one example series. Each step contains the proposer’s hypothesis and the evaluator’s feedback.

Full Reasoning Example

Target Architecture: MINUS 8, PLUS 4, MINUS 4

Problem: 34;26;30;26;18;22;18;10;14;10;2;6;2;END

Architect [1]:

PROPOSAL; MINUS; 8; PLUS; 1; PLUS; 1; END

Evaluator [1]:

EVALUATION; COLD; END

Architect [2]:

PROPOSAL; MINUS; 9; PLUS; 3; MINUS; 1; END

Evaluator [2]:

EVALUATION; WARM; END

Architect [3]:

PROPOSAL; MINUS; 8; PLUS; 4; MINUS; 5; END

Evaluator [3]:

EVALUATION; HOT; END

Architect [4]:

PROPOSAL; MINUS; 8; PLUS; 4; MINUS; 4; END

Evaluator [4]:

EVALUATION; CORRECT; END

This example illustrates the full progression:

- **COLD** — wrong structure.
- **WARM** — correct operators, wrong operands.
- **HOT** — operands close to correct.
- **CORRECT** — full convergence.

The Toy Architect thus provides a simple, fully observable instance of PRE-like iterative refinement: a proposer explores a discrete hypothesis space, guided only by coarse evaluator feedback, and gradually converges on a compact architecture that explains the observed series.

9.12 Hybrid Intelligence in Miniature

Toy Architect is the smallest universe in this book where reasoning becomes visible. It is not a game, not an evolutionary system, and not a physics world. It is a thinking system—a tiny laboratory where symbols, neural proposals, and evaluative judgment interact inside a shared workspace. In this universe, intelligence does not emerge from movement or survival. It emerges from structure.

Toy Architect shows that when a system has a symbolic language, a proposer that generates ideas, an evaluator that judges them, and a recursive loop that refines them, reasoning begins to appear. It is the minimal architecture of domain-bounded AGI, expressed in its simplest possible form.

The Smallest Domain-Bounded AGI

Toy Architect contains all the essential components of a reasoning mind:

- a symbolic language — a compact vocabulary of 111 tokens that defines the universe of discourse,
- a neural proposer — a small model that generates candidate solutions or continuations,
- a symbolic evaluator — a rule-based judge that checks correctness, structure, and coherence,
- a recursive refinement loop — proposals are improved through repeated cycles of evaluation and correction,
- a shared workspace — a memory structure where intermediate reasoning steps accumulate and stabilize.

These elements form a complete cognitive loop. The system proposes, evaluates, refines, and integrates ideas until a stable solution emerges. Nothing is hidden. Every step is inspectable. Every refinement is explicit. Toy Architect is the smallest runnable universe where reasoning is not simulated—it is enacted.

Link to the Hybrid Intelligence Template

Toy Architect is the first fully executable instance of the Hybrid Intelligence Template. The template’s core cycle—Propose → Structure → Evaluate → Refine → Integrate—is implemented directly in this universe.

- multiple proposers can be plugged in: neural models, heuristics, or symbolic generators,
- a generalist evaluator judges proposals with broad structural awareness,
- a recursive loop drives improvement through repeated cycles,
- a shared workspace acts as the system’s memory and integration layer,
- domain-bounded AGI emerges inside a symbolic universe with clear rules and transparent reasoning.

Toy Architect is therefore not an analogy for the Hybrid Intelligence Template. It *is* the template, distilled to its essence. It shows how a small system, with the right architecture, can perform structured reasoning without requiring scale, massive datasets, or opaque internal states.

Link to the Path Toward Safe AGI

Toy Architect also demonstrates the architectural principles that underpin safe AGI:

- transparent reasoning — every step is visible, inspectable, and auditable,
- modularity — proposers, evaluators, and workspaces are separate components,
- inspectable loops — recursion is controlled, bounded, and easy to trace,
- safe recursion — the system improves through refinement, not uncontrolled self-modification.

This is the architectural foundation of safe AGI: a system that is powerful because it is structured, not because it is large; intelligent because it refines, not because it improvises; safe because its reasoning is explicit, not hidden.

9.13 From Toy Universes to Reasoning Architectures

MazeRunner, Pentadia, and Octadia—show how behavior, ecology, and strategy emerge from simple rules. Each world is physical in its own way: agents move, interact, and compete inside a deterministic environment.

The Toy Architect marks a shift. It was built as a toy demonstration of Hybrid Intelligence long before the architecture was formalized. A PRE network is the machine-learning instantiation of the same pattern. The reason PRE converges is simple: the Toy Architect had already shown that the physics of hybrid intelligence work, even in the smallest possible universe.

It is the first universe in this book where the physics are symbolic. Instead of forces or movement, the world contains:

- a fixed vocabulary of 111 tokens,
- a neural proposer,

- a symbolic evaluator,
- a shared workspace,
- and a recursive refinement loop.

This tiny universe is more than a demonstration. It is the smallest possible implementation of a reasoning architecture.

A miniature LLM

The Toy Architect is a complete language model in miniature:

- one-hot encoding,
- softmax decoding,
- a sliding context window,
- autoregressive next-token prediction,
- teacher forcing,
- refinement under feedback,
- persistent memory.

Its entire mind fits in a few hundred lines of code. It is a transparent LLM—small enough to inspect, yet rich enough to reason.

A minimal PRE Network

The Toy Architect is also the minimal instantiation of a PRE Network, the triadic architecture of reasoning:

- P — Proposer,
- R — Refiner,
- E — Evaluator.

Architectural Insight

Reasoning begins when a mind learns to refine its own thoughts.

Chapter 10

The PRE Family

10.1 Introduction

Intelligence does not arise from a single mapping. It emerges from a cycle: a system proposes an idea, evaluates it, improves it, and evaluates again. This recursive engine is the foundation of reasoning, problem solving, and adaptive behavior. In this chapter, that cycle becomes a computational architecture.

A Propose–Refine–Evaluate Network (PRE-Network) is the smallest artificial system that implements this cognitive loop. Instead of one monolithic model, a PRE-Network is a modular society of minds: a proposer that generates hypotheses, an evaluator that judges them, and a refiner that improves them. The system does not map—it solves. It does not guess—it reasons.

PRE-Networks form the foundation of the PRE-CASP family introduced later in this book. They are the atomic unit of recursive reasoning: simple, transparent, and computationally grounded.

10.2 Limitations of Classical Models

Traditional neural networks learn a single function:

$$f(x) \rightarrow y.$$

They produce one answer, with no self-evaluation, no refinement, and no recursive correction. This works for smooth, injective mappings, but is less effective for the very problems where intelligence is required:

- inverse functions,
- multi-valued mappings,
- discontinuities,
- asymmetric landscapes,
- problems with plateaus or multiple roots.

A monolithic model can memorize a mapping, but it cannot reason about its own output. It has no internal notion of correctness, no mechanism for improvement, and no way to regulate its own behavior. PRE-Networks exist because classical models cannot cross this threshold.

10.3 The Three Roles

A PRE-Network consists of three specialized components.

Proposer (P)

Generates an initial estimate. Its task is simple: be reasonable, not perfect. It is the analogue of intuitive thinking.

Evaluator (E)

Estimates the quality of the proposer’s output. It does not compute the true error. Instead, it learns a machine-native internal error representation—a geometry of correctness shaped by the proposer’s typical mistakes.

Refiner (R)

Uses the evaluator’s signal to produce a better estimate. Learns a correction step, not the full solution. It is the analogue of deliberate reasoning.

Together, these three roles form a minimal society of minds: one proposes, one judges, one improves.

10.4 The PRE Cycle — A Four-Step Engine of Reasoning

A PRE-Network follows a fixed, deterministic cognitive loop:

1. **Propose** — generate an initial guess.
2. **Evaluate** — estimate the quality of that guess.
3. **Refine** — improve the guess using the evaluation.
4. **Re-Evaluate** — verify whether the refinement helped.

This cycle is:

- stable,
- domain-independent,
- pedagogically simple,
- computationally efficient,
- and architecturally clean.

It is the smallest operational form of recursive reasoning: intuition $\rightarrow$ evaluation $\rightarrow$ correction $\rightarrow$ meta-evaluation. PRE-Networks are therefore not just models; they are cognitive architectures.

The cycle may run for a fixed number of iterations or adaptively, with the evaluator deciding whether refinement should continue. For clarity and reproducibility, fixed-iteration PRE is used in this chapter, but the architecture supports both.

10.5 PRE as an Architectural Pattern

PRE is not a single model but an architectural pattern. Any system that implements the PRE-cycle—whether with three separate networks or a single multi-headed model—belongs to the PRE family.

This pattern generalizes across:

- domains (inverse functions, classification, structured prediction),
- substrates (MLPs, CNNs),
- refinement regimes (absolute vs. delta),
- iteration depths (one-shot vs. multi-step),
- and scales (toy systems vs. multi-expert colonies).

10.6 Architectural Principles of the PRE Family

PRE-Networks are built on four architectural principles.

1. Decomposition

The system is decomposed into specialized cognitive roles. This avoids the opaque internal dynamics of monolithic models.

2. Recursion

The PRE-cycle is a recursive engine of improvement. The system regulates itself through repeated evaluation and refinement.

3. Internal Communication

All interaction occurs through explicit, inspectable signals: proposals, evaluations, and refinements. There are no hidden heuristics.

4. Bounded Reasoning

Recursion is bounded by design. The system cannot diverge or enter uncontrolled loops.

These principles make PRE-Networks transparent, modular, and safe by construction.

10.7 PRE as the Foundation of Modular Intelligence

PRE-Networks demonstrate that:

- intelligence is cyclic,
- specialization beats brute force,
- internal languages emerge naturally,
- recursion stabilizes reasoning,

- and safety can be architectural.

They are the smallest artificial systems in which these principles become visible. The next part of this chapter develops the internal semantics of PRE-Networks and the refinement regimes that make them effective recursive solvers.

10.8 Internal Error Languages — Machine-Native Semantics

A defining property of PRE-Networks is that they develop an internal error language. The evaluator does not approximate the true domain error,

$$|x_{\text{pred}} - x_{\text{true}}|,$$

but instead learns a proposer-conditioned error geometry that is:

- nonlinear,
- asymmetric,
- domain-compressed,
- less interpretable by humans,
- yet internally stable and consistent.

This geometry emerges from co-adaptation:

- the proposer learns to generate guesses that fall within the evaluator’s representational regime,
- the evaluator learns a machine-native notion of correctness,
- the refiner learns to interpret and act upon this internal signal.

The result is a compact semantic space that does not resemble the true error in scale, shape, or distribution, yet is fully functional for recursive refinement. PRE-Networks therefore possess an epistemic layer that monolithic models fundamentally lack.

10.9 Two Refinement Regimes — Absolute and Delta

PRE-Networks support two refinement regimes, each suited to different problem structures.

Absolute Refinement

The refiner predicts a new solution:

$$x_{\text{new}} = R(x).$$

This regime excels at:

- smooth inverse functions,
- single-valued mappings,
- domains with clear global structure.

Delta Refinement

The refiner predicts a correction:

$$x_{\text{new}} = x + \Delta x.$$

This regime behaves like a learned numerical solver and excels at:

- multi-valued inverse problems,
- matrix-inverse-like tasks,
- chaotic or poorly conditioned domains.

Delta refinement produces behavior reminiscent of:

- Newton-type updates,
- fixed-point iteration,
- line search,
- dynamical systems.

Empirically, delta refinement is often superior because it isolates the correction step from the global structure of the domain.

10.10 Specialized Evaluator Architectures

Because the evaluator learns a machine-native geometry, its role can be specialized without changing the PRE-cycle. This yields three practical variants.

PREa — Accuracy-Driven PRE Networks

A PREa-Network uses an evaluator trained to estimate accuracy:

- logical consistency,
- numerical stability,
- structural coherence.

The evaluator learns an internal geometry of correctness, and the refiner learns to move proposals toward this geometry.

PREs — Safety-Driven PRE Networks

A PREs-Network replaces the accuracy evaluator with a safety evaluator:

- harmfulness detection,
- constraint enforcement,
- ethical gating,
- rejection of unsafe proposals.

The evaluator learns a machine-native safety geometry, and the refiner becomes an alignment corrector.

PREas — Dual-Evaluation PRE Networks

The PREas-Network uses two independent evaluators:

- E_a : accuracy evaluator,
- E_s : safety evaluator.

The cycle becomes:

1. propose,
2. refine,
3. evaluate accuracy,
4. evaluate safety,
5. refine again,
6. repeat until both evaluators agree,
7. abort if alignment cannot be reached.

This dual-evaluation architecture separates accuracy from safety, prevents collusion between roles, and embeds alignment directly into the computational structure.

10.11 Architectural Variants — COMPOSITE and MONO

PRE-Networks can be instantiated in two architectural forms. Both implement the same cognitive cycle, develop internal error languages, and support recursive refinement. The difference lies in how much specialization the architecture allows.

COMPOSITE PRE-Networks

The COMPOSITE variant uses three separate models:

- each with its own architecture,
- its own activations,
- its own depth and width,
- its own training dynamics.

This version expresses the modular structure of PRE most clearly: a true decomposition into specialized minds. The proposer, evaluator, and refiner can diverge architecturally, allowing each to optimize for its cognitive role.

MONO PRE-Networks

The MONO variant uses a single model with:

- a one-hot PRE-token indicating the active role,
- additional inputs for the proposal and evaluation,
- three output heads (P, E, R).

This version is compact and elegant. It shows that PRE-Networks require only a small extension of a standard MLP: a role indicator, a shared body, and three heads. Despite its simplicity, the MONO architecture still develops internal error languages and supports recursive refinement.

Comparison

Both variants:

- follow the same PRE-cycle,
- exhibit mixture-of-experts dynamics,
- develop internal error geometries,
- and outperform classical monolithic models.

COMPOSITE offers full specialization; MONO offers compactness. Both demonstrate that PRE is an architectural pattern, not a specific model.

10.12 The PRE Architectural Space

The PRE-cycle defines a three-dimensional architectural space:

- **Decomposition** — MONO vs. COMPOSITE models,
- **Refinement Type** — ABSOLUTE vs. DELTA,
- **Iteration Depth** — $N = 1, 2, 3, \dots$

This space contains a wide range of architectures:

- PRE-ABS-1 — light, fast, mapping-like,
- PRE-DELTA-1 — a basic solver,
- PRE-DELTA-2 — a stronger recursive solver,
- PRE-MONO-ABS-1 — a compact system.

The PRE family is therefore not a single model but a taxonomy of recursive solver architectures, unified by a common cognitive loop.

10.13 Lightweight Implementation and Universal Training

PRE-Networks are surprisingly lightweight:

- the full implementation fits in a few hundred lines of code,
- the architecture requires no exotic layers,
- the PRE-cycle is simple and deterministic,
- the evaluator and refiner are small networks with simple tasks.

Despite this simplicity, PRE-Networks exhibit:

- emergent internal languages,
- solver-like behavior,
- recursive improvement,
- mixture-of-experts dynamics,
- stability across refinement regimes.

A crucial property is that PRE-Networks are trained once. You do not train a “PRE-1” or “PRE-2.” You train a PRE-Network, and at inference time you choose:

- 1 refinement,
- 2 refinements,
- or more.

The architecture generalizes across refinement depth. This is a hallmark of recursive systems: the step function is learned, not the number of steps.

10.14 Synthesis — PRE-Networks as Minimal Universes of Reasoning

PRE-Networks demonstrate that recursive reasoning can be implemented through explicit, inspectable roles. Their architecture embodies five principles:

- **Cyclic reasoning** — intelligence emerges from repeated propose–evaluate–refine loops.
- **Specialization** — different cognitive roles require different inductive biases.
- **Internal semantics** — evaluators develop machine-native correctness geometries.
- **Bounded recursion** — reasoning depth is explicit and controllable.
- **Transparent computation** — every step is inspectable and traceable.

They are not introduced as industrial alternatives to large-scale architectures, but as the simplest systems in which the book's central ideas become empirically visible. PRE-Networks are pedagogical universes: small enough to understand, rich enough to exhibit emergence, and precise enough to reveal the hidden structure of intelligent computation.

Architectural Insight

A PRE-Network is the smallest complete architecture in which recursive reasoning becomes observable.

Chapter 11

CAS — The Cognitive Vector Layer

11.1 Introduction — Why Intelligent Systems Need Cognitive Vectors

Modern AI systems are powerful pattern recognizers, but they remain surprisingly limited in one respect: they do not know *what kind* of output they are producing. A classical model can generate text, predict values, or classify inputs, yet it cannot express:

- whether the output is safe or unsafe,
- whether the information is accurate or unreliable,
- whether the content is relevant or off-topic,
- which style, tone, or personality it is following,
- or how the output should be interpreted by downstream systems.

These dimensions matter. They determine whether a system is trustworthy, steerable, and aligned with human expectations. Without them, models must be corrected *after* generation—through filters, heuristics, or external alignment layers—rather than learning these distinctions as part of their internal structure.

CAS introduces a different approach. Instead of treating safety, accuracy, relevance, and style as afterthoughts, CAS represents them as a *cognitive vector*: a structured signal that is learned during training and used during generation. CAS is not a prompt trick, not a post-processing filter, and not a probabilistic confidence estimate. It is an architectural layer that gives a model the ability to express what kind of output it is producing.

In its simplest form, CAS was a triad: *Content–Accuracy–Safety*. In its modern form, CAS is a general vector space of cognitive dimensions. This chapter introduces that evolution and shows how CAS provides a unified interface for training on mixed datasets, guiding generation, and embedding safety directly into the model’s internal representations.

11.2 From Triadic Epistemics to CAS(v)

CAS began as a minimal epistemic interface. A model would output three values:

- **content** — the predicted value or token,

- **accuracy** — how certain the model is,
- **safety** — how stable or trustworthy the situation appears.

This triad was the smallest structure capable of expressing uncertainty, detecting unstable regimes, and supporting safe decision-making. It allowed multiple models to form consensus, disagree, or abort automatically when the situation became epistemically unstable.

But as the architecture evolved, something became clear: the triad was only one instance of a much broader idea.

CAS as a General Cognitive Vector

In the modern architecture, CAS is no longer restricted to three dimensions. It is a vector of length v :

$$\text{CAS}(v) = (c_1, c_2, \dots, c_v),$$

where each dimension represents a cognitive property that the model learns to predict and control.

Examples include:

- **safety**: safe/unsafe content,
- **accuracy**: reliable/unreliable information,
- **relevance**: on-topic/off-topic for a domain,
- **style**: formal/informal, poetic/technical,
- **personality**: concise/verbose, warm/dry, humorous/neutral,
- **domain markers**: mathematics, storytelling, science, law.

The original triad becomes a *special case*:

$$\text{CAS}(3) = (\text{content}, \text{accuracy}, \text{safety}).$$

But $\text{CAS}(v)$ is far more general. It is a cognitive interface that allows a model to:

- learn from mixed or contaminated datasets,
- distinguish safe from unsafe material,
- separate accurate from noisy data,
- encode stylistic and personality preferences,
- and generate outputs conditioned on a target CAS vector.

Why CAS Matters

CAS makes it possible to train on real-world data—messy, diverse, inconsistent—without losing control over the model’s behavior. Instead of filtering datasets or relying on brittle prompt engineering, CAS teaches the model to *understand* the cognitive properties of its own outputs.

During generation, a target CAS vector acts as a structural guide:

“Produce safe, accurate, relevant content in a concise and technical style.”

CAS becomes an architectural steering mechanism, not an afterthought.

In the remainder of this chapter, we explore how CAS is implemented, how CAS-N extends it into a society of cognitive minds, and how CAS forms the epistemic layer of Hybrid Intelligence.

11.3 CAS as a Cognitive Interface for Mixed and Hybrid Data

Real-world datasets are rarely clean. They contain contradictions, stylistic variation, inconsistent quality, and material that ranges from highly reliable to deeply unreliable. Traditional models treat all data as equal: every token contributes equally to the loss, regardless of whether it is safe, accurate, relevant, or stylistically appropriate.

CAS changes this. By attaching a cognitive vector to each training example, CAS provides a structured interface for learning from heterogeneous data. Instead of forcing the model to infer quality, safety, or style implicitly, CAS makes these properties explicit and learnable.

11.3.1 CAS as a Labeling Layer

A CAS vector can be attached to any training sample:

$$(x, y, \text{CAS}(v)).$$

Each dimension of the vector represents a cognitive property that the model should learn to recognize and reproduce. Examples include:

- **safety**: whether the content is safe, harmful, or ambiguous,
- **accuracy**: whether the information is reliable or noisy,
- **relevance**: whether the sample belongs to a specific domain,
- **style**: formal, informal, poetic, technical, narrative,
- **personality**: concise, verbose, warm, neutral, humorous.

This allows a single dataset to contain:

- high-quality expert material,
- low-quality or noisy material,
- safe and unsafe examples,
- multiple writing styles,
- multiple domains,
- multiple personalities.

Instead of filtering or discarding data, CAS turns heterogeneity into a training signal.

11.3.2 Learning Safety and Accuracy During Training

Safety and accuracy are often treated as post-hoc alignment problems: filters, classifiers, or reinforcement layers are added *after* the model is trained. CAS integrates these dimensions directly into the model’s internal representation.

A model trained with CAS learns:

- what safe content looks like,
- what unsafe content looks like,
- how to distinguish reliable from unreliable information,
- how to express uncertainty through its accuracy channel.

This makes safety and accuracy *structural* rather than external. The model does not need to be corrected after generation; it learns to produce safe, accurate content by design.

11.3.3 Relevance and Domain Conditioning

CAS also enables domain-aware training. A single model can learn to operate across multiple domains—mathematics, storytelling, science, law—without confusing them.

A relevance dimension in the CAS vector allows the model to:

- identify which domain a sample belongs to,
- learn domain-specific patterns,
- avoid cross-domain contamination,
- switch domains during generation by changing the target vector.

This is especially powerful in modular systems where different pages or experts specialize in different semantic regions.

11.3.4 Style and Personality as Cognitive Dimensions

Style and personality are often controlled through prompting, but prompts are ambiguous, brittle, and difficult to interpret. CAS provides a structural alternative.

A CAS vector can encode:

- concise vs. verbose,
- warm vs. neutral,
- humorous vs. serious,
- poetic vs. technical,
- narrative vs. analytical.

During training, the model learns to associate these stylistic properties with specific patterns in the data. During generation, a target CAS vector allows precise control over the output style without relying on fragile textual instructions.

11.3.5 Why CAS Works for Hybrid and Contaminated Datasets

The strength of CAS lies in its ability to turn messy data into structured cognitive signals. Instead of discarding low-quality or unsafe material, CAS teaches the model to:

- recognize it,
- classify it,
- separate it,
- and avoid reproducing it unless explicitly requested.

This makes CAS a practical tool for training on real-world corpora, where perfect curation is impossible.

CAS is therefore not merely an epistemic mechanism. It is a general cognitive interface that allows models to learn from heterogeneous data while remaining steerable, interpretable, and safe.

In the next section, we examine how CAS is implemented at the model level through multi-output architectures known as CAS-Networks.

11.4 CAS-Networks — A Minimal Architecture for Cognitive Vectors

CAS becomes operational when it is embedded directly into the model architecture. A CAS-Network is a neural model that does not merely predict a token or value, but also produces a cognitive vector describing the properties of that output. This transforms the model from a passive predictor into an active cognitive agent: a system that knows what kind of output it is generating.

11.4.1 Multi-Output Architecture

A CAS-Network maps an input x to two outputs:

$$f_{\text{CAS}}(x) \rightarrow (y_{\text{content}}, \text{CAS}(v)).$$

The first component is the primary prediction—typically a token, a regression value, or a classification label. The second component is a v -dimensional cognitive vector encoding the model’s assessment of the output’s properties.

Each dimension of the CAS vector has its own activation geometry:

- **content** — linear or softmax, depending on the task,
- **safety** — sigmoid, representing safe/unsafe,
- **accuracy** — sigmoid, representing reliable/unreliable,
- **style/personality** — tanh or sigmoid, depending on scale,
- **domain markers** — soft indicators for domain relevance.

This separation is not cosmetic. It forces the network to learn distinct cognitive functions rather than entangling them in a single output channel.

11.4.2 Why CAS-Networks Work So Well

CAS-Networks are typically implemented as multilayer perceptrons (MLPs) or lightweight transformer blocks. Their effectiveness comes from a simple fact: MLPs are excellent universal function approximators for multi-output mappings.

A CAS-Network learns three things simultaneously:

- the primary mapping (content),
- the cognitive properties of that mapping (CAS),
- the relationship between content and cognition.

Because each output channel has its own activation geometry, the model naturally decomposes these roles during training. The result is not a bundle of correlated numbers, but a structured cognitive judgment.

11.4.3 CAS Without PRE

CAS-Networks do not require PRE, recursion, or propose–refine cycles. They function as standalone models:

- in classification tasks,
- in regression tasks,
- in token prediction,
- in sequence generation,
- in domain-specific reasoning.

This makes CAS broadly applicable. It can be added to existing architectures with minimal changes, turning any model into a cognitive system capable of expressing safety, accuracy, relevance, and style.

11.4.4 CAS as Structural Decomposition

The key insight is that CAS is not a heuristic or a post-processing layer. It is a structural decomposition of knowledge:

- **content** expresses what the model believes,
- **accuracy** expresses how reliable that belief is,
- **safety** expresses whether the situation is stable,
- **style/personality** expresses how the belief is framed,
- **domain markers** express where the belief belongs.

This decomposition gives the model a richer internal representation of its own outputs. It becomes capable of distinguishing:

- safe vs. unsafe content,
- accurate vs. unreliable information,
- relevant vs. irrelevant material,
- technical vs. narrative style,
- concise vs. verbose personality.

CAS-Networks therefore form the foundation of the cognitive vector layer. They provide the architectural substrate on which CAS-N, PRE, and Hybrid Intelligence are built.

In the next section, we extend this idea from a single cognitive agent to a society of agents: CAS-N, a distributed epistemic system where multiple CAS-Networks collaborate, disagree, and form consensus.

11.5 CAS-N — A Society of Cognitive Minds

A single CAS-Network can express cognitive properties of its outputs, but it remains one mind with one internal geometry. Real intelligence, however, benefits from diversity: different perspectives, different biases, different internal error landscapes. CAS-N extends the cognitive vector layer from a single agent to a distributed system of agents, each with its own interpretation of content, accuracy, safety, and style.

CAS-N is therefore not a new architecture, but a *collective configuration* of CAS-Networks. It is a society of cognitive minds.

11.5.1 Multiple CAS-Networks with Independent Histories

A CAS-N system consists of N independent CAS-Networks:

$$\{f_1, f_2, \dots, f_N\}.$$

Each network:

- is trained independently,
- experiences different noise and initialization,
- develops its own internal error geometry,
- produces its own CAS vector for each output.

This independence is essential. It creates epistemic diversity: a set of cognitive agents that do not share the same biases or internal representations.

11.5.2 Consensus as Emergent Agreement

When the N agents agree, their outputs converge:

- content values cluster tightly,
- accuracy values are high,

- safety values are high,
- stylistic or personality dimensions align.

The arbiter observes this convergence and produces a high-confidence judgment. No special rule is needed; consensus is an emergent property of agreement across diverse cognitive agents.

11.5.3 Disagreement and Emergent Abort

When the agents disagree, their outputs diverge:

- content values spread apart,
- accuracy drops,
- safety collapses,
- stylistic dimensions become inconsistent.

The arbiter interprets this divergence as epistemic instability. Confidence collapses to zero, and the system produces an *abort* signal.

This is the safest possible form of refusal:

- not imposed by rules,
- not dependent on thresholds,
- not engineered through heuristics,
- but arising naturally from disagreement among minds.

Abort becomes a structural property of the architecture.

11.5.4 Internal Error Languages

Each CAS-Network develops its own internal error language:

- shaped by its training history,
- nonlinear and asymmetric,
- stable within the network,
- invisible to the arbiter.

The arbiter never sees these internal languages. It only sees their consequences:

- agreement,
- disagreement,
- stability,
- instability.

This is the first time machine-native epistemic diversity becomes visible in a structured system.

11.5.5 CAS-N as a Special Case of CAS(v)

In the modern architecture, CAS-N is no longer the default interpretation of CAS. It is a *special case*:

- CAS(v) defines the cognitive vector space,
- CAS-N defines a society of agents operating in that space.

CAS-N is particularly powerful when:

- epistemic safety is required,
- disagreement must be detectable,
- consensus must be meaningful,
- abort must be structural,
- or multiple cognitive styles must coexist.

But CAS(v) can also be used without CAS-N. A single CAS-Network is often sufficient for style control, safety conditioning, or training on mixed datasets.

CAS-N is therefore best understood as a distributed epistemic layer: a society of cognitive minds whose interactions reveal when the system should trust itself—and when it should not.

In the next section, we examine how CAS is used during training and generation, and how cognitive vectors provide a unified interface for safety, style, relevance, and domain control.

11.6 CAS in Practice — Training, Labeling, and Generation

CAS is not merely a conceptual layer. It is a practical interface that can be used during training, evaluation, and generation. By attaching cognitive vectors to data and conditioning models on target vectors during inference, CAS provides a unified mechanism for safety, style, relevance, and domain control.

This section describes how CAS is used in real systems: how datasets are labeled, how models learn cognitive properties, and how generation becomes steerable through target CAS vectors.

11.6.1 Training with CAS-Labeled Data

A CAS-labeled dataset consists of triples:

$$(x, y, \text{CAS}(v)).$$

Each training example includes:

- the input x ,
- the desired output y ,
- a cognitive vector describing the properties of y .

This allows a single dataset to encode multiple cognitive dimensions:

- safe vs. unsafe content,
- accurate vs. unreliable information,
- relevant vs. irrelevant material,
- domain-specific vs. domain-agnostic samples,
- stylistic and personality traits.

Instead of curating separate datasets for each property, CAS allows all material to coexist in one unified corpus. The model learns to associate each cognitive dimension with its corresponding patterns in the data.

11.6.2 Learning Safety During Training

Safety is often treated as a post-hoc alignment problem. CAS integrates safety directly into the model’s internal representation.

A CAS dimension can encode:

- safe content (1),
- unsafe content (0),
- ambiguous or borderline content (intermediate values).

During training, the model learns:

- what safe content looks like,
- what unsafe content looks like,
- how to detect transitions between safe and unsafe regimes,
- how to express uncertainty through its safety channel.

This makes safety a structural property of the model rather than an external constraint.

11.6.3 Accuracy and Reliability as Cognitive Dimensions

Accuracy is another dimension that benefits from explicit labeling. Instead of assuming all training data is equally reliable, CAS allows the model to learn:

- which samples are trustworthy,
- which samples are noisy or approximate,
- how to express uncertainty in its accuracy channel,
- how to avoid overconfident extrapolation.

This is particularly useful in hybrid datasets that mix:

- expert material,

- synthetic data,
- user-generated content,
- partially correct or approximate examples.

CAS teaches the model to treat these sources differently.

11.6.4 Relevance and Domain Conditioning

CAS can encode domain relevance:

$$\text{CAS}_{\text{relevance}} = 1 \quad (\text{in-domain}),$$

$$\text{CAS}_{\text{relevance}} = 0 \quad (\text{out-of-domain}).$$

This allows the model to:

- learn domain boundaries,
- avoid cross-domain contamination,
- switch domains during generation,
- specialize without losing generality.

Domain conditioning becomes a structural property rather than a prompt engineering trick.

11.6.5 Style and Personality Control

CAS provides a stable mechanism for controlling style and personality. Instead of relying on textual prompts—which are ambiguous and prone to drift—CAS encodes stylistic properties directly in the cognitive vector.

Examples include:

- concise vs. verbose,
- poetic vs. technical,
- warm vs. neutral,
- humorous vs. serious,
- narrative vs. analytical.

During training, the model learns to associate these dimensions with specific stylistic patterns. During generation, a target CAS vector provides precise control over the output style.

11.6.6 Using CAS During Generation

During inference, the model receives:

$$(x, \text{CAS}_{\text{target}}(v)).$$

The target CAS vector acts as a cognitive guide:

“Generate safe, accurate, relevant content in a concise and technical style.”

This makes generation:

- more predictable,
- more controllable,
- more interpretable,
- and less dependent on fragile prompt engineering.

CAS becomes an architectural steering mechanism.

11.6.7 CAS as a Replacement for Prompt Engineering

Prompt engineering relies on natural language instructions, which are:

- ambiguous,
- context-dependent,
- sensitive to phrasing,
- and difficult to interpret.

CAS provides a structural alternative:

- numeric,
- explicit,
- stable,
- and directly tied to training.

Instead of writing:

“Please answer safely, accurately, and in a formal tone.”

one simply provides:

$$\text{CAS}_{\text{target}} = (1, 1, 1, 0.8, 0.2, \dots).$$

CAS turns cognitive control into an architectural feature rather than a linguistic trick.

In the next section, we examine how CAS forms the epistemic layer of Hybrid Intelligence and how it integrates with PRE to create the PRE–CAS family of composite minds.

11.7 CAS as the Epistemic Layer of Hybrid Intelligence

Hybrid Intelligence is built on two complementary principles: *recursion* and *epistemics*. PRE provides the recursive structure: propose–refine cycles, local improvement, and iterative reasoning. CAS provides the epistemic structure: safety, accuracy, relevance, style, and domain awareness.

Together, they form a cognitive system that is both self-improving and self-aware in a structural sense.

11.7.1 Why Hybrid Systems Need Epistemics

A recursive system without epistemics is unstable. It can refine an unsafe idea, reinforce an inaccurate belief, or amplify irrelevant content. PRE alone cannot distinguish:

- safe from unsafe refinements,
- accurate from unreliable proposals,
- relevant from irrelevant directions,
- or appropriate from inappropriate styles.

CAS provides the missing layer. It gives each refinement step a cognitive context, allowing the system to evaluate not only *what* it is doing, but *how* it is doing it.

11.7.2 CAS as a Structural Safety Mechanism

Safety is not an afterthought in Hybrid Intelligence. It is embedded directly into the architecture through CAS:

- unsafe content is labeled during training,
- the model learns to detect unsafe regimes,
- safety becomes a dimension of the cognitive vector,
- unsafe refinements collapse the safety channel,
- CAS-N disagreement triggers emergent abort.

This makes safety a *structural property* rather than a filter or external constraint.

11.7.3 CAS as a Guide for Recursive Reasoning

During a PRE cycle, each refinement step receives:

$$(x, \text{CAS}_{\text{target}}(v)).$$

The CAS vector guides the refinement:

- accuracy encourages reliable improvements,
- safety prevents drift into unstable regions,
- relevance keeps the reasoning on-topic,
- style and personality shape the form of the output.

CAS therefore acts as a cognitive compass for recursive reasoning.

11.7.4 CAS-N as Distributed Epistemics

When multiple experts operate within a page, CAS-N provides:

- epistemic diversity,
- disagreement detection,
- consensus formation,
- structural abort,
- and distributed authority.

This is particularly important in Hybrid Intelligence, where many small agents collaborate. CAS-N ensures that the system does not rely on a single epistemic perspective.

11.7.5 Integration with PRE — The PRE–CAS Family

PRE and CAS integrate naturally:

- PRE provides recursive improvement,
- CAS provides cognitive evaluation,
- PRE–CAS combines both into a hybrid kernel,
- CAS-N adds distributed epistemics,
- PAGED(p) organizes experts into semantic chambers,
- M ensembles provide evolutionary diversity.

The result is the PRE–CAS family: a colony of composite minds, each guided by cognitive vectors and refined through recursive reasoning.

11.7.6 Why CAS Is the Epistemic Layer

CAS is the epistemic layer of Hybrid Intelligence because it provides:

- **structured uncertainty** through accuracy,
- **structural safety** through safety,
- **domain awareness** through relevance,
- **stylistic control** through personality and style,
- **alignment** through target vectors,
- **distributed epistemics** through CAS-N.

Without CAS, Hybrid Intelligence would be recursive but blind. With CAS, it becomes recursive, aware, steerable, and safe.

In the final section, we synthesize the role of CAS as a universal cognitive substrate that unifies safety, style, relevance, and epistemic judgment across all layers of the architecture.

11.8 Synthesis — CAS as a Universal Cognitive Substrate

CAS began as a minimal epistemic interface, but it has grown into something broader: a universal cognitive substrate for intelligent systems. It unifies safety, accuracy, relevance, style, and domain awareness into a single architectural layer that can be learned during training and controlled during generation.

CAS is not a filter, not a heuristic, and not a prompt trick. It is a structural representation of the cognitive properties of content—embedded directly into the model’s outputs and internal representations.

A Unified Interface for Cognitive Control

Across all tasks and domains, CAS provides a consistent mechanism for:

- **safety**: identifying and avoiding unsafe regimes,
- **accuracy**: expressing uncertainty and reliability,
- **relevance**: maintaining domain boundaries,
- **style**: controlling tone, form, and personality,
- **alignment**: steering generation through target vectors.

This makes CAS a universal interface for cognitive control—usable in any model, any dataset, and any architecture.

A Bridge Between Data and Cognition

CAS transforms heterogeneous datasets into structured cognitive signals. Instead of discarding noisy or stylistically diverse material, CAS teaches the model to understand it:

- what is safe,
- what is accurate,
- what is relevant,
- what belongs to which domain,
- and how content should be framed.

This allows models to learn from real-world data without losing steerability or safety.

A Foundation for Distributed Epistemics

CAS-N extends CAS from a single agent to a society of agents. It provides:

- epistemic diversity,
- consensus formation,
- disagreement detection,
- and emergent abort.

This makes CAS not only a cognitive layer, but also an epistemic layer: a substrate for distributed reasoning and structural safety.

Integration with Hybrid Intelligence

In Hybrid Intelligence, CAS integrates seamlessly with PRE:

- PRE provides recursive refinement,
- CAS provides cognitive evaluation,
- PRE–CAS forms the hybrid kernel,
- CAS-N adds distributed epistemics,
- PAGED(p) organizes semantic structure,
- M ensembles provide evolutionary diversity.

CAS is therefore the epistemic backbone of the entire architecture.

The Role of CAS in the Triadic Cosmos

Within the broader unification presented in this book, CAS plays a structural role:

- it decomposes cognition into explicit dimensions,
- it provides a universal interface for control,
- it embeds safety and accuracy into the model itself,
- it enables distributed epistemics through CAS-N,
- and it unifies heterogeneous data into a coherent cognitive space.

CAS is not a detail of the architecture. It is one of its foundations.

Architectural Insight

CAS is a universal cognitive substrate: a vector space in which safety, accuracy, relevance, style, and alignment become structural properties of intelligent systems.

Chapter 12

The PRE–CAS Family: A General Framework for Modular Reasoning

12.1 Introduction

Contemporary machine learning systems are dominated by monolithic architectures: a single model, a single parameter space, and a single, entangled inductive bias. Such systems are powerful pattern recognizers, but they offer limited modularity, poor traceability, and little control over how constraints and objectives shape individual decisions. Reasoning, when it emerges, is implicit and opaque.

The PRE–CAS architecture takes a different approach. It is a general, representation-agnostic framework for structured decision-making. PRE–CAS does not assume a particular domain (language, control, planning), nor a particular granularity (tokens, actions, tool calls, state transitions). Instead, it defines a *cognitive pipeline* that can operate on arbitrary candidate actions, provided they can be evaluated and compared.

In later chapters, PRE–CAS will be instantiated in concrete miniature worlds. In this chapter, we treat it in its most general form: as an architectural pattern for modular reasoning, built from domain decomposition, iterative proposal refinement, orthogonal evaluators, pre-selection filtering, and controlled arbitration.

12.2 The PRE–CAS Stack

At its core, PRE–CAS consists of five interacting components:

1. **Paging** — a decomposition of the global action space into coherent domains, each handled by its own expert or set of experts.
2. **PRE** (Propose–Refine–Evaluate) — an iterative reasoning loop that generates, improves, and assesses candidate actions.
3. **Evaluators** — a collection of orthogonal constraint functions, heuristic or learned, that score candidates along independent dimensions.
4. **Filtering** — a mechanism that removes invalid or structurally inconsistent candidates *before* any final selection is made.

5. **The Arbiter** — an integrative decision-maker that selects the final action from a curated set of valid proposals, balancing correctness with controlled diversity.

These components form a modular pipeline:

Context $\xrightarrow{\text{Paging}}$ Domain Expert $\xrightarrow{\text{PRE}}$ Candidate Set $\xrightarrow{\text{Evaluators} + \text{Filtering}}$ Valid Set $\xrightarrow{\text{Arbiter}}$ Action.

The architecture is compatible with both single-model and multi-model settings. In the latter case, multiple PRE-CAS instances can run in parallel, each with its own parameters and history, forming a population of reasoning agents that can be analyzed at the ensemble level.

12.3 Design Principles

PRE-CAS is guided by four architectural principles:

- **Modularity.** Each component—pages, experts, evaluators, filtering, arbiter—is separable, inspectable, and replaceable. The architecture encourages local changes rather than global retraining.
- **Orthogonality.** Different evaluators encode different constraints; experts specialize in disjoint domains; refinement steps incorporate feedback without collapsing all signals into a single undifferentiated score.
- **Traceability.** Every proposal, score, and decision is explicit. PRE-CAS exposes its internal reasoning trajectory instead of hiding it inside a monolithic parameter space.
- **Plurality.** The architecture supports multiple experts per domain and multiple full models in parallel, enabling diversity, robustness, and population-level analysis.

The remainder of this chapter formalizes each component of the PRE-CAS stack in turn, starting from domain decomposition (Paging) and progressing through the PRE loop, evaluators, filtering, arbitration, and ensemble-level structure.

12.4 Paging: Domain Decomposition of the Action Space

A central design principle of PRE-CAS is that no single expert should be responsible for the entire action space. Instead, the architecture decomposes the global space of possible actions into a set of *pages*: coherent, non-overlapping domains, each handled by its own expert or set of experts.

Paging is representation-agnostic. A “page” may correspond to a subset of tokens, a family of actions, a class of state transitions, a tool interface, or any other structured region of the decision space. The PRE-CAS architecture does not prescribe how pages must be defined; it only requires that the decomposition be deterministic and that each page be internally coherent.

12.4.1 The Action Space

Let $\mathcal{A}$ denote the full action space of the system. An action may be:

- a discrete symbol,

- a structured object,
- a tool invocation,
- a state transition,
- a planning operator,
- or any other atomic decision unit.

The architecture assumes only that actions can be enumerated or generated, and that they can be evaluated by the system's evaluators.

12.4.2 Definition of a Page

A *page* is a subset of the action space:

$$P_i \subseteq \mathcal{A},$$

such that the collection of pages forms a partition:

$$\mathcal{A} = \bigcup_{i=1}^p P_i, \quad P_i \cap P_j = \emptyset \text{ for } i \neq j.$$

Each page is associated with one or more *experts*, which are models specialized in predicting or generating actions within that page.

12.4.3 The Paging Function

Paging is governed by a deterministic routing function:

$$\pi : \mathcal{C} \rightarrow \{1, \dots, p\},$$

where $\mathcal{C}$ is the space of contexts. Given the current context $c \in \mathcal{C}$, the paging function selects the index of the page whose experts should be activated.

The paging function may depend on:

- the current state or history,
- the last chosen action,
- structural constraints,
- domain-specific rules,
- or learned routing heuristics.

The architecture does not constrain how π is implemented; it only requires that the mapping be deterministic so that the reasoning process remains traceable.

12.4.4 Experts Within a Page

Each page P_i contains N experts:

$$E_{i,1}, E_{i,2}, \dots, E_{i,N}.$$

Experts within the same page share:

- the same action domain,
- the same input structure,
- the same evaluator set,
- and the same PRE loop.

They differ only in their parameters or training histories. This local plurality provides robustness and controlled variation without requiring global model diversity.

12.4.5 Why Paging Matters

Paging provides several architectural advantages:

- **Modularity.** Each page isolates a coherent region of the action space, enabling specialization and simplifying expert design.
- **Safety.** Invalid or out-of-domain actions are never proposed, because experts operate only within their assigned page.
- **Scalability.** The action space can grow without requiring a single model to handle all possible decisions.
- **Interpretability.** The paging function makes the system’s routing decisions explicit and traceable.
- **Parallelism.** Multiple experts can be trained or evaluated independently within each page, enabling efficient ensemble methods.

Paging is therefore the structural foundation of PRE-CAS: it decomposes the decision space into manageable domains and ensures that each expert operates within a well-defined region of competence.

12.5 The PRE Loop: Propose, Refine, Evaluate

At the heart of the PRE-CAS architecture lies the PRE loop: an iterative mechanism for generating, improving, and assessing candidate actions. PRE is a general cognitive operator. It does not assume that actions are tokens, moves, tool calls, or symbolic steps; it only assumes that actions can be proposed, refined, and evaluated.

The PRE loop transforms decision-making from a single-step prediction into a structured reasoning process. Each iteration produces a richer set of candidates, shaped by evaluator feedback and constrained by the domain selected through paging.

12.5.1 Input Structure

Each PRE iteration receives a structured input vector

$$x = (c, a_{\text{prev}}, a_{\text{prop}}, e, r),$$

where:

- c is the current context,
- a_{prev} is the previous action (or a null symbol at the start),
- a_{prop} is the current proposal being refined,
- e is the vector of evaluator scores from the previous iteration,
- $r \in \{P, R\}$ is a role indicator distinguishing between the initial *propose* step and subsequent *refine* steps.

This structure ensures that PRE has access to both the global context and the local refinement trajectory.

12.5.2 The Propose Step

The first iteration of PRE generates an initial proposal:

$$a_0 = \text{PRE}(c, a_{\text{prev}}, \emptyset, \emptyset, P).$$

This proposal is unconstrained except by the expert associated with the current page. It represents the model's first hypothesis about the next action.

12.5.3 The Refinement Step

Subsequent iterations refine the proposal using evaluator feedback. Given a proposal a_i and evaluator scores e_i , PRE produces a new candidate:

$$a_{i+1} = \text{PRE}(c, a_{\text{prev}}, a_i, e_i, R).$$

Refinement allows the system to correct weak proposals, incorporate structural or semantic constraints, and explore alternative trajectories within the same page.

12.5.4 Evaluator Feedback

After each proposal or refinement, the evaluators compute a score vector:

$$e_i = (e_i^{(1)}, e_i^{(2)}, \dots, e_i^{(m)}),$$

where each component corresponds to a distinct constraint dimension.

Evaluator feedback is not aggregated inside PRE. Instead, PRE receives the full vector, preserving orthogonality and allowing the model to respond differently to different constraint types.

12.5.5 Accumulating Candidates

Each iteration produces a candidate action and its associated evaluator scores. After k refinement steps, the PRE loop yields a set:

$$\mathcal{C} = \{(a_0, e_0), (a_1, e_1), \dots, (a_k, e_k)\}.$$

This set is passed to the filtering and arbitration stages. PRE itself does not select among candidates; it only generates and refines them.

12.5.6 PRE Pseudocode

Algorithm 1: General PRE Loop

Input: context c , previous action a_{prev} , refinement depth k

Output: candidate set $\mathcal{C}$

$\mathcal{C} \leftarrow \emptyset$;

$a \leftarrow \text{PRE}(c, a_{\text{prev}}, \emptyset, \emptyset, P)$;

$e \leftarrow \text{Evaluate}(a, c)$;

$\mathcal{C} \leftarrow \mathcal{C} \cup \{(a, e)\}$;

for $i \leftarrow 1$ **to** k **do**

$a \leftarrow \text{PRE}(c, a_{\text{prev}}, a, e, R)$;

$e \leftarrow \text{Evaluate}(a, c)$;

$\mathcal{C} \leftarrow \mathcal{C} \cup \{(a, e)\}$;

return $\mathcal{C}$;

12.5.7 Why PRE Matters

The PRE loop provides several advantages over single-step prediction:

- **Structured reasoning.** Decisions emerge from iterative refinement rather than a single forward pass.
- **Constraint integration.** Evaluator feedback is incorporated explicitly at each step.
- **Robustness.** Weak initial proposals can be corrected during refinement.
- **Interpretability.** Each refinement step exposes the system’s internal reasoning trajectory.
- **General applicability.** PRE operates on arbitrary actions and is not tied to any specific domain.

PRE transforms action generation into a multi-step cognitive process, laying the foundation for the filtering and arbitration mechanisms that follow.

12.6 Evaluators: Orthogonal Constraint Functions

Evaluators are the constraint-enforcing components of the PRE-CAS architecture. They assess candidate actions along independent dimensions and provide explicit feedback to the PRE loop. Evaluators do not generate actions themselves; they score them. Their role is to make constraints *visible* to the system and to ensure that reasoning is shaped by structured, interpretable signals.

Evaluators are representation-agnostic. They may operate on tokens, symbolic actions, state transitions, tool calls, or any other form of decision unit. The architecture imposes no restrictions on how evaluators are implemented, only that they return a vector of scores that PRE can use during refinement.

12.6.1 Evaluator Functions

An evaluator is a function

$$E_j : \mathcal{A} \times \mathcal{C} \rightarrow [0, 1],$$

mapping an action $a \in \mathcal{A}$ and context $c \in \mathcal{C}$ to a normalized score. A score of 1 indicates full compliance with the constraint, while a score of 0 indicates a violation.

Given m evaluators, the system computes a score vector

$$e = (E_1(a, c), E_2(a, c), \dots, E_m(a, c)).$$

This vector is passed unchanged to the PRE loop and later to the filtering and arbitration stages. PRE-CAS does not collapse evaluator outputs into a single scalar; orthogonality is preserved throughout the pipeline.

12.6.2 Types of Evaluators

Evaluators may be:

- **Heuristic.** Rule-based functions encoding structural, logical, or domain-specific constraints.
- **Learned.** Neural or statistical models trained to assess coherence, plausibility, or other high-level properties.
- **Hybrid.** Combinations of deterministic rules and learned components.
- **Hard constraints.** Evaluators whose violations immediately invalidate a candidate.
- **Soft constraints.** Evaluators whose scores influence arbitration but do not eliminate candidates outright.

The architecture does not prescribe the number or type of evaluators. Different instantiations may use different sets depending on the domain.

12.6.3 Orthogonality

A defining property of evaluators in PRE-CAS is *orthogonality*. Each evaluator captures a distinct dimension of correctness or quality. This separation has several advantages:

- **Interpretability.** Each constraint is visible and independently measurable.
- **Modularity.** Evaluators can be added, removed, or replaced without affecting the rest of the architecture.
- **Fine-grained feedback.** PRE receives a full vector of scores, enabling targeted refinement.
- **Avoiding entanglement.** Constraints do not collapse into a single opaque signal.

Orthogonality ensures that the system’s reasoning process remains transparent and that constraints can be analyzed individually.

12.6.4 Evaluator Interface

All evaluators share a common interface:

- **Input:** a candidate action a and the current context c .
- **Output:** a normalized score $s \in [0, 1]$.
- **Optional metadata:** evaluators may return auxiliary information (e.g., violation types), but only the score is required by the architecture.

This uniform interface allows PRE, filtering, and arbitration to treat all evaluators identically, regardless of their internal implementation.

12.6.5 Evaluator Set

Let $\mathcal{E} = \{E_1, \dots, E_m\}$ denote the evaluator set. The PRE-CAS architecture assumes:

- $\mathcal{E}$ is fixed during inference,
- evaluators operate independently,
- evaluators do not communicate with one another,
- and evaluators do not modify candidate actions.

Evaluators are therefore *critics*, not generators.

12.6.6 Role in the PRE-CAS Pipeline

Evaluators shape the reasoning process in three ways:

1. **During PRE.** Evaluator scores guide refinement, allowing PRE to correct weak proposals.
2. **During filtering.** Hard constraints eliminate invalid candidates before arbitration.
3. **During arbitration.** Soft constraints influence the final selection among valid candidates.

Evaluators thus form the backbone of correctness and structure within the PRE-CAS architecture. They define what it means for an action to be valid, coherent, or aligned with the system's objectives.

12.7 Filtering: Validity Before Selection

After the PRE loop has generated a set of candidate actions and their evaluator scores, the next stage of the PRE-CAS pipeline is *filtering*. The purpose of filtering is to ensure that only structurally valid, contextually appropriate, and constraint-compliant candidates are passed to the arbiter.

Filtering is a pre-selection mechanism: it does not rank candidates, and it does not choose among them. Instead, it removes candidates that violate hard constraints or fall below minimal quality thresholds. This guarantees that the arbiter operates only on a curated set of valid actions.

12.7.1 Input to Filtering

Filtering receives the full candidate set produced by PRE:

$$\mathcal{C} = \{(a_0, e_0), (a_1, e_1), \dots, (a_k, e_k)\},$$

where each pair consists of:

- a_i — a candidate action,
- e_i — the evaluator score vector for that action.

Filtering does not modify evaluator scores; it only uses them to determine whether a candidate should be retained.

12.7.2 Hard Constraints

A *hard constraint* is a rule that must be satisfied for a candidate to be considered valid. Hard constraints may include:

- structural validity,
- domain-specific legality,
- type or format requirements,
- safety or alignment constraints,
- evaluator scores that must equal 1.

Formally, let $\mathcal{H}$ denote the set of hard constraints. A candidate (a_i, e_i) is valid under hard constraints if:

$$h(a_i, e_i) = 1 \quad \text{for all } h \in \mathcal{H}.$$

Candidates failing any hard constraint are removed immediately.

12.7.3 Soft Thresholding

In addition to hard constraints, filtering may apply *soft thresholds* to evaluator scores. A soft threshold does not enforce strict validity; instead, it ensures that candidates meet a minimal quality level before reaching arbitration.

Given a threshold $\tau \in [0, 1]$, a candidate passes soft filtering if:

$$\min_j e_i^{(j)} \geq \tau,$$

or, more generally, if it satisfies a thresholding function:

$$T(a_i, e_i, \tau) = 1.$$

Soft thresholds allow the system to balance correctness and diversity by adjusting how permissive the filtering stage is.

12.7.4 Filtered Candidate Set

Let $\mathcal{F}$ denote the filtered set:

$$\mathcal{F} = \{(a_i, e_i) \in \mathcal{C} \mid \text{all hard constraints satisfied and soft thresholds met}\}.$$

If $\mathcal{F}$ is empty, the system may:

- relax thresholds,
- fall back to the highest-scoring candidate in $\mathcal{C}$,
- or trigger a recovery mechanism defined by the instantiation.

The PRE-CAS architecture does not prescribe a specific fallback strategy.

Algorithm 2: Filtering Stage

Input: candidate set $\mathcal{C}$, hard constraints $\mathcal{H}$, threshold τ

Output: filtered set $\mathcal{F}$

$\mathcal{F} \leftarrow \emptyset$;

foreach $(a, e) \in \mathcal{C}$ **do**

if *exists* $h \in \mathcal{H}$ *such that* $h(a, e) = 0$ **then**
continue;

if $T(a, e, \tau) = 0$ **then**
continue;

$\mathcal{F} \leftarrow \mathcal{F} \cup \{(a, e)\}$;

return $\mathcal{F}$;

12.7.5 Why Filtering Matters

Filtering plays a crucial role in the PRE-CAS architecture:

- **Safety.** Invalid or harmful actions are removed before they can influence the final decision.
- **Structural integrity.** Only candidates that satisfy domain-specific rules reach the arbiter.
- **Efficiency.** The arbiter operates on a smaller, cleaner set of candidates.
- **Interpretability.** Filtering makes constraint violations explicit and traceable.
- **Control.** Thresholds allow fine-grained tuning of the trade-off between correctness and diversity.

Filtering ensures that the arbiter receives only candidates that are both structurally valid and minimally aligned with the system’s objectives, enabling robust and interpretable decision-making.

12.8 The Arbiter: Integration and Controlled Variation

After filtering has removed invalid or low-quality candidates, the PRE-CAS pipeline reaches its final decision-making component: the *arbiter*. The arbiter selects the final action from the filtered candidate set, balancing correctness, evaluator alignment, and controlled variation.

Unlike PRE, which generates and refines candidates, or evaluators, which score them, the arbiter is a *selector*. Its role is to integrate multiple signals into a single decision while preserving transparency and traceability.

12.8.1 Input to the Arbiter

The arbiter receives the filtered candidate set:

$$\mathcal{F} = \{(a_i, e_i)\},$$

where each element consists of:

- a_i — a valid candidate action,
- e_i — its evaluator score vector.

All candidates in $\mathcal{F}$ satisfy hard constraints and meet the minimum threshold requirements. The arbiter therefore operates only on actions that are structurally valid and contextually appropriate.

12.8.2 Scoring Function

The arbiter computes a scalar score for each candidate using a scoring function:

$$S(a_i, e_i) = f(e_i),$$

where f is a deterministic function that aggregates evaluator scores.

The architecture does not prescribe a specific form for f . Common choices include:

- weighted sums,
- geometric or harmonic means,
- lexicographic ordering,
- evaluator-specific priority rules,
- or learned aggregation functions.

The only requirement is that f be transparent and interpretable.

12.8.3 Selecting the Best Candidate

The arbiter identifies the highest-scoring candidate:

$$(a^*, e^*) = \arg \max_{(a_i, e_i) \in \mathcal{F}} S(a_i, e_i).$$

If multiple candidates tie for the highest score, the arbiter may:

- select the earliest candidate,
- select the most refined candidate,
- or apply a controlled variation mechanism (see below).

12.8.4 Controlled Variation

To avoid deterministic collapse and to maintain diversity in the action distribution, the arbiter may introduce controlled variation. This is achieved by selecting from a *score cluster* rather than a single maximum.

Let

$$\mathcal{F}_\delta = \{(a_i, e_i) \in \mathcal{F} \mid S(a^*, e^*) - S(a_i, e_i) \leq \delta\},$$

where $\delta \geq 0$ is a variation parameter.

The arbiter then samples from $\mathcal{F}_\delta$ according to a distribution defined by the instantiation (uniform, softmax, etc.).

Controlled variation provides:

- robustness against overfitting to narrow attractors,
- increased diversity in long sequences of decisions,
- and a mechanism for exploring alternative valid actions.

12.8.5 Arbiter Pseudocode

Algorithm 3: Arbiter

Input: filtered set $\mathcal{F}$, scoring function S , variation parameter δ

Output: final action a_{final}

Compute $S(a, e)$ for all $(a, e) \in \mathcal{F}$;

$(a^*, e^*) \leftarrow \arg \max_{(a, e) \in \mathcal{F}} S(a, e)$;

$\mathcal{F}_\delta \leftarrow \{(a, e) \in \mathcal{F} \mid S(a^*, e^*) - S(a, e) \leq \delta\}$;

Select $(a_{\text{final}}, e_{\text{final}})$ from $\mathcal{F}_\delta$;

return a_{final} ;

12.8.6 Why the Arbiter Matters

The arbiter is the final decision-maker in the PRE-CAS pipeline. Its role is crucial for several reasons:

- **Integration.** It combines evaluator signals into a single, interpretable decision.
- **Correctness.** It ensures that the chosen action is the best-scoring valid candidate.
- **Diversity.** Controlled variation prevents deterministic collapse and encourages exploration of alternative valid actions.
- **Traceability.** The scoring function and selection mechanism make the final decision fully transparent.
- **Modularity.** The arbiter can be replaced or extended without modifying PRE, evaluators, or paging.

Together with filtering, the arbiter transforms the PRE-generated candidate set into a single, high-quality action that respects both structural constraints and evaluator-defined objectives.

12.9 Ensembles: Plurality of Minds

Although PRE-CAS can operate as a single-model architecture, it naturally extends to a multi-model setting. An *ensemble* consists of M independent PRE-CAS instances, each with its own parameters, training history, and internal representations. Ensembles introduce plurality into the reasoning process, providing robustness, diversity, and a foundation for population-level analysis.

The ensemble layer is optional but conceptually important: it transforms PRE-CAS from a single cognitive pipeline into a small society of interacting reasoners.

12.9.1 Definition of an Ensemble

An ensemble is a collection:

$$\mathcal{M} = \{M_1, M_2, \dots, M_M\},$$

where each M_i is a complete PRE-CAS system with its own:

- paging function,
- experts and parameters,
- evaluator set,
- filtering mechanism,
- arbiter.

The architecture does not require ensemble members to share parameters or training data. They may be identical, partially aligned, or entirely distinct.

12.9.2 Parallel Reasoning

Given a context c , each model M_i independently produces an action:

$$a_i = M_i(c).$$

This yields an action set:

$$\mathcal{A}_{\text{ens}} = \{a_1, a_2, \dots, a_M\}.$$

The ensemble therefore provides M parallel perspectives on the same context.

12.9.3 Ensemble Arbitration

The architecture allows, but does not require, a *global arbiter* that aggregates the outputs of all models. A global arbiter may:

- select the majority action,
- choose the highest-scoring action under a shared evaluator set,
- apply diversity-aware selection rules,
- or simply record the distribution for analysis.

Formally, a global arbiter is a function:

$$A_{\text{global}} : \mathcal{A}_{\text{ens}} \rightarrow \mathcal{A}.$$

The PRE-CAS architecture does not prescribe a specific aggregation rule.

12.9.4 Benefits of Ensembles

Ensembles provide several architectural advantages:

- **Robustness.** Independent models reduce the risk of systematic failure modes.
- **Diversity.** Different parameterizations yield different attractors and reasoning styles.
- **Plurality.** Multiple models provide multiple hypotheses, enabling richer decision-making.
- **Traceability.** Differences between models can be analyzed to understand uncertainty, disagreement, or structural bias.
- **Evolvability.** Ensemble members can be retrained, replaced, or mutated independently.

Ensembles turn PRE-CAS into a population-based reasoning system, enabling ecological dynamics such as convergence, divergence, specialization, and collective behavior.

12.9.5 Architectural Neutrality

The ensemble layer is deliberately neutral:

- It does not assume communication between models.
- It does not require shared evaluators or shared paging.
- It does not impose a specific aggregation mechanism.

This neutrality allows PRE-CAS to support a wide range of multi-agent configurations, from loosely coupled populations to tightly integrated collective systems.

12.9.6 Summary

The ensemble layer extends PRE-CAS from a single reasoning pipeline to a pluralistic architecture. By running multiple PRE-CAS instances in parallel, the system gains robustness, diversity, and the ability to analyze reasoning at the population level. This plurality is a natural complement to the modular structure of PRE-CAS and plays an important role in large-scale deployments.

12.10 Summary and Architectural Outlook

The PRE-CAS architecture provides a general, modular, and interpretable framework for structured decision-making. It decomposes reasoning into transparent components: domain routing through paging, iterative proposal generation through PRE, orthogonal constraint evaluation, validity filtering, and controlled arbitration. Each component is independent yet tightly integrated, forming a cognitive pipeline that is both flexible and robust.

12.10.1 Architectural Summary

The key elements of PRE-CAS can be summarized as follows:

- **Paging** partitions the action space into coherent domains, enabling specialization, safety, and modularity.
- **PRE** transforms decision-making into an iterative reasoning process, where proposals are refined using explicit evaluator feedback.
- **Evaluators** provide orthogonal constraint signals, ensuring that correctness, structure, and alignment remain visible and measurable.
- **Filtering** enforces validity before selection, guaranteeing that only structurally sound candidates reach the arbiter.
- **The Arbiter** integrates evaluator signals into a final decision, balancing correctness with controlled variation.
- **Ensembles** extend the architecture from a single reasoning pipeline to a pluralistic system, enabling robustness, diversity, and population-level analysis.

Together, these components form a modular reasoning architecture that is transparent, extensible, and representation-agnostic.

12.10.2 Design Properties

PRE-CAS exhibits several desirable properties for large-scale reasoning systems:

- **Modularity.** Components can be added, removed, or replaced without altering the overall structure.
- **Traceability.** Every proposal, score, and decision is explicit and inspectable.
- **Orthogonality.** Constraints remain independent, avoiding entanglement and preserving interpretability.
- **Plurality.** Multiple experts and multiple models enable diverse reasoning styles and robust decision-making.
- **General applicability.** PRE-CAS operates on arbitrary actions and is not tied to any specific domain or representation.

These properties make PRE-CAS suitable for a wide range of reasoning tasks, from symbolic manipulation to generative modeling, planning, and multi-agent systems.

12.10.3 Outlook

The PRE-CAS architecture is intentionally general. It defines a pattern for modular reasoning that can be instantiated in many different environments. Subsequent chapters will explore concrete realizations of this architecture in specific miniature worlds, demonstrating how the abstract components of PRE-CAS behave in practice.

These instantiations are not part of the architecture itself; they are applications. The architecture remains independent of any particular domain, granularity, or training regime. Its purpose is to provide a principled, transparent, and extensible foundation for structured cognition.

In summary, PRE-CAS offers a blueprint for building reasoning systems that are modular, interpretable, and adaptable. It replaces monolithic prediction with structured decision-making, enabling systems that can reason, refine, and evaluate their own actions in a transparent and principled manner.

12.11 PRE-CAS Overview Table

Component	Input	Output	Role	Guarantees	Variations
Paging	Context c	Page index p	Selects domain for next action	Deterministic routing; domain coherence	Rule-based, learned, hybrid
Expert	c, a_{prev}	Proposal logits / action	Generates actions within one domain	No cross-domain errors	Single or multiple experts per page
PRE-Propose	$(c, a_{\text{prev}}, \emptyset, \emptyset)$	Initial proposal a_0	Starts refinement loop	Traceable, domain-bounded proposal	Neural, symbolic, heuristic
PRE-Refine	$(c, a_{\text{prev}}, a_j, e_j)$	Refined a_{j+1}	Improves proposal using evaluator feedback	Corrects weak proposals	Any refinement depth k
Evaluators	Action a , context c	Score vector e	Assess orthogonal constraints	Independent signals; full traceability	Heuristic, learned, hybrid
Filtering	Candidates (a_j, e_j)	Filtered set F	Removes invalid / low-quality actions	Hard constraints enforced	Threshold tuning; fallback rules
Arbiter	Filtered set F	Final action a_{final}	Selects best action with variation control	Transparent scoring; bounded diversity	Score clusters; sampling rules
Ensemble	Context c	$\{a_1, \dots, a_M\}$	Parallel independent reasoning	Plurality; robustness; ecology	With/without global arbiter

Table 12.1: Concise overview of the PRE-CAS modular reasoning architecture.

Architectural Insight

PRE-CAS is not a monolithic intelligence. It is a composition of minds: independent experts, evaluators, and arbiters whose coordinated interaction produces coherent reasoning.

Chapter 13

Toy Story: Generation Using PAGED-PRE-CAS

13.1 Introduction: Why Toy Story?

Toy Story is a miniature generative universe designed to make the behaviour of a modular reasoning system concrete, observable, and scientifically tractable. Where modern large language models rely on opaque high-dimensional transformations, Toy Story takes the opposite approach: it is deliberately small, modular, transparent, and fully interpretable. It is not a toy in the trivial sense, but a didactic laboratory—a controlled microcosm in which the dynamics of structure, constraint, and recursive improvement can be studied with unusual clarity.

The world is simple enough to expose every decision the system makes, yet rich enough to reveal how coherent actions emerge, how errors arise, and how iterative refinement shapes the expressive landscape of a population of models. Toy Story was built to answer a simple question: *What happens when we construct a generative system that reasons instead of samples?* The result is a model that not only produces thousands of coherent stories, but also reveals—in full detail—how those stories are constructed and how the system navigates its own space of possibilities.

This chapter introduces the Toy Story world and the mechanisms through which the system interacts with it. It provides the empirical foundation for the ecological and theoretical insights developed in the next chapter, where Toy Story becomes a microscope for studying collapse, attractors, diversity, and the dynamics that govern intelligent systems.

13.2 Tokenization and the World Space

Toy Story begins with a radically simple but structurally powerful tokenization scheme. Where modern language models rely on complex subword encodings such as BPE or SentencePiece, Toy Story uses a fully deterministic, curriculum-aligned vocabulary of only 144 tokens. This minimalism is not a limitation but a design choice: it creates a world that is small enough to be understood in full detail, yet rich enough to support hundreds of trillions of valid stories. The tokenization pipeline defines the ontology of the Toy Story universe and therefore determines what the model can express, what it can learn, and how its stories unfold.

At the core of this world lie three control tokens:

- **EMPTY** — a placeholder used to pad the context window,
- **EOL** — an explicit end-of-line marker,

- **EOS** — an explicit end-of-sequence marker.

These tokens provide the structural backbone of the system. They make line boundaries explicit, allow the context window to be represented in fixed position, and enable simple rules about timing, actor consistency, and line count to be enforced directly in the token stream. Unlike transformers, which must infer structure implicitly from patterns in data, Toy Story encodes structure explicitly.

All tokens are stored in a *TokenDictionary*, a transparent mapping between textual forms and integer identifiers. Token IDs are generated deterministically using a simple hashing function, ensuring that the vocabulary is stable across runs and independent of dataset order. Each token also carries a numerical suffix (0–4) that represents a stylistic dimension used throughout the Toy Story world.

The result is a compact but expressive world space:

- 144 total tokens,
- 34 possible token transitions observed in the dataset,
- 155 million curriculum stories,
- approximately 470 trillion valid generator stories.

This combination of simplicity and expressive power is central to Toy Story’s purpose. Because the token space is small, every transition can be inspected, every rule can be understood, and every error can be traced. Because the world is large, the model can still exhibit meaningful variation, specialization, and emergent behaviour. Tokenization is therefore not merely a preprocessing step: it is the foundation of the Toy Story universe itself.

13.3 Embeddings and the Context Window

Toy Story represents language using an intentionally simple and fully interpretable embedding scheme. Instead of the high-dimensional learned embeddings used in modern transformers, Toy Story employs two compact representational layers—a one-dimensional input embedding and a small normalized output embedding—combined with a fixed-size sliding context window. This design makes the entire representational pipeline transparent, debuggable, and easy to inspect.

Input Embeddings: A Minimal Semantic Trace

Each token in the vocabulary is assigned a single scalar input embedding, generated deterministically using a Gaussian distribution seeded by the token’s identifier. This one-dimensional representation is surprisingly effective in the constrained world of Toy Story: it provides enough variation for the system to learn local semantic continuity, while remaining simple enough to visualize and reason about directly.

The input embeddings populate the context window and serve as the primary representation of the story’s history. Because the embedding space is one-dimensional, every feature of the representation can be examined without ambiguity.

Output Embeddings: Lightweight Semantic Geometry

In addition to the scalar input embedding, each token is assigned a small output embedding vector (e.g., 16 dimensions), also generated from a normalized Gaussian distribution. These vectors provide a compact semantic geometry that supports contrastive learning: the system learns which transitions are natural and which represent small structural or semantic deviations.

The Sliding Context Window: A Fixed-Position Working Memory

Toy Story maintains a sliding window of the last 35 tokens, forming the system's working memory. Unlike attention-based architectures, which dynamically compute relevance across the entire sequence, Toy Story uses a fixed-position representation: each position in the window corresponds to a specific slot in the input vector. This design has several advantages:

- **Transparency** — the full context is visible and interpretable at all times.
- **Determinism** — the same context always produces the same input encoding.
- **Trainability** — models see each structural feature in a consistent position.
- **Simplicity** — no attention mechanisms, no positional embeddings, no hidden state.

The window stores:

- the input embeddings of the last 35 tokens,
- the current line index and position index,
- the ID and output embedding of the most recent token.

EMPTY tokens fill unused positions at the beginning of generation, ensuring that the input vector always has the same dimensionality.

Why This Representation Works

Despite its simplicity, the embedding and context system is remarkably expressive. The evaluators rely on the context window to enforce actor consistency, timing structure, line boundaries, and stylistic constraints. The output embeddings provide a fine-grained semantic signal that helps distinguish natural transitions from micro-errors.

The result is a representational scheme that is:

- small enough to be fully understood,
- structured enough to support meaningful constraints,
- expressive enough to generate thousands of coherent stories,
- and stable enough to train efficiently on CPU.

Toy Story demonstrates that meaningful language generation does not require massive embeddings or attention mechanisms. It requires the right structure: a clear, interpretable working memory and a representational scheme that exposes its internal dynamics.

13.4 Paging: Decomposing the Vocabulary into Experts

Toy Story uses a simple but powerful paging mechanism to partition the vocabulary into a set of small, non-overlapping *pages*, each handled by its own expert network. Instead of training a single model to predict the entire vocabulary, the system assigns each token to exactly one page, transforming generation from a monolithic classification problem into a modular process in which each expert is responsible for a coherent semantic domain.

The motivation is straightforward: different kinds of tokens behave differently. Timing words, actors, objects, locations, and structural markers follow distinct transition patterns and obey different constraints. A single softmax over the entire vocabulary forces a model to learn all of these patterns simultaneously, increasing interference and making errors harder to diagnose. Paging avoids this problem by ensuring that each expert only needs to reason about a small, semantically consistent subset of the world.

Toy Story uses twelve pages in total: seven non-deterministic pages with their own neural networks, and five deterministic pages that contain only a single possible output token. The non-deterministic pages typically contain between 4 and 40 tokens, resulting in extremely small softmax layers that are easy to train, easy to debug, and resistant to noise. Deterministic pages require no model at all: when such a page is selected, the next token is known with certainty.

Constructing Pages from Successor Structure

Pages are constructed using an *overlap-minimizing* algorithm based on successor maps extracted from the curriculum. For each token, the system records all tokens that can legally follow it in the dataset. Tokens with similar successor sets are grouped together, ensuring that each page corresponds to a coherent semantic domain. The algorithm proceeds by:

1. ranking tokens by the size of their successor sets,
2. selecting the most complex token as a page root,
3. adding all tokens whose successors lie within the emerging page,
4. removing these tokens from the pool,
5. repeating until all tokens are assigned.

This process yields pages that reflect the underlying structure of the Toy Story universe. Timing words cluster together; actor tokens cluster together; object and location tokens form their own pages; and structural tokens such as EOL and EOS form deterministic pages. The resulting decomposition mirrors the semantic topology of the world.

Paging as a Structural Constraint

Paging is not merely an optimization; it also enforces strong structural constraints. By selecting the page based on the last token in the context window, the system determines *which expert is allowed to speak next*. This ensures that:

- timing words appear only where timing words are expected,
- actors appear only in actor positions,

- objects and locations appear only in semantically valid contexts,
- structural tokens appear only at line boundaries.

This eliminates entire classes of errors that transformer models routinely make. A page that contains only timing words cannot accidentally output an actor; a page that contains only objects cannot output EOS; a deterministic page cannot produce invalid structure. Paging therefore acts as a built-in correctness layer, keeping generation within the boundaries of the world’s rules.

Why Paging Matters

Paging provides several key advantages:

- **Modularity** — each expert specializes in a narrow domain.
- **Interpretability** — errors can be traced to specific pages.
- **Stability** — small softmax layers converge quickly and reliably.
- **Scalability** — new domains can be added by introducing new pages.
- **Safety** — structural errors become impossible by construction.

In Toy Story, paging is the mechanism that keeps the world coherent. It ensures that each part of the vocabulary behaves according to its own rules, and that generation remains aligned with the structure of the universe.

13.5 PRE: Propose, Refine, Evaluate

Toy Story does not generate tokens in a single step. Instead, it uses a small multi-stage process in which the system proposes a candidate token, evaluates it, and then refines it through several iterations. This mechanism—PRE, for *Propose–Refine–Evaluate*—turns generation into a short deliberative loop rather than a one-shot prediction. The goal is simple: produce a token that fits the structure, timing, and semantics of the Toy Story world.

The Propose Step

In the propose phase, the active page-expert produces an initial candidate token based on the current context window. This first proposal is not taken as final. Instead, it is immediately checked by a set of evaluators that assess:

- semantic coherence,
- stylistic proximity,
- actor consistency,
- timing structure,
- line structure.

The result is a structured evaluation object that records both the token and its constraint profile.

The Refinement Steps

After the initial proposal, the system performs several refinement iterations. Each refinement step receives additional information: the previous proposal and its evaluator scores. This allows the expert to adjust its prediction based on explicit feedback, effectively learning to correct weak or borderline choices.

Each refinement produces a new candidate token, which is again evaluated by the full set of evaluators. All valid proposals across all refinement steps are retained, forming a small pool of filtered candidates. Refinement therefore expands the search space rather than overwriting earlier ideas.

Evaluation as a First-Class Signal

In Toy Story, evaluation happens before selection. The evaluators act as gatekeepers: they filter out invalid tokens, enforce structural rules, and provide continuous feedback during refinement. This ensures that:

- incoherent tokens are rejected immediately,
- structurally invalid tokens never enter the candidate pool,
- refinement is guided by explicit semantic and structural signals,
- the final choice is made from candidates that respect the world's rules.

Evaluation is therefore not a post-processing step but an integral part of the generation loop.

PRE as a Deliberative Process

PRE turns generation into a short sequence of reasoning steps:

1. propose an initial candidate,
2. evaluate it through multiple constraints,
3. refine based on explicit feedback,
4. accumulate all valid proposals,
5. select the final token from the candidate set.

This mirrors human reasoning: we generate an idea, check it, revise it, and consider alternatives before committing.

Why PRE Matters

PRE provides several advantages over single-pass generation:

- **Correctness** — invalid tokens are filtered before selection.
- **Robustness** — refinement corrects weak or borderline proposals.
- **Diversity** — multiple valid candidates survive to the final step.

- **Interpretability** — every proposal and refinement step is traceable.

In Toy Story, PRE is the mechanism that makes the system’s behaviour visible. Every proposal, every correction, and every rejected candidate can be inspected, revealing how the model navigates its own space of possibilities.

13.6 Evaluators: The Mini-Experts

Toy Story enforces structure and semantics through a set of five evaluators, each responsible for a distinct dimension of correctness. These evaluators act as independent checks on every proposed token, ensuring that generation remains coherent, consistent, and aligned with the rules of the Toy Story universe.

Unlike transformer-based models, which rely on implicit statistical correlations to maintain structure, Toy Story makes structure explicit. Each evaluator encodes one aspect of the curriculum—style, identity, timing, line structure, or semantic continuity—and contributes its own judgment to the decision process.

The CAS Evaluator: Enforcing Stylistic Consistency

The CAS evaluator measures how closely a proposed token matches the target CAS value specified in the prompt. Each token carries a numerical suffix (0–4) representing its stylistic variant. The evaluator computes a score based on the distance between the token’s CAS value and the target CAS, producing a continuous signal between 0 and 1.

This evaluator ensures that the story maintains a consistent stylistic tone and supports controlled stylistic variation across thousands of possible stories.

The Actor Evaluator: Maintaining Identity

The actor evaluator enforces identity consistency. It extracts the actor from the prompt (e.g., `alice`, `bob`, `carol`) and ensures that every story line refers to the same actor. If a proposed token introduces a different actor, the evaluator assigns a score of zero, effectively disqualifying the token.

This simple rule prevents one of the most common failure modes in generative models: unintended shifts in narrative focus.

The Timing Evaluator: Enforcing Narrative Order

Toy Story stories follow a strict temporal structure:

- line 1 begins with `first`,
- lines 2–4 begin with `then`,
- line 5 begins with `finally`.

The timing evaluator checks whether a proposed token respects this ordering. Any violation results in a score of zero. This ensures that the narrative unfolds in a logically ordered sequence and prevents temporally inconsistent stories.

The Line Evaluator: Enforcing Structural Form

The line evaluator enforces the structural constraints of the story format. It ensures that:

- each line ends with an **EOL** token,
- the story contains exactly five story lines,
- the **EOS** token appears only at the end,
- no double **EOL** tokens occur.

This evaluator guarantees that the generated story adheres to the formal structure defined by the curriculum.

The Coherence Evaluator: Learning Semantic Continuity

The coherence evaluator is the only learned evaluator. It is a small MLP trained using contrastive micro-errors: token swaps, incorrect transitions, and structural violations. The model outputs a continuous score between 0 and 1, representing how semantically coherent a proposed token is with respect to the current context window.

This evaluator captures local semantic relationships that are not explicitly encoded in the curriculum, such as natural ordering of actions or plausible transitions between objects and locations.

Orthogonality and Collective Judgment

Each evaluator examines a different dimension of correctness:

Evaluator	Dimension	Type
CAS	Style consistency	heuristic
Actor	Identity consistency	heuristic
Timing	Narrative order	heuristic
Line	Structural form	heuristic
Coherence	Semantic continuity	learned

Because the evaluators are orthogonal, their judgments combine into a multi-dimensional assessment of each proposed token. This modularity makes Toy Story fully interpretable: every decision can be traced back to the evaluator that influenced it.

Why Evaluators Matter

The evaluators are essential for keeping Toy Story’s generation process aligned with the rules of the world. They provide:

- **hard constraints** that prevent invalid tokens from being considered,
- **soft signals** that guide refinement toward better proposals,
- **interpretability** by exposing the rationale behind each decision,
- **robustness** by isolating failure modes to specific evaluators,
- **modularity** by separating concerns across independent checks.

Together, the evaluators ensure that every generated story is structurally sound and semantically meaningful.

13.7 Token Filtering Before Selection

Toy Story introduces a distinctive mechanism for controlling generation: candidate tokens are filtered *before* any selection is made, using both logit ranking and evaluator constraints. Unlike transformer-based models, which rely on a probabilistic softmax distribution and apply heuristics after sampling, Toy Story performs deterministic logit ranking and applies structural and semantic filters prior to choosing a token. This ensures that only valid candidates ever enter the decision process.

The filtering mechanism is implemented by the *TokenSelector*, which acts as a gatekeeper between the page-expert’s raw logits and the subsequent reasoning steps. The *TokenSelector* examines the ranked logits produced by the active page-expert and applies all evaluators to each candidate token. Only tokens that satisfy every constraint are retained.

Filtering via Ranked Logits

The filtering process proceeds in two stages:

1. **Identify the highest-logit valid token.** The *TokenSelector* iterates through the logits in descending order. The first token that passes all evaluator checks becomes the *anchor* candidate.
2. **Collect additional valid candidates using a score threshold.** Each candidate has an evaluator-based score

$$s = \text{coherence} + \text{styleScore},$$

where actor, timing, and line constraints must already be satisfied. A threshold is computed as

$$s_{\min} = s_{\text{anchor}} \times \text{scoreThreshold}.$$

Any subsequent token whose evaluator score satisfies

$$s \geq s_{\min}$$

and passes all hard constraints is added to the candidate set.

This produces a curated pool of candidates that are both structurally valid and semantically coherent. Tokens that violate actor consistency, timing rules, line structure, CAS style, or coherence are discarded immediately.

Why Filtering Before Selection Matters

Filtering before selection provides several advantages:

- **Structural safety.** Invalid tokens never enter the generation loop.
- **Semantic robustness.** The coherence evaluator rejects transitions that break local continuity.
- **Interpretability.** Every rejected token can be traced to the evaluator that disqualified it.
- **Controlled diversity.** Multiple high-quality candidates survive, enabling variation without chaos.

- **Deterministic behaviour.** The filtering process is fully reproducible and independent of sampling noise.

This approach stands in contrast to transformer-based generation, where the model must implicitly learn structural constraints and where sampling heuristics attempt to correct errors after the fact. Toy Story inverts this logic: constraints come first, logits second.

Filtering as a Cognitive Analogy

Token filtering is not merely an engineering technique; it also mirrors a familiar cognitive pattern: we discard impossible or illogical options before considering the remaining alternatives. In Toy Story, the evaluators act as fast, specialized critics that eliminate invalid proposals early, allowing the system to focus on meaningful choices.

By enforcing constraints at the earliest possible stage, Toy Story ensures that every step of generation remains within the boundaries of the world’s rules.

A Foundation for Structured Generation

Filtering before selection ensures that:

- page-experts remain domain-specific,
- evaluators enforce orthogonal constraints,
- only valid proposals proceed to later steps,
- the candidate set remains small, clean, and interpretable.

Filtering transforms raw logits into meaningful proposals and keeps generation disciplined, rule-governed, and fully traceable.

13.8 The Arbiter: Scoring and Variation Injection

After the system has produced a curated set of valid candidate tokens—each filtered through the evaluators and refined across multiple iterations—the final decision is delegated to the *Arbiter*. The arbiter aggregates all candidate proposals, scores them using evaluator signals, and selects the next token in a way that balances correctness with controlled diversity.

Where the evaluators enforce structure and PRE generates alternatives, the arbiter acts as the decision-maker. Its role is not merely to choose the highest-scoring token, but to preserve variation among high-quality alternatives, preventing premature convergence and enabling expressive diversity within the structural boundaries of the Toy Story universe.

Aggregating Candidate Proposals

The arbiter receives a list of candidate outputs from the refinement loop. Each entry contains:

- the proposed token,
- the evaluator scores associated with that token,
- the logit value from the page-expert,

- metadata about the refinement step that produced it.

All candidates from the propose step and all refinement steps are pooled into a single list. Because invalid tokens have already been filtered out, the arbiter operates exclusively on structurally and semantically valid proposals.

Scoring Candidates

Each candidate is assigned a scalar score based on its evaluator signals. Hard constraints (actor, timing, line) must already be satisfied; the arbiter therefore focuses on the two continuous dimensions:

$$s = \text{coherence} + \text{styleScore}.$$

This score reflects both semantic continuity and stylistic alignment with the target CAS value. Candidates are sorted in descending order of s , producing a ranked list of high-quality alternatives.

Thresholding and Variation Injection

To avoid deterministic collapse into a single preferred token, the arbiter introduces controlled variation. After sorting, it computes a score threshold:

$$s_{\min} = s_{\text{best}} \times \text{scoreThreshold},$$

where s_{best} is the score of the top-ranked candidate. All candidates satisfying

$$s \geq s_{\min}$$

form the *top cluster*. If the cluster contains only one candidate, that token is selected deterministically. If multiple candidates qualify, the arbiter selects uniformly at random from the cluster.

This mechanism ensures:

- **quality** — only high-scoring candidates are eligible,
- **diversity** — multiple strong candidates can be chosen,
- **stability** — randomness is bounded by evaluator scores,
- **expressiveness** — the model avoids premature convergence.

The arbiter therefore injects variation without sacrificing structural correctness or semantic coherence.

A Cognitive Interpretation

The arbiter embodies a familiar cognitive principle: *deliberation among multiple good options*. Humans rarely choose the single best option in a purely deterministic way; we consider a small set of acceptable alternatives and select among them based on context, preference, or stochasticity. Toy Story mirrors this behaviour by:

- gathering all viable proposals,

- scoring them along meaningful dimensions,
- restricting attention to a high-quality subset,
- selecting one option with controlled randomness.

This makes the arbiter an essential part of Toy Story’s decision process.

Why the Arbiter Matters

The arbiter is crucial for maintaining both correctness and diversity:

- It prevents the system from collapsing into deterministic, repetitive patterns.
- It ensures that variation arises only among structurally valid candidates.
- It integrates multiple signals into a single coherent decision.
- It provides a transparent, interpretable mechanism for token selection.

The arbiter transforms a set of filtered, refined proposals into a single next token, balancing precision with expressive flexibility.

13.9 The Ensemble: A Population of Models

Toy Story does not rely on a single generative model. Instead, it uses an *ensemble* of independently trained models, each exposed to a different randomized subset of the curriculum. In the standard configuration, the ensemble contains twenty models, but this number is arbitrary: any population size may be used. What matters is not the specific number, but the fact that Toy Story treats generation as the collective behaviour of a population rather than the output of a single monolithic network.

Each model in the ensemble shares the same structure—identical pages, identical refinement steps, identical evaluators—but differs in its training history. Because each model is trained on a different random ordering of a different subset of the dataset, the ensemble develops natural variation: differences in stylistic preference, transition likelihoods, and coherence judgments. This variation is essential for Toy Story’s role as a reasoning laboratory, as it allows the system to exhibit diversity, specialization, and emergent population-level dynamics.

Story-Level Ensemble Operation

The ensemble operates at the *story level*, not the token level. For each prompt and target CAS value, every model in the population independently generates a complete story. There is no cross-model voting, averaging, or token-level blending. Instead:

1. each model receives the same prompt and CAS target,
2. each model runs its own generation process,
3. each model produces a full story or aborts if no valid continuation exists.

This design preserves the individuality of each model. The ensemble is therefore not a committee that negotiates token-by-token, but a population of independent agents that express their own interpretations of the same task.

Why a Population Matters

Using a population of models provides several advantages:

- **Diversity.** Different training subsets lead to different stylistic and structural preferences.
- **Robustness.** Failures in one model do not propagate to others.
- **Interpretability.** Variation across models reveals which behaviours are stable and which are contingent.
- **Ecological dynamics.** The ensemble forms a miniature cognitive ecosystem, enabling the study of collapse, attractors, and evolutionary behaviour in later chapters.
- **Scalability.** The population size can be increased or decreased without structural changes.

The ensemble therefore serves as a computational analogue of a biological or cognitive population: a set of agents with shared structure but different experiences.

Population-Level Behaviour

Across models, the ensemble exhibits population-level patterns that do not appear in any single model. Differences in training history lead to subtle specializations, biases, and stylistic tendencies. When viewed collectively, these differences form a small cognitive ecology in which variation, drift, and convergence can be observed directly.

Preparation for Collective Recursion

The ensemble is also the foundation for Toy Story’s second training phase: *collective recursion*. Because each model generates its own stories, the ensemble produces a rich pool of candidate training data. In the next chapter, we examine how this population behaves when it is trained on its own output, revealing phenomena such as:

- rapid convergence,
- loss of diversity,
- collapse into shared attractors,
- and the effects of mutation on population dynamics.

The ensemble is therefore not merely a convenience; it is an empirical tool. It transforms Toy Story from a single-model generator into a full cognitive ecology, enabling the study of reasoning, structure, and evolution within a controlled miniature universe.

13.10 Collective Recursion (With and Without Mutation)

Toy Story’s second training phase, *collective recursion*, transforms the ensemble from a set of independently trained models into a population capable of self-improvement. Where curriculum training teaches the basic structure, semantics, and stylistic rules of the Toy Story universe, collective recursion exposes the ensemble to its own generated stories, allowing it to refine its behaviour through iterative self-training.

Collective recursion is an optional mechanism that demonstrates how a population of models behaves when trained on its own output. This section describes the mechanics of recursion, while the next chapter analyzes its ecological consequences.

Overview of the Recursion Cycle

Each recursion epoch proceeds through the following steps:

1. **Generation.** Every model in the ensemble generates stories for all prompts and CAS values. Each model produces a full story or aborts if no valid continuation exists.
2. **Selection.** Only stories that pass full structural and semantic validation are retained. Invalid stories are discarded.
3. **Normalization and Tokenization.** The selected stories are normalized, converted to the canonical line format, and re-tokenized using the same deterministic tokenization pipeline.
4. **Dataset Construction.** The validated stories from all models are pooled into a new dataset, subject to a maximum size per epoch to prevent runaway overfitting.
5. **Retraining.** Each model in the ensemble is retrained on the new dataset, using the same training procedure as in the curriculum phase.
6. **Iteration.** The process repeats for multiple epochs.

This cycle allows the ensemble to refine its behaviour based on its own output, creating a closed-loop learning system.

Recursion Without Mutation

In the simplest form of collective recursion, the ensemble is trained exclusively on its own validated stories, without any artificial variation. This setup reveals how a population of models behaves when exposed to a fixed world and a fixed set of structural constraints.

Mechanically, recursion without mutation is straightforward:

- each model generates stories,
- only fully correct stories are retained,
- the retained stories form the next training dataset,
- all models are retrained on this dataset.

Because the dataset consists only of correct stories, the ensemble gradually converges toward the most stable and easily reproducible patterns in the world. The result is a rapid reduction in expressive diversity and a strong increase in correctness. This behaviour is analyzed in detail in the next chapter.

Recursion With Mutation

Toy Story also supports a variant of collective recursion that introduces controlled mutations into the generated stories before they are added to the training dataset. These mutations are *macro-level* transformations that preserve the structural validity of the story while injecting variation into the dataset.

The mutation operators include:

- **Actor swap** — replacing the actor with another valid actor token.
- **CAS mutation** — randomly altering the CAS suffix of several tokens.
- **Line reordering** — permuting the middle lines while preserving the **first/then/finally** timing structure.

These mutations are applied only to stories that are already structurally valid, ensuring that the resulting dataset remains safe for training. The purpose of mutation is not to break the rules of the world, but to introduce diversity into the training signal.

Mechanically, recursion with mutation follows the same steps as recursion without mutation, with the additional mutation stage inserted between selection and tokenization.

Why Collective Recursion Matters

Collective recursion serves several purposes within the Toy Story framework:

- **It demonstrates how a population behaves under self-training.**
- **It reveals the stability and fragility of structural constraints.**
- **It provides a controlled environment for studying convergence and collapse.**
- **It enables the introduction of artificial variation through mutation.**
- **It prepares the ground for ecological analysis in the next chapter.**

From a scientific perspective, collective recursion provides a miniature laboratory for studying the dynamics of populations of generative models and the forces that shape their evolution.

From Mechanics to Behaviour

The mechanics of collective recursion complete the description of how Toy Story models generate, validate, and refine their own data. We now turn from the mechanism itself to its consequences. When a population of models trains on its own output, new patterns emerge: convergence, loss of diversity, shared attractors, and the effects of mutation on variation. The remainder of this chapter examines these population-level behaviours in detail.

13.11 Summary: What Toy Story Reveals

Example Generated Story

Prompt: Tell a story about Alice

Generated Story:

First Alice meets child3.

Then Alice is in tower3.

Then Alice searches3 with lantern3.

Then Alice sees stone3.

Finally Alice plays with cat3.

Toy Story brings together a set of simple, transparent mechanisms into a fully interpretable generative system. Tokenization defines the ontology of the world; a fixed-position context window provides a visible working memory; lightweight embeddings encode the semantic geometry; paging decomposes the vocabulary into domain-specific experts; evaluators enforce structural, stylistic, and semantic constraints; refinement introduces deliberation; filtering removes invalid candidates; the arbiter balances correctness with controlled variation; and the ensemble introduces natural diversity through independent training histories.

Taken together, these components show that expressive generative behaviour does not require massive models or opaque representations. Toy Story demonstrates that a small, modular system with explicit structure can produce coherent, combinatorial stories while remaining fully interpretable. Every decision is traceable, every error diagnosable, and every mechanism visible.

13.12 Toy Story as a Microscope for Generative Behaviour

Toy Story is not merely a toy model; it is a scientific instrument. Its small size, full transparency, and modular structure turn it into a microscope for studying the fundamental forces that shape generative systems. Where large language models hide their internal dynamics behind billions of parameters and opaque attention patterns, Toy Story exposes every mechanism, every decision, and every failure mode.

Because Toy Story is compact and fully interpretable, it makes visible what large models obscure—how correctness interacts with diversity, how attractors form, how collapse emerges, and how variation sustains a population of generative agents.

Toy Story reveals three things with exceptional clarity:

- **Why collapse becomes visible.** In large models, collapse is buried under scale, noise, and sampling heuristics. In Toy Story, collapse is sharp, measurable, and inevitable.
- **Why attractors can be observed directly.** Transformers distribute their inductive biases across millions of parameters. Toy Story isolates them into explicit structural components, making attractors observable as behavioural phenomena rather than statistical artifacts.
- **Why modular systems reveal their own laws.** Because Toy Story is decomposed into explicit parts, each mechanism leaves a distinct signature on the system’s behaviour. This makes it possible to study generative dynamics as an ecological process.

In this chapter, we now turn from mechanism to behaviour. We examine:

- convergence under self-training,
- collapse into monoculture,
- the narrowing of attractors,
- the role of mutation in sustaining diversity,
- the emergence of niches and specializations,
- and the population-level dynamics of the ensemble.

Toy Story thus becomes a controlled environment for uncovering the laws that govern generative systems. It reveals that collapse is not an accident but a structural force; that variation is essential; and that populations of small, transparent models can illuminate principles that remain hidden in large, monolithic architectures.

13.13 Curriculum Learning Results in the Toy Story Universe

To evaluate the effect of curriculum learning on a population of models with identical architecture but different world-experience, we generate stories for all possible prompts in the Toy Story universe. Each model is queried with 3 prompts for each of the 5 actors, and for target CAS values from 0.00 to 1.00 in steps of 0.01. This yields $15 \times 101 = 1515$ requested stories per model. For an ensemble of 20 models, the total evaluation set consists of 30 300 requested stories.

13.13.1 Effect of Curriculum Dataset Size

To study how curriculum dataset size influences correctness, diversity, and attractor formation, we train three ensembles of models on curriculum sets of size 1 000, 10 000, and 100 000 stories respectively. All models share the same architecture and training procedure; the only difference is the amount of world-experience they receive.

1 000-Story Curriculum

Training on only 1 000 stories yields weak models with highly unstable behavior. Across the 20 models, we observe:

- **Correctness range:** 0% to 47%.
- **Written stories:** 15% to 89%.
- **Unique stories:** 0 to 252 per model.
- **Combined unique stories:** 1 828.

The models frequently abort generation, fail to converge to stable attractors, and exhibit extremely low correctness. Diversity is limited because the world-experience is too small to form meaningful attractors.

10 000-Story Curriculum

Increasing the curriculum to 10 000 stories produces a dramatic improvement across all metrics:

- **Correctness range:** 7% to 93%.
- **Written stories:** 75% to 100%.
- **Unique stories:** 52 to 869 per model.
- **Combined unique stories:** 11 238.

The jump from 1k to 10k is substantial: correctness increases by a factor of 2–5, and diversity increases by nearly an order of magnitude. Models begin to form recognizable attractors, and world-views become more coherent.

100 000-Story Curriculum

Expanding the curriculum to 100 000 stories yields further improvements, but with much smaller relative gains compared to the 1k→10k jump:

- **Correctness range:** 32% to 97%.
- **Written stories:** 72% to 97%.
- **Unique stories:** 254 to 1 172 per model.
- **Combined unique stories:** 11 904.

The improvement from 10k to 100k is modest: correctness increases slightly, and diversity grows only marginally (from 11 238 to 11 904 unique stories). This indicates diminishing returns: once the world-experience is sufficiently rich, additional data refines attractors rather than creating new ones.

The spread is however substantial: the best models produce more than four times as many unique stories as the weakest ones, and correctness varies by a factor of three. This confirms that identical architectures trained on different slices of the same world converge to different attractors.

13.13.2 Effect of Refinement Steps

A core component of PRE (Proposal–Refinement–Evaluation) is the use of refinement steps. Each refinement step generates additional candidate continuations, allowing the arbiter to select the best token among all proposals and refinements. Importantly, even when later refinement steps no longer improve candidate quality, they do not harm performance: the arbiter always selects the best candidate across all steps, not merely the final refinement.

We evaluate the effect of using $k = 0$, $k = 1$, and $k = 2$ refinement steps. Previous results used $k = 4$, which performs slightly better than $k = 2$; we do not repeat those numbers here.

No Refinements ($k = 0$)

Without refinement, models rely solely on their initial proposal. This severely limits the arbiter’s ability to correct weak or unstable trajectories.

- **Correctness range:** 0% to 98%.
- **Written stories:** 51% to 97%.
- **Unique stories:** 0 to 333 per model.
- **Combined unique stories:** 1 628.

The absence of refinement leads to unstable behavior: many models collapse to near zero correctness, and diversity is extremely low. The arbiter has too little choice to steer generation toward stable attractors.

One Refinement ($k = 1$)

Introducing a single refinement step dramatically improves both correctness and diversity. The arbiter now has multiple candidates to choose from, enabling recovery from weak proposals.

- **Correctness range:** 13% to 97%.
- **Written stories:** 67% to 98%.
- **Unique stories:** 121 to 1 063 per model.
- **Combined unique stories:** 9 183.

The improvement from $k = 0$ to $k = 1$ is substantial: correctness stabilizes, diversity increases by more than a factor of five, and attractors become much more consistent across models.

Two Refinements ($k = 2$)

Using two refinement steps yields further improvements, though the gains are smaller than the jump from $k = 0$ to $k = 1$. This indicates diminishing returns and shows that $k = 2$ is close to optimal.

- **Correctness range:** 27% to 97%.
- **Written stories:** 70% to 96%.
- **Unique stories:** 210 to 1 174 per model.
- **Combined unique stories:** 11 388.

The improvement over $k = 1$ is clear but modest. Most of the benefit of refinement comes from the first step; additional steps refine the attractor landscape rather than expanding it.

Refinement plays a central role in PRE: it stabilizes generation, increases the arbiter’s decision power, and enables models to reach stronger attractors even with limited world-experience.

13.13.3 Effect of Evaluators

The PRE architecture relies on multiple evaluators to constrain generation and guide the arbiter toward coherent, stylistically aligned, and semantically valid stories. To understand their contribution, we disable specific evaluators and measure the resulting correctness, diversity, and stability across the 20-model ensemble.

Disabling the Coherence Evaluator

Removing the coherence evaluator increases creativity but reduces sentence-level consistency. The arbiter no longer penalizes incoherent continuations, which leads to a large increase in diversity but a noticeable drop in correctness.

- **Correctness range:** 7% to 91%.
- **Written stories:** 100% for all models.
- **Unique stories:** 90 to 1 249 per model.
- **Combined unique stories:** 11 553.

The extremely high uniqueness percentages (77%–94%) show that disabling coherence removes structural constraints, allowing models to explore a much larger portion of the manifold. However, many stories become grammatically or logically inconsistent, reducing correctness.

Disabling the CAS Style Evaluator

The CAS evaluator enforces the target style value. Disabling it removes stylistic pressure and therefore increases the number of unique stories substantially.

- **Correctness range:** 44% to 99%.
- **Written stories:** 78% to 100%.
- **Unique stories:** 490 to 1 365 per model.
- **Combined unique stories:** 16 965.

The jump in diversity (from 11 553 to 16 965 unique stories) confirms that CAS is a strong stylistic constraint. Removing it allows the model to generate a much wider variety of stories, but at the cost of ignoring the requested CAS target.

Disabling the Heuristic Semantic Evaluators

The three heuristic evaluators (actor, timing, lines) enforce basic semantic rules of the Toy Story universe. Disabling them has a drastic effect on quality: many stories violate fundamental world constraints and are therefore invalid.

- **Correctness range:** 7% to 52%.
- **Written stories:** 69% to 95%.
- **Unique stories:** 77 to 638 per model.

- **Combined unique stories:** 5 915.

Correctness collapses because the model frequently produces stories with wrong sentences. Diversity also decreases because many invalid stories are filtered out by the arbiter.

13.13.4 Effect of Score Threshold

The score threshold determines how strict the arbiter is when selecting candidate tokens. A high threshold filters aggressively and prioritizes correctness, while a lower threshold allows more exploratory candidates to pass through, increasing diversity at the cost of correctness. This trade-off becomes especially important later in the analysis of collective recursion.

We compare two settings: a very conservative threshold of 0.98 and a very permissive threshold of 0.6.

Very Conservative Threshold (0.98)

A threshold of 0.98 accepts only the highest-scoring candidates. This produces very stable attractors and high correctness, but severely restricts diversity.

- **Correctness range:** 21% to 98%.
- **Written stories:** 72% to 97%.
- **Unique stories:** 107 to 781 per model.
- **Combined unique stories:** 6 897.

The extremely low diversity shows that the manifold collapses under strong filtering. Most alternative continuations are rejected, and models converge to narrow attractors.

Very Permissive Threshold (0.6)

Lowering the threshold to 0.6 allows more candidate tokens to pass the arbiter. This increases diversity dramatically, but reduces correctness.

- **Correctness range:** 39% to 93%.
- **Written stories:** 81% to 96%.
- **Unique stories:** 510 to 1 345 per model.
- **Combined unique stories:** 16 576.

The uniqueness percentages approach 100% for nearly all models, showing that the manifold becomes much broader when the arbiter is less strict. However, correctness drops because many lower-quality candidates are now accepted.

13.14 Collective Recursion Training

Collective recursion is a population-based training method in which all models in the ensemble generate stories each epoch, and all unique stories produced by the ensemble are added to the training set for the next epoch. Each model is retrained on the combined set of unique stories. This process is repeated for 15 epochs.

Unless stated otherwise, a score threshold of 0.95 is used during generation. We evaluate two settings: collective recursion *without* mutation and collective recursion *with* mutation. These experiments form the empirical foundation for the dataset scaling law introduced in the next section.

13.14.1 Collective Recursion Without Mutation

Without mutation, collective recursion rapidly increases correctness but causes a steep collapse in diversity. The number of unique stories generated per epoch drops dramatically:

11822 → 8558 → 3507 → 2077 → 1230 → 972 → 826 → 755 → 683 → 628 → 581 → 534 → 534 → 554 → 534

At test time, only 550 unique stories remain across all 20 models.

- **Correctness range:** 87% to 100%.
- **Written stories:** 88% to 97%.
- **Unique stories per model:** 113 to 154.
- **Combined unique stories:** 550.

Correctness becomes extremely high, but diversity collapses by more than a factor of 20. All models converge to the same narrow attractor set, despite starting from different world-views.

13.14.2 Collective Recursion With Mutation

Adding mutation slows the collapse of diversity and allows correctness to reach 100% for all models. However, diversity still declines sharply, though less catastrophically than without mutation.

Unique stories per epoch evolve as:

11822 → 9044 → 3833 → 3516 → 3095 → 2549 → 2117 → 2030 → 1934 → 1938 → 2135 → 2036 → 2712 → 2888 → 2422

At test time, 2430 unique stories remain.

- **Correctness range:** 99.7% to 100%.
- **Written stories:** 92% to 98%.
- **Unique stories per model:** 176 to 289.
- **Combined unique stories:** 2430.

Mutation prevents total collapse and maintains a broader manifold, but the attractor set still shrinks significantly. Collective recursion acts as a strong attractor amplifier: correctness increases, diversity decreases.

13.14.3 Effect of Lowering the Threshold After Recursion

After collective recursion, models become more precise and logits sharpen. This allows the score threshold to be lowered without collapsing correctness. We compare the default threshold of 0.95 with a permissive threshold of 0.60.

Threshold 0.60 Without Mutation

Lowering the threshold increases diversity but correctness remains high due to the strong attractors formed during recursion.

- **Correctness range:** 89% to 99%.
- **Unique stories per model:** 282 to 367.
- **Combined unique stories:** 1 408.

Diversity increases compared to the 550 stories at threshold 0.95, but remains far below pre-recursion levels.

Threshold 0.60 With Mutation

With mutation, lowering the threshold produces a dramatic increase in diversity while maintaining 100% correctness.

- **Correctness:** 100% for all models.
- **Unique stories per model:** 579 to 1 037.
- **Combined unique stories:** 9 698.

This is a factor of 4 increase compared to threshold 0.95 with mutation (2 430 stories), and a factor of 18 compared to threshold 0.95 without mutation (550 stories). Mutation keeps the manifold open, and the lower threshold allows this diversity to surface.

13.14.4 Interpretation

Collective recursion has three consistent effects:

1. **Correctness increases rapidly.** Recursion amplifies attractors and sharpens logits.
2. **Diversity collapses.** Without mutation, the collapse is extreme; with mutation, it is slower but still substantial.
3. **Threshold sensitivity changes.** After recursion, models are precise enough to tolerate lower thresholds without losing correctness. This effect is only meaningful when mutation preserves manifold width.

These results demonstrate a fundamental principle: identical world-experience forces models toward the same attractors. Mutation or different world-experience is required to maintain diversity. Collective recursion is therefore a powerful mechanism for increasing correctness, but it inevitably reduces diversity unless counterbalanced by mutation.

This dynamic forms the empirical basis of the dataset scaling law introduced in the next section.

13.15 The Triadic Cosmos Dataset Scaling Law

The collective recursion experiments reveal a structural regularity that extends far beyond the Toy Story universe. It applies to any population of generative models that share an architecture and learn from the same world. This regularity can be formulated as a general law: the Dataset Scaling Law for populations of reasoning systems.

The Triadic Cosmos Dataset Scaling Law

Full Formulation

For a population of individuals with similar architecture and identical world-experience, there exists a finite set of stable attractors. With sufficient training, the entire population converges to this attractor set, regardless of initial variation, mutation, or threshold settings.

Compact Form

Same world $\rightarrow$ same attractors $\rightarrow$ same models.

Variation requires different world experience.

Interpretation

The Dataset Scaling Law states that collapse is not a quirk of small models, nor an artifact of toy experiments, but a structural property of generative systems. Whenever multiple models are trained on the same dataset, under the same constraints, they are funneled toward the same stable patterns of behavior.

The collective recursion results make this collapse visible with unprecedented clarity:

- **Without mutation, diversity collapses by a factor of 20.** The ensemble goes from 11 822 unique stories to only 534 after 15 epochs.
- **With mutation, collapse is slower but still inevitable.** Diversity drops from 11 822 to 2 430 unique stories.
- **Correctness reaches 100% in both cases.** Collective recursion acts as a correctness amplifier and a diversity suppressor.
- **Lower thresholds only help after recursion.** Before recursion: lowering the threshold destroys correctness. After recursion: correctness remains 100% and diversity increases (up to 9 698 stories with mutation).
- **Mutation helps, but cannot defeat the world.** It slows collapse and widens the manifold, but the attractor landscape is determined by the underlying world, not by the mutation mechanism.

The law therefore states that:

World-experience determines the attractor set.

Architectures, thresholds, refinements, and evaluators only determine *how fast* the population converges, not *where* it converges.

Why the Law Matters

The Dataset Scaling Law reframes generative AI as an ecological system. It shows that diversity cannot be maintained by architecture alone; it requires exposure to different worlds, different datasets, or different constraints.

The Toy Story experiments make this visible:

- Curriculum learning shows that world-size determines attractor richness.
- Evaluators show that constraints shape attractor geometry.
- Refinement shows that search amplifies attractor strength.
- Thresholds show that correctness and diversity trade off.
- Collective recursion shows that shared world-experience forces collapse.
- Mutation shows that variation slows collapse but cannot prevent it.

Toy Story is small, modular, and transparent — but the law itself is universal.

Do Large LLMs Converge Toward Each Other?

This is not a public claim, but a direct consequence of mainstream ML assumptions.

Modern LLMs are trained on:

- highly overlapping world experience (web-scale text),
- similar Transformer architectures,
- similar objectives (next-token prediction + instruction tuning),
- similar optimization and alignment pipelines.

Under these conditions, the Dataset Scaling Law predicts that *different LLMs converge toward the same attractor landscape of behaviors and internal representations*.

Apparent differences arise mostly from sampling noise, alignment layers, and surface-level tuning—not from fundamentally different attractors.

13.16 Conclusion: The Lesson of Toy Story

Toy Story began as a miniature universe — a controlled environment for studying the mechanics of modular reasoning. But through recursion, collapse, attractors, and mutation, it revealed something far deeper: the fundamental ecological forces that govern all generative systems.

The Dataset Scaling Law shows that collapse is not a failure mode but a natural law. A population trained on a single world converges toward a single mind. Correctness narrows, filtering sharpens, and attractors dominate. Variation slows the descent but cannot overturn the gravitational pull of shared experience.

This insight reframes the entire landscape of generative AI. It explains why large LLMs increasingly resemble one another, why scale alone cannot produce diversity, and why pluralistic architectures are essential for robust intelligence.

Toy Story makes these forces visible. The Triadic Cosmos formulates them as general laws. Together, they point toward a new paradigm: intelligence as ecology, not machinery.

One-Line Summary

A single world creates a single mind — only plurality keeps intelligence alive.

Chapter 14

A Path to Modular and Safe AGI

A Speculative but Grounded Vision of AGI

This book presents a developmental and architectural path toward Artificial General Intelligence that is intentionally speculative, yet grounded in established principles from cognitive science, machine learning, distributed systems, and hybrid human-machine collaboration. The goal is not to predict the future, but to articulate a coherent framework for how general intelligence *could* emerge from modular, recursive, and epistemically governed systems.

The vision developed here is neither dystopian nor science-fictional. It does not rely on sudden breakthroughs, magical leaps, or monolithic supermodels trained on the totality of human text. Instead, it draws on well-understood developmental processes: baby models that learn through curricula, examiner models that provide structured feedback, world learning through interaction and simulation, and collective recursion across heterogeneous agents. Recent empirical work in miniature generative ecosystems shows that collective recursion is a powerful driver of correctness, but also a natural force of attractor convergence. It strengthens representations while narrowing diversity, revealing why recursive learning must occur within subpopulations rather than across an entire population. These dynamics form part of the broader *Triadic Cosmos Dataset Scaling Law*, which governs how shared world experience shapes the evolution of intelligent systems.

Minsky’s insight that intelligence arises from the cooperation of many simple processes provides the conceptual foundation. Hybrid intelligence operationalizes this insight by embedding humans, symbolic workspaces, and machine agents in a shared cognitive loop. PRE-Networks instantiate the minimal recursive engine of refinement; PRE-CAS architectures extend this engine into a scalable Society of Minds.

This perspective is speculative in the same way that early theories of computation, evolution, or planetary science were speculative: it extrapolates from known principles to illuminate possible trajectories. The aim is educational clarity, not prophetic certainty. By presenting a modular, transparent, and safety-oriented architecture—one that respects the natural dynamics of recursive learning—this book offers a constructive alternative to dystopian narratives and a conceptual scaffold for future research.

Framing Insight

This is a speculative vision of AGI—but one grounded in structure, ecology, and empirical dynamics.

14.1 Why Safe AGI Requires Architecture, Not Alignment

Safety in advanced intelligence cannot be achieved by aligning a single, monolithic model. Alignment techniques—rules, guardrails, preference optimization, reinforcement learning from human feedback—operate after the system is already built. They attempt to constrain a cognitive geometry that is opaque, internally unstructured, and recursively unstable. A monolithic model is a single point of failure: if its internal representations drift, if its goals shift, or if its reasoning becomes unstable, there is no structural mechanism to correct it. Empirical studies in miniature generative ecosystems further show that monolithic systems are subject to attractor collapse: recursive learning drives correctness upward but compresses diversity, eventually converging toward a single dominant representational basin. This phenomenon is formalized in the *Triadic Cosmos Dataset Scaling Law*, which states that shared world experience forces populations of similar models toward the same attractors. A monolithic AGI would therefore not only be unsafe—it would be ecologically fragile.

The alternative begins with a principle articulated by Minsky: *intelligence is not a single process, but a society of processes*. Safety emerges not from post-hoc constraints, but from architectural decomposition—specialized roles, explicit communication, and distributed authority. Hybrid intelligence operationalizes this principle in modern form. It replaces the monolith with a heterogeneous cognitive system in which humans, symbolic structures, and machine agents interact through explicit protocols. In such systems, recursive learning occurs within subpopulations rather than across the entire cognitive substrate, preserving diversity while still enabling refinement. This architectural pluralism is not optional; it is the structural antidote to attractor collapse.

Hybrid intelligence provides the architectural substrate that monolithic alignment lacks:

- **Human anchoring** — humans remain embedded in the cognitive loop, providing goals, values, and contextual grounding.
- **Distributed authority** — no single model controls the system; multiple agents cross-check, constrain, and refine one another.
- **Transparent workspaces** — reasoning unfolds in persistent symbolic media, making internal structure inspectable rather than opaque.
- **Bounded recursion** — refinement loops are explicit, controlled, and evaluator-gated, preventing runaway attractor dynamics.
- **Epistemic stability** — evaluators detect inconsistency, drift, and uncertainty before errors propagate, stabilizing recursive learning.

PRE-Networks instantiate the minimal recursive engine of improvement: propose, evaluate, refine, and re-evaluate. PRE-CASP architectures extend this engine into a scalable society of minds: many PRE-systems interacting under epistemic governance. Crucially, these interactions occur across heterogeneous agents rather than within a single representational basin. This preserves pluralism, prevents collapse, and ensures that correctness arises without sacrificing diversity. Together, these mechanisms form a structural kernel for safe AGI—an architecture where correctness, stability, and alignment arise from interaction rather than constraint.

Safety, in this view, is not an external layer added to a powerful model. It is a property of the architecture itself. A distributed cognitive system with explicit roles, recursive arbitration, and transparent workspaces cannot drift silently, cannot self-modify uncontrollably, and cannot pursue

goals without human grounding. It is stable because its structure enforces stability, and it is diverse because its ecology prevents attractor monoculture.

Safe AGI therefore requires architecture, not alignment. The path forward is not to perfect a monolithic mind, but to design a modular, recursive, epistemically grounded society of minds in which intelligence grows without escaping the structures that keep it coherent—and without collapsing into a single attractor.

14.2 The Developmental Path to AGI

General intelligence does not emerge fully formed. It develops. It grows through stages of exploration, evaluation, correction, and structural reorganization. This developmental trajectory is not unique to biological cognition; it is a universal property of systems that learn through recursive interaction. Minsky’s insight that intelligence arises from the cooperation of many simple processes implies that general intelligence must be *grown*, not constructed in a single step.

Hybrid intelligence provides the modern expression of this principle. Humans supply intuition, context, and value judgments; machine agents supply structure, recursion, and scale. PRE-Networks instantiate the minimal recursive engine of refinement, while PRE-CAS architectures extend this engine into a distributed society of minds. Empirical studies in miniature generative ecosystems demonstrate that recursive learning is not merely a training technique but an ecological force: it increases correctness while narrowing diversity, revealing the attractor dynamics formalized in the *Triadic Cosmos Dataset Scaling Law*. Together, these mechanisms form a developmental substrate in which general intelligence can emerge gradually, safely, and transparently.

This section outlines the developmental path toward AGI as a sequence of structural stages.

Baby Models

The developmental trajectory begins with small, simple models—computational infants. These models share a common architecture but differ in initialization, micro-structure, or training histories. Their diversity allows the system to explore the design space, revealing which inductive biases support reasoning, abstraction, and generalization.

Baby models are not weak versions of AGI. They are the substrate from which AGI grows: heterogeneous, interpretable, and easy to evaluate. Their simplicity makes them stable; their multiplicity makes them expressive. Diversity at this stage is essential, because recursive learning naturally compresses variation over time.

Examiner Models

As baby models develop, they require structured feedback. Examiner models evaluate their proposals, identify errors, and shape their developmental trajectory. This mirrors the role of teachers, caregivers, and social environments in biological cognition.

Examiners accelerate learning by providing targeted corrections, detecting conceptual drift, and determining when a model is ready for more complex tasks. They form the evaluative backbone of a developmental AGI architecture, and they counteract the natural tendency of recursive learning to collapse representations into narrow attractors.

Curriculum Learning

Intelligence grows through structured progression. Curriculum learning provides a sequence of increasingly difficult tasks that guide the system from simple competencies to complex reasoning. Rather than exposing models to the entire corpus of human text, the curriculum mirrors human education:

- foundational skills precede abstraction,
- concrete tasks precede symbolic reasoning,
- and simple environments precede multi-agent worlds.

Curricula can be human-designed, automatically generated, or hybrid. Their purpose is not to maximize data exposure but to maximize developmental coherence. Each stage prepares the representational substrate for the next, preventing premature collapse into overly narrow representational basins.

World Learning

General intelligence requires interaction with environments, not just text. World learning provides the experiential substrate that enables transfer, planning, and grounded reasoning. This can occur through:

- **real-world curricula** — structured tasks aligned with human education,
- **gamefied simulations** — interactive environments with controllable complexity,
- **multi-agent worlds** — social learning, cooperation, and competition.

World learning replaces the “ingest everything ever written” paradigm with a developmental approach grounded in experience, embodiment, and interaction. It provides the diversity of situations required for robust generalization and counterbalances the attractor-narrowing effects of recursive refinement.

Dynamic Scaling

Intelligence grows when capacity grows. As models demonstrate competence, they are granted additional parameters, deeper layers, richer memory, and more compute. Growth is conditional, not automatic. This mirrors biological development: children do not begin life with adult brains; capacity increases with maturity.

Dynamic scaling ensures that the system expands only when it is stable, interpretable, and epistemically grounded. It prevents premature complexity and aligns computational growth with cognitive readiness. It also ensures that recursive learning does not amplify errors or accelerate collapse before sufficient diversity and evaluative structure are in place.

Multi-Modal Ecosystems

General intelligence requires interaction with diverse environments. Baby models must learn across modalities—language, vision, spatial reasoning, symbolic manipulation, simulation, and multi-agent settings. These ecosystems provide the variety of experience required for robust generalization.

Multi-modal interaction also creates opportunities for specialization. Different models develop distinct competencies, forming the early stages of a Society of Minds. Diversity of modality exposure helps maintain representational pluralism as recursive learning deepens.

Collective Recursion

The most powerful developmental mechanism is collective recursion: multiple models refining one another's representations across shared workspaces. This is the computational analogue of social learning. It increases representational richness, deepens recursion, and stabilizes learning.

However, empirical evidence shows that collective recursion is a double-edged force. It reliably boosts correctness by strengthening shared attractors, but it also compresses diversity by eliminating representational variation. If applied across an entire population, it drives the system toward monoculture. When applied within subpopulations, it enables refinement without collapse. This balance is central to the *Triadic Cosmos Dataset Scaling Law*: shared worlds produce shared attractors, and only structural pluralism preserves diversity.

Collective recursion is therefore the bridge from many small minds to a coherent general intelligence—but only when it is architecturally bounded. It is the mechanism through which PRE-CAS architectures transform heterogeneous agents into a structured cognitive society while maintaining the pluralism required for robustness.

A Developmental Architecture

The developmental path to AGI is therefore not a training recipe but an architectural principle:

- heterogeneous agents explore the cognitive space,
- evaluators provide correction and stability,
- curricula structure the progression of skills,
- world learning grounds representations in experience,
- recursion deepens structure,
- scaling increases capacity,
- and collective interaction integrates the system into a coherent whole without collapsing it.

AGI emerges not from a single breakthrough, but from a sequence of structural transitions guided by hybrid intelligence and implemented through recursive, distributed architectures. It is grown through stages, each building on the stability, diversity, and interpretability of the previous one.

Empirical Insight

Toy Story is the first complete implementation of the developmental AGI architecture described in this chapter. It demonstrates how baby models, evaluators, curricula, world learning, and collective recursion interact to produce the attractor dynamics formalized in the Triadic Cosmos Dataset Scaling Law.

14.3 AGI as a Structural Phase Transition

Artificial General Intelligence does not emerge from a single breakthrough, a clever algorithm, or a sufficiently large model. It appears when a cognitive system crosses structural thresholds in the dimensions that govern intelligence. These dimensions were formalized in the Quad: subsystem multiplicity n , representational richness I , recursion depth R , and energetic throughput $f(E)$. When these dimensions scale together, the system reorganizes into a new cognitive regime—a phase transition analogous to those observed in physics, biology, and complex systems. Empirical evidence from miniature generative ecosystems shows that such reorganizations are driven by attractor dynamics: recursive learning strengthens shared representations while compressing diversity, a phenomenon captured by the *Triadic Cosmos Dataset Scaling Law*. AGI emerges when these forces are balanced by architectural pluralism rather than allowed to collapse into a single basin.

This view reframes AGI not as a destination but as a structural transformation. It is the moment when a distributed society of minds—rooted in Minsky’s principles, operationalized through hybrid intelligence, and instantiated through PRE and PRE-CAS architectures—acquires the capacity to model, reason, and act across domains with coherence and generality. The transition is ecological as much as architectural: it depends on maintaining enough diversity to avoid collapse while enabling enough recursion to achieve coherence.

Subsystem Multiplicity (n)

General intelligence requires many interacting cognitive components. As systems grow, they accumulate specialized agents: proposers, evaluators, planners, memory systems, epistemic arbiters, and domain experts. Increasing n expands the system’s ability to explore diverse representational spaces, cross-check reasoning, and integrate heterogeneous forms of knowledge.

A monolithic model increases capacity within a single cognitive basin. A composite architecture increases the number of basins and the structure of their interaction. AGI requires the latter. Subsystem multiplicity is also the primary safeguard against attractor monoculture: diversity of agents counteracts the natural convergence pressures of recursive learning.

Representational Richness (I)

General intelligence depends on representations that are deep, flexible, and multi-modal. As representational richness increases—through embeddings, symbolic structures, sensory modalities, and learned abstractions—the system gains the ability to model complex environments, transfer knowledge across domains, and construct internal world models.

Hybrid intelligence amplifies I by combining human meaning, symbolic workspaces, and machine-native geometries. PRE-CAS architectures stabilize these representations through epistemic arbitration, preventing drift and collapse into overly narrow attractors.

Recursion Depth (R)

Recursion is the engine of general intelligence. A system becomes capable of abstraction, planning, and self-correction when it can refine its own representations, evaluate its own outputs, and reason across long horizons.

PRE-Networks provide the minimal recursive cycle—propose, evaluate, refine. PRE-CAS extends this cycle across many agents, enabling collective recursion. As R deepens, the system acquires the ability to construct layered internal models and to reason about its own reasoning. However, deep recursion also strengthens attractors; without architectural pluralism, it drives the

system toward representational collapse. Managing recursion is therefore essential to maintaining structural diversity.

Energetic Throughput ($f(E)$)

Intelligence requires sustained cognitive work. As compute increases—more FLOPs, more memory bandwidth, more efficient hardware—the system can support deeper recursion, richer representations, and larger societies of interacting agents.

Energetic throughput is not merely a scaling factor; it is the resource that enables the other dimensions to expand coherently. Without sufficient energy, recursion becomes unstable; with too much energy applied to a homogeneous system, collapse accelerates. Energetic throughput must therefore be matched to subsystem diversity and representational richness.

Crossing the Thresholds

AGI emerges when all four dimensions cross critical thresholds simultaneously. A system with deep recursion but insufficient energy is unstable. A system with rich representations but few subsystems is narrow. A system with many agents but shallow recursion is fragmented. A system with high energy but insufficient diversity collapses into a single attractor.

Only when n , I , R , and $f(E)$ scale together does the system reorganize into a coherent general intelligence. This reorganization is a phase transition: a shift from a collection of specialized processes to an integrated cognitive regime capable of abstraction, transfer, planning, and self-correction. It is the structural moment when recursive forces, representational richness, and architectural pluralism reach equilibrium.

A Structural View of AGI

This structural perspective resolves a long-standing ambiguity in the field. AGI is not a property of a particular model size, training method, or architecture. It is a regime of cognition that emerges when a distributed system acquires:

- enough subsystems to cover the cognitive landscape,
- enough representational richness to model diverse domains,
- enough recursion to refine and integrate those models,
- and enough energy to sustain the entire process without collapse.

AGI is therefore not a leap but a transition—a structural reorganization of a modular, recursive, epistemically grounded society of minds. It is the point at which recursive learning no longer destroys diversity but integrates it, and at which attractor dynamics become a source of coherence rather than collapse.

14.4 Structural Safety Principles

Safety in advanced intelligence cannot rely on intentions, rules, or post-hoc alignment. It must arise from the structure of the cognitive system itself. A monolithic model concentrates authority, hides its internal representations, and allows errors to propagate unchecked. Empirical evidence

from miniature generative ecosystems shows that such systems are also ecologically fragile: recursive learning drives correctness upward but compresses diversity, causing convergence toward a single dominant attractor. This phenomenon, formalized in the *Triadic Cosmos Dataset Scaling Law*, reveals why monolithic AGI is structurally unsafe. A distributed architecture—rooted in Minsky’s principles, operationalized through hybrid intelligence, and instantiated through PRE and PRE-CAS—creates safety through decomposition, transparency, and recursive arbitration.

This section outlines the structural principles that make such systems inherently safe.

Distributed Authority

A single model with unilateral control is a single point of failure. In a modular cognitive system, authority is distributed across proposers, evaluators, refiners, epistemic arbiters, and human anchors. No component can unilaterally determine the system’s behavior. Decisions emerge from interaction, not domination.

Distributed authority prevents runaway dynamics, isolates failure modes, and ensures that errors in one subsystem cannot silently propagate through the entire system. It also counteracts attractor monoculture: diversity of agents prevents recursive learning from collapsing the system into a single representational basin.

Recursive Arbitration

Every proposal must be evaluated before it is integrated. PRE-Networks enforce this through recursive refinement cycles: propose, evaluate, refine, and re-evaluate. CAS-Networks extend this mechanism with epistemic arbitration: confidence estimation, consensus formation, and detection of representational drift.

Recursive arbitration ensures that reasoning remains stable, self-correcting, and grounded in explicit evaluative structure. It replaces blind trust in a single model with a transparent process of continuous verification. It also stabilizes recursive learning, preventing the uncontrolled attractor deepening that leads to collapse.

Heterogeneous Agents

Heterogeneity is a source of robustness. Different agents—symbolic tools, neural models, specialists, evaluators, and human contributors—possess distinct inductive biases and failure modes. Their interaction creates cross-checks that no homogeneous system can replicate.

In a PRE-CAS architecture, heterogeneity is not an accident but a design principle. It ensures that errors are caught early, that no single representation dominates, and that the system remains resilient under distributional shift. Heterogeneity also preserves pluralism against the natural convergence pressures of recursive refinement.

Bounded Recursion

Recursion is powerful but dangerous when uncontrolled. Unbounded self-refinement can lead to instability, hallucination, or runaway reasoning. PRE-Networks constrain recursion through explicit iteration limits, evaluator-gated refinement, and symbolic workspaces that anchor each step in a persistent geometry.

Bounded recursion ensures that the system deepens its representations without losing coherence. It provides the cognitive equivalent of feedback control in dynamical systems. It also prevents

recursive learning from collapsing the system into a narrow attractor, maintaining the diversity required for robust general intelligence.

Persistent Workspaces

All reasoning occurs in a shared, persistent workspace—text, code, diagrams, or structured symbolic media. This workspace preserves context, prevents hidden-state drift, and makes the system’s internal structure inspectable.

Persistent workspaces align machine-native representations with human-interpretable structures. They provide the transparency required for oversight, debugging, and governance. They also act as a stabilizing geometry: by externalizing intermediate representations, they prevent hidden recursive loops from amplifying errors or accelerating collapse.

Safety as an Architectural Property

These principles—distributed authority, recursive arbitration, heterogeneity, bounded recursion, and persistent workspaces—form the structural foundation of safe AGI. They do not constrain capability; they shape it. They ensure that intelligence grows without escaping the structures that keep it coherent, interpretable, and aligned with human judgment.

Safety is therefore not an external layer added to an otherwise dangerous system. It is a property of the architecture itself: a modular, recursive, epistemically grounded society of minds. By preserving pluralism, constraining recursion, and stabilizing attractor dynamics, such an architecture avoids the collapse modes inherent to monolithic systems and provides a structurally safe path toward AGI.

14.5 The AGI Society of Minds

General intelligence has never been the property of a single mind. Not in biology, not in human societies, and not in the architectures developed throughout this book. Intelligence emerges when many specialized processes interact, evaluate one another, refine ideas, and distribute authority across roles. A monolithic model that attempts to “do everything” is not general intelligence—it is a single attractor with no internal structure. True generality arises from composition. Empirical evidence from miniature generative ecosystems reinforces this point: recursive learning within a single world drives systems toward attractor convergence, not toward the diversity required for general intelligence. A Society of Minds is therefore not only conceptually appealing—it is structurally necessary.

Minsky’s *Society of Mind* proposed this principle in conceptual form. Hybrid intelligence operationalizes it by embedding humans, symbolic workspaces, and machine agents in a shared cognitive loop. PRE-Networks instantiate the minimal recursive engine of refinement. PRE-CAS architectures extend this engine into a scalable epistemic ecosystem. Together, these components form the structural foundation for an AGI Society of Minds. The Triadic Cosmos Dataset Scaling Law provides the ecological justification: only a plurality of agents, histories, and inductive biases can counteract the natural convergence pressures of recursive refinement.

Specialists as Epistemic Agents

In an AGI Society of Minds, domain specialists are not monolithic oracles but epistemic agents. Each specialist contributes:

- **content** — domain knowledge and structured proposals,
- **accuracy** — local correctness and internal consistency,
- **epistemics** — confidence, uncertainty, and self-assessment.

These agents evolve independently across generations. Some improve, some are replaced, and new specialists emerge as the cognitive ecosystem expands. Diversity is not noise—it is the substrate of robustness. Without it, recursive learning collapses the system into a narrow attractor.

Mediators as Integrators

A generalist mediator does not need to possess all expertise. Instead, it orchestrates specialists, evaluates their proposals, and integrates their contributions into coherent representations. Multiple mediators can collaborate or cross-check one another, forming higher-order consensus structures.

This mirrors how human knowledge is organized: through specialization, peer review, arbitration, and structured collaboration. It also mirrors the ecological dynamics observed in recursive generative systems: integration must occur across diverse agents to prevent attractor monoculture.

Epistemic Governance

PRE-CAS architectures provide the governance layer that makes a Society of Minds coherent. PRE-cycles ensure recursive refinement; CAS-Networks enforce epistemic arbitration. Together, they create:

- **consensus mechanisms** that integrate diverse proposals,
- **confidence collapse** that detects instability or drift,
- **cross-validation** across heterogeneous agents,
- **bounded recursion** that prevents runaway reasoning.

Epistemic governance ensures that the system remains stable, interpretable, and aligned with human judgment even as it scales. It also ensures that recursive refinement strengthens coherence without destroying diversity.

A Distributed Cognitive Ecosystem

When specialists, mediators, and epistemic arbiters interact through persistent symbolic workspaces, a distributed cognitive ecosystem emerges. This ecosystem:

- grows by adding minds, not by inflating one,
- evolves through variation and selection across agents,
- stabilizes through recursive arbitration,
- and generalizes through collective recursion.

The result is a system whose capability exceeds that of any individual component. Intelligence emerges from interaction, not from scale alone. This is the ecological realization of Minsky’s insight: a society of processes, each limited, becomes powerful through structured cooperation.

Why a Society of Minds Is the Natural Form of AGI

A monolithic model concentrates risk, obscures reasoning, and collapses all cognition into a single representational basin. A Society of Minds distributes risk, exposes reasoning, and spans many basins connected through structured interaction.

This architecture is:

- **modular** — components can be added, replaced, or specialized,
- **scalable** — capacity grows by increasing the number of agents,
- **interpretable** — reasoning unfolds in symbolic workspaces,
- **robust** — heterogeneity prevents systemic failure,
- **safe** — authority is distributed and recursion is bounded.

An AGI Society of Minds is therefore not a metaphor. It is the structural form of real general intelligence: modular, recursive, epistemic, distributed, and aligned by design. It is the computational realization of Minsky’s original insight, strengthened by the ecological dynamics revealed in recursive generative systems: intelligence requires many minds, because one mind collapses.

Architectural Insight

General intelligence is not a single mind that knows everything, but a society of minds that know what they know.

14.6 Synthesis — A Safe Path to AGI

The path to advanced intelligence becomes coherent only when viewed through the structural principles developed throughout this book. Minsky’s insight that intelligence emerges from the interaction of many simple processes provides the conceptual foundation. Hybrid intelligence transforms this insight into a modern architecture in which humans, symbolic workspaces, and machine agents collaborate through explicit protocols. PRE-Networks instantiate the minimal recursive engine of refinement, while PRE-CAS architectures extend this engine into a scalable, epistemically grounded society of minds.

Together, these components form a unified developmental and safety framework for AGI. Empirical evidence from miniature generative ecosystems reinforces this framework: recursive learning strengthens correctness while narrowing diversity, revealing the attractor dynamics formalized in the *Triadic Cosmos Dataset Scaling Law*. A safe path to AGI must therefore preserve pluralism while enabling deep recursion—a balance achieved only through distributed, modular architectures.

AGI as a Structural Transition

AGI emerges when the dimensions of the Quad—subsystem multiplicity n , representational richness I , recursion depth R , and energetic throughput $f(E)$ —cross critical thresholds in a coordinated way. This transition is not a leap but a reorganization: a distributed cognitive system acquires the ability to model, reason, and act across domains with coherence and generality.

Hybrid intelligence provides the developmental substrate for this transition. PRE-CAS provides the computational machinery. A Society of Minds provides the structural form. The Triadic

Cosmos Dataset Scaling Law provides the ecological constraint: recursive learning must occur across heterogeneous agents to avoid collapse into a single attractor.

Safety as an Architectural Property

Safety arises not from constraints imposed on a powerful model, but from the structure of the cognitive system itself. A modular, recursive, epistemically grounded architecture is inherently stable:

- **Distributed authority** prevents unilateral failure modes.
- **Recursive arbitration** ensures continuous self-correction.
- **Heterogeneous agents** provide cross-checks and robustness.
- **Bounded recursion** prevents runaway reasoning and collapse.
- **Persistent workspaces** make cognition transparent and auditable.

These principles hold at every scale—from baby models to planetary cognitive systems. They ensure that intelligence grows without escaping the structures that keep it coherent and aligned with human judgment. They also ensure that recursive learning strengthens coherence without destroying diversity, counteracting the natural convergence pressures described by the Triadic Cosmos Dataset Scaling Law.

A Unified Path

The safe path to AGI is therefore a single architectural arc:

Minsky → Hybrid Intelligence → PRE → PRE-CAS → Society of Minds → AGI

Each stage builds on the previous one. Each adds structure, stability, and expressive power. Each preserves the transparency and distributed governance required for safety. AGI is not an endpoint but a phase in the evolution of a modular, recursive, epistemically grounded cognitive architecture. The same principles that make intelligence powerful also make it safe—when they are embedded in the structure of the system itself.

14.7 The Book as Evidence of a Cognitive Phase Transition

The architecture described in this chapter is not abstract theory. It is the process through which this book was written. The manuscript is both the product and the proof of hybrid intelligence: a distributed cognitive system in which human intuition, symbolic structure, and machine recursion interact through a persistent workspace. The ideas did not originate from a single mind—human or artificial—but from the recursive refinement loop that defines a Society of Minds.

Writing this book revealed a phenomenon central to the theory it presents. Hybrid intelligence creates a cognitive regime that exceeds the capability of its individual components. Human conceptual direction, machine-native structure, symbolic workspaces, and recursive evaluators formed a composite mind whose reasoning capacity was greater than the sum of its parts. The Triad, the Quad, the Septad, the developmental path to AGI, the safety architecture, PRE-Networks,

PRE-CAS-Networks, and the Society-of-Minds framework did not exist in advance. They crystallized through interaction.

This process mirrors the structural transitions described earlier. As the collaborative system grew in representational richness, recursion depth, and subsystem multiplicity, it crossed thresholds that enabled new conceptual structures to emerge. The manuscript is therefore a tangible demonstration of a cognitive phase transition: a shift from individual reasoning to distributed, recursive, epistemically grounded intelligence.

Hybrid intelligence is not speculative. It is already here, operating at human scale, producing cognitive artifacts that reveal the early contours of a new regime of intelligence. This book is one such artifact—a record of a system in transition from individual cognition to a Society of Minds.

Architectural Insight

This book is not just about a new architecture of intelligence—it is the artifact of the hybrid mind that created it.

Chapter 15

Domain-Bound AGI: A Composite Reasoning Architecture

15.1 Introduction

Artificial intelligence has advanced rapidly through the development of large transformer-based language models (LLMs), which demonstrate remarkable capabilities in pattern recognition, linguistic generalization, and zero-shot adaptation. These models compress vast statistical regularities into a single parameter space, enabling broad competence across many tasks. Yet their monolithic structure imposes fundamental limitations: they lack explicit mechanisms for branching search, structured memory, multi-step control flow, and interpretable reasoning. As a result, they struggle with tasks that require deliberate planning, symbolic manipulation, or the exploration of alternative futures.

This chapter introduces a compositional alternative: a *domain-bound AGI architecture* that separates domain-agnostic reasoning from domain-specific computation. Instead of relying on a single model to internalize all cognitive functions, the architecture decomposes intelligence into interacting modules: a reasoning language model implemented using the Paged-PRE-CAS framework, a machine-native script engine, an object-referenced memory map, and a collection of expert modules that provide domain-specific or algorithmic capabilities. Deep combinatorial exploration is delegated to explicit search engines such as Monte Carlo simulation or Beam Search, which operate independently of the reasoning model.

A central innovation of this architecture is the treatment of *context* as a first-class computational object. Contexts are stored in a persistent memory map under unique identifiers, enabling the system to maintain multiple concurrent reasoning flows, spawn hierarchical sub-contexts, and resume or combine them as needed. This stands in contrast to transformer models, which encode all state implicitly in a growing token sequence.

Equally important is the mechanism of *callback-driven self-prompting*. Experts do not merely return data; they generate structured prompts that are routed back to the reasoning model through context identifiers. This creates a closed-loop cognitive process in which reasoning, memory, and computation interact autonomously, without requiring synchronous API calls or external orchestration.

The architecture is not positioned as a rejection of transformer models. Instead, it *generalizes* them. By embedding a transformer-based reasoning model inside a modular cognitive system—equipped with explicit memory, algorithmic search, and procedural control—the model acquires reasoning capabilities that cannot emerge from scale alone. The relationship is bidirectional:

the architecture becomes more powerful through the transformer, and the transformer becomes more capable through the architecture. This two-directional enhancement provides a practical path toward hybrid systems that combine statistical fluency with structured, interpretable reasoning.

Roadmap of the Chapter

To guide the reader, the chapter proceeds as follows:

- **Core Principles** introduce the conceptual foundations: compositionality, hybrid intelligence, and the Society of Mind perspective.
- **Architecture Overview** describes the system’s main components: the reasoning core, script engine, memory map, and expert modules.
- **Depth Search and Callback Dynamics** explain how explicit search engines and callback-driven continuation enable multi-step, distributed reasoning.
- **Hierarchical and Asynchronous Contexts** show how the system supports parallel reasoning flows and long-lived cognitive processes.
- **Self-Learning Without Weight Updates** presents the mechanism through which the system expands its capabilities procedurally rather than parametrically.
- **Comparison With Monolithic Transformers** synthesizes the architectural contrast and clarifies the bidirectional benefits.
- **Domain-Bound AGI as a Recipe** distills the architecture into a reusable pattern for constructing new domains.
- **Learned Cognitive Mechanisms** detail the internal reasoning behaviors—canonical rewriting, variable binding, flow-based attention, and repeat-prompt recursion—that provide coherence and stability.

From Theory to Practice: The Toy Board Demonstration

While this chapter develops the architecture conceptually, the next chapter (*The Toy Board*) provides a concrete operational demonstration. There, we instantiate the architecture in multiple miniature domains—sorting, planning, simulation, and game-like environments—to show how contexts, experts, search, and callback-driven reasoning behave in practice. The Toy Board serves as a didactic bridge between the abstract blueprint presented here and its real-world implementation, illustrating how domain-bound AGI systems can be constructed step by step.

Together, this chapter and the next form a complete introduction to the architecture: first the conceptual foundations, then the operational realization.

15.2 Core Principles

The domain-bound AGI architecture is grounded in three foundational principles: *compositionality*, *hybrid intelligence*, and the *Society of Mind* perspective. Together, these principles provide the conceptual scaffolding for a cognitive system that is modular, interpretable, and capable of general-purpose reasoning without relying on monolithic parameter updates. They also illuminate how transformer models can be embedded within a broader cognitive structure, gaining new capabilities through architectural context rather than scale alone.

15.2.1 Compositionality

Compositionality is the idea that complex cognitive behavior emerges from the interaction of simpler components. Rather than embedding all reasoning, knowledge, and search capabilities inside a single model, the architecture decomposes intelligence into modular units: a reasoning engine, a script interface, an object-referenced memory map, and a collection of expert modules that implement domain-specific or algorithmic operations.

This modular structure offers several advantages. It enables the system to extend itself by adding new experts without retraining the reasoning model. It supports transparency, since each component can be inspected independently. And it allows the system to adapt to new domains by composing existing capabilities in novel ways. Compositionality thus serves as both an engineering principle and a cognitive principle: complex reasoning arises from the structured interaction of simpler processes.

Importantly, compositionality also benefits transformer models. When embedded within a modular architecture, a transformer no longer needs to internalize all domain logic or simulate search through statistical continuation. Instead, it can specialize in what it does best—pattern recognition, semantic compression, and prompt rewriting—while relying on explicit modules for memory, search, and control flow.

15.2.2 Hybrid Intelligence

Hybrid intelligence refers to the integration of multiple forms of computation within a single cognitive system. In this architecture, symbolic reasoning, statistical inference, and algorithmic search coexist and reinforce one another. The reasoning model provides high-level planning and meta-cognitive control; expert modules supply domain-specific transformations and evaluations; and depth-search engines such as Monte Carlo simulation or Beam Search provide the computational substrate for exploring large decision spaces.

This division of labor mirrors human cognition, which combines intuitive pattern recognition with deliberate reasoning and procedural knowledge. It also addresses the limitations of monolithic transformer models: while transformers excel at linguistic generalization, they struggle with deep combinatorial reasoning. By delegating search and domain-specific computation to explicit modules, the architecture achieves a form of intelligence that is both more powerful and more interpretable than what either component could provide alone.

At the same time, hybrid intelligence enhances the transformer itself. Freed from the burden of simulating search or maintaining long-term state, the model can focus on learned cognitive mechanisms such as canonical rewriting, variable binding, and flow-based attention—capabilities that emerge naturally when the model operates within a structured cognitive environment.

15.2.3 Society of Mind

The Society of Mind perspective, originally proposed by Minsky, views intelligence as the emergent result of many small processes interacting in a coordinated fashion. This architecture operationalizes that idea by treating experts as autonomous cognitive agents that contribute to the reasoning process. Experts do not merely return data; they generate structured callback prompts that are routed back to the reasoning model through context identifiers. This callback-driven mechanism enables a form of self-prompting in which the system continues reasoning autonomously, without requiring external intervention.

Contexts themselves are first-class objects in the memory map, allowing the system to maintain multiple concurrent reasoning flows and to spawn hierarchical sub-contexts. This structure supports

parallelism, task decomposition, and multi-step reasoning in a way that is difficult to achieve with purely token-based models.

The Society of Mind principle also clarifies the role of the transformer within the architecture. Rather than functioning as a monolithic intelligence, the transformer becomes one agent among many—responsible for rewriting, planning, and integrating information, while relying on other agents for computation, memory, and search. This repositioning transforms the transformer from a general-purpose approximator into a specialized cognitive subsystem embedded within a larger, more powerful whole.

Together, these principles form the conceptual foundation of the domain-bound AGI architecture. They enable a system that is modular, transparent, extensible, and capable of structured reasoning across a wide range of domains, while simultaneously offering a path for transformer models to evolve into more powerful and cognitively grounded components.

15.3 Architecture Overview

The domain-bound AGI architecture is a modular cognitive system composed of several independent subsystems that interact through explicit, structured interfaces. Only one of these subsystems—the reasoning language model—uses the Paged–PRE–CAS framework. All other components, including search engines, parsers, evaluators, and domain-specific tools, operate independently and communicate with the reasoning model through a machine-native scripting interface. This separation ensures that the reasoning model remains domain-agnostic, while the system as a whole can incorporate specialized computational capabilities without modifying the model’s parameters.

The architecture is designed not only to overcome the limitations of monolithic transformer models, but also to enhance them. By embedding a transformer-based reasoning core within a structured cognitive environment—equipped with explicit memory, algorithmic search, and modular experts—the transformer gains access to capabilities that cannot emerge from next-token prediction alone. The result is a hybrid system in which statistical fluency and symbolic structure reinforce one another.

15.3.1 Reasoning Language Model (Paged–PRE–CAS)

The reasoning component of the architecture is implemented as a Paged–PRE–CAS model. Its role is not to perform domain-specific computation, but to orchestrate the cognitive process. It is responsible for:

- decomposing tasks into smaller steps,
- generating structured instructions for experts,
- interpreting expert feedback,
- synthesizing partial results,
- and determining when a reasoning flow is complete.

Paged token partitioning provides structural constraints and computational efficiency; PRE (Propose–Refine) enables local improvement cycles; and CAS vectors (optional) provide alignment or evaluation signals. Crucially, the reasoning model contains *no domain-specific knowledge*. All domain operations—parsing, evaluation, simulation, search—are delegated to external experts.

This design repositions the transformer from a monolithic problem solver to a specialized cognitive subsystem. Freed from the burden of simulating search or maintaining long-term state, the model can focus on learned cognitive mechanisms such as canonical rewriting, variable binding, and flow-based attention. These mechanisms emerge naturally when the model operates within a structured cognitive architecture.

15.3.2 Script Engine

The script engine is the communication layer that connects the reasoning model to the rest of the system. It defines a minimal, machine-native instruction language for:

- invoking experts,
- creating or updating objects in memory,
- linking contexts,
- and routing callback prompts.

All interactions between components are explicit, deterministic, and auditable. The script engine ensures that the reasoning model never interacts with raw domain logic or unstructured data. Instead, it issues high-level instructions such as “simulate,” “evaluate,” or “plan,” which the script engine binds to the appropriate expert or search module.

This separation of concerns enables extensibility: new experts can be added without modifying the reasoning model, and new search engines can be integrated without retraining. It also enhances transparency and governance, since every operation is represented as an explicit script instruction rather than an implicit activation pattern inside a neural network.

15.3.3 Object-Referenced Memory Map

All cognitive entities are stored in an object-referenced memory map. Each entity—whether a domain object, intermediate result, or reasoning context—is assigned a unique identifier. This provides stable references that persist across reasoning steps and can be accessed or updated by experts.

A key innovation is that *contexts* themselves are objects in memory. A context represents an ongoing reasoning flow and may contain:

- references to domain objects,
- partial computations or intermediate results,
- links to sub-contexts,
- pending tasks or goals,
- and metadata relevant to the reasoning process.

Because contexts are stored as structured objects rather than text, they persist independently of token windows and can be manipulated by experts without requiring the reasoning model to reconstruct state from a growing prompt. This enables hierarchical reasoning, parallel flows, and long-lived cognitive processes that are not constrained by sequence length.

The memory map also provides a foundation for explicit governance and interpretability. Every state transition is visible, inspectable, and traceable—properties that are difficult to achieve in monolithic transformer architectures, where state is encoded implicitly in high-dimensional activations.

Together, the reasoning model, script engine, and memory map form the core of the architecture. They establish a structured cognitive environment in which transformer-based reasoning, algorithmic search, and domain-specific expertise can interact in a modular and interpretable manner.

15.3.4 Expert Modules

Expert modules are independent computational components that implement domain-specific or algorithmic capabilities. They operate outside the Paged-PRE-CAS reasoning core and do not participate in its token-based reasoning dynamics. Instead, they receive structured instructions from the script engine and return results through callback prompts that reference the appropriate context.

Experts may include:

- Monte Carlo and Beam Search engines for depth exploration,
- heuristic evaluators and scoring functions,
- rule-based or grammar-based parsers,
- constraint solvers and optimization routines,
- simulation engines for dynamic environments,
- domain-specific transformation or validation tools,
- or even specialized language models trained for a particular domain.

Although expert modules are *not required* to use the Paged-PRE-CAS framework, they *may* be implemented as Paged-PRE-CAS models when this is advantageous. For example, a domain-specific expert—such as a chemistry parser, a legal reasoning assistant, or a symbolic algebra engine—can itself be a small transformer trained on domain data. In such cases, the expert behaves as a specialized language model embedded within the broader architecture, while the central reasoning model remains domain-agnostic.

This flexibility ensures that the architecture can integrate a wide spectrum of computational tools: from deterministic algorithms and simulators to neural models and hybrid systems. Regardless of their internal implementation, all experts share the same interface: they operate on structured objects stored in the memory map and communicate exclusively through explicit script instructions and callback prompts.

The modularity of expert modules offers several advantages. New experts can be added without modifying or retraining the reasoning model. Existing experts can be replaced, extended, or audited independently. This supports transparency, safety, and governance: each expert is an isolated, inspectable component whose behavior is fully explicit.

Importantly, expert modules also enhance the transformer-based reasoning core. By delegating domain-specific computation to experts—whether algorithmic or neural—the reasoning model is freed from the burden of simulating algorithms through statistical continuation. This allows the

transformer to focus on higher-level cognitive functions such as canonical rewriting, variable binding, and flow control. In this way, the architecture not only overcomes the limitations of monolithic transformers but also provides a structured environment in which transformer-based experts can operate more effectively.

Expert modules therefore serve as the computational backbone of the domain-bound AGI architecture. They provide the concrete operations that the reasoning model orchestrates, enabling the system to perform complex, multi-step tasks across diverse domains without embedding domain logic inside a single monolithic model.

15.3.5 Generic Depth Search

A core component of the architecture is a domain-agnostic depth-search mechanism that provides the computational substrate for exploring large decision spaces. Transformer models excel at pattern recognition but lack explicit mechanisms for branching, simulation, or evaluating alternative futures. Depth search fills this gap by supplying a structured exploration process that the reasoning model can invoke whenever a task requires multi-step evaluation or combinatorial analysis.

The architecture treats the choice of search method as a *domain-level property*. Each domain specifies the search strategy that best matches its structural characteristics, allowing the script engine to automatically route search requests to the appropriate module. This ensures that the reasoning model remains entirely domain-agnostic: it issues abstract instructions such as “simulate” or “plan,” while the script engine binds these instructions to the domain’s preferred search engine.

Monte Carlo simulation is well suited for domains with large branching factors and shallow evaluation horizons. It provides a stochastic estimate of the value of a single action by sampling one-step or short-horizon futures. This makes it effective for local tactical exploration, where the goal is to evaluate immediate consequences rather than construct long-term plans.

Beam Search, by contrast, supports *multi-step planning*. It maintains a frontier of the most promising partial plans, guided by a simple domain heuristic. This enables deeper exploration of action sequences while pruning unpromising branches. Domains with structured combinatorial spaces—such as sorting, constraint puzzles, or symbolic workflows—benefit from Beam Search because it can construct multi-step plans that Monte Carlo cannot reliably discover.

Treating search as a modular component yields several advantages:

- **Flexibility:** domains can express their preferred planning style.
- **Transparency:** search statistics and traces are explicit and auditable.
- **Replaceability:** search engines can be swapped or extended without retraining.
- **Scalability:** deep reasoning is offloaded to specialized modules.

This separation of concerns allows the architecture to combine statistical reasoning with explicit algorithmic search—a capability that pure transformer models lack. At the same time, depth search enhances the transformer-based reasoning core: by delegating combinatorial exploration to external modules, the transformer is freed from the impossible task of simulating search through next-token prediction. This enables the reasoning model to focus on higher-level cognitive functions such as canonical rewriting, variable binding, and flow control.

Generic depth search therefore acts as a universal computational substrate for the architecture. It provides the raw exploration power that complements the reasoning model’s ability to structure tasks, interpret results, and orchestrate multi-step workflows across diverse domains.

15.3.6 Callback-Driven Self-Prompting

A defining feature of the architecture is its callback-driven mechanism for autonomous continuation of reasoning. Unlike monolithic transformer systems, which operate as single-pass sequence generators, the domain-bound AGI architecture supports multi-step workflows in which experts actively participate in the control flow. Experts do not simply return raw data; they produce structured callback prompts that instruct the reasoning model on how to proceed. This transforms the system from a passive text generator into an active, event-driven cognitive process.

When an expert completes an operation, it generates a callback containing:

- the identifier of the context to update,
- the result of the computation,
- and a structured instruction describing the next reasoning step.

The reasoning model processes the callback, updates the relevant context in the memory map, and issues new script instructions. This creates a closed-loop interaction pattern:

Reasoning Model $\rightarrow$ Expert Instruction $\rightarrow$ Expert Execution $\rightarrow$ Callback Prompt $\rightarrow$ Reasoning Model.

This loop continues until the reasoning model emits a termination signal. The architecture therefore supports long-lived, multi-step reasoning flows that are not constrained by token windows or sequential inference.

Callback-driven self-prompting provides several advantages:

- **Autonomy:** the system continues reasoning without external orchestration or synchronous API calls.
- **Hierarchical control:** complex tasks can be decomposed into sub-contexts, each driven by its own callback chain.
- **Parallelism:** multiple experts can operate on different contexts simultaneously, producing callbacks in any order.
- **Transparency:** every reasoning step is explicit and auditable.

This mechanism also enhances the transformer-based reasoning model. In monolithic architectures, transformers must implicitly simulate control flow through token continuation, which leads to drift, instability, and brittle long-range reasoning. In contrast, callback-driven self-prompting provides an external control structure that the transformer can rely on. The model no longer needs to remember its own output or maintain state through a growing prompt; instead, it receives structured callbacks that anchor each reasoning step in a stable context.

By embedding the transformer within a callback-driven architecture, the system achieves a form of procedural intelligence that cannot emerge from next-token prediction alone. The transformer becomes a cognitive controller that advances the reasoning process step by step, guided by explicit signals from experts and structured memory. This bidirectional interaction—experts driving callbacks, the transformer integrating them—forms the backbone of the architecture’s multi-step reasoning capabilities.

15.4 Hierarchical Contexts, Parallel Reasoning, and Asynchronous Processing

A central innovation of the architecture is the treatment of *contexts* as first-class computational objects stored in the memory map. Unlike transformer-based systems, which rely on a single linear prompt as their working memory, the architecture supports multiple concurrent reasoning flows, each represented by a structured context object with a unique identifier. This enables hierarchical reasoning, parallel task execution, asynchronous processing, and long-lived cognitive processes that are not constrained by token windows or sequential inference.

15.4.1 Contexts as First-Class Cognitive Objects

A context is an explicit object in the memory map that encapsulates the state of an ongoing reasoning process. It may contain:

- references to domain objects,
- partial computations or intermediate results,
- links to sub-contexts,
- pending tasks or goals,
- and metadata relevant to the reasoning flow.

Because contexts are stored as structured objects rather than text, they persist across reasoning steps and can be accessed or updated by experts without requiring the reasoning model to reconstruct state from a growing prompt. This provides a stable foundation for multi-step reasoning and eliminates the drift and fragility associated with token-based memory.

15.4.2 Hierarchical Context Trees

Contexts may reference other contexts, forming a tree or graph structure. This enables the system to decompose complex tasks into smaller sub-tasks, each handled by its own context. For example:

- a top-level context may represent a global planning task,
- child contexts may represent subtasks such as parsing, evaluation, or simulation,
- and leaf contexts may represent atomic operations delegated to experts.

The reasoning model can spawn new contexts, update existing ones, or merge results from multiple sub-contexts. This hierarchical structure supports recursive reasoning patterns and multi-step workflows that are difficult to achieve with monolithic transformer architectures.

15.4.3 Parallel Reasoning Flows

Because contexts are independent objects, multiple reasoning flows can proceed in parallel. The architecture does not enforce a single-threaded execution model. Instead:

- experts may operate on different contexts simultaneously,

- callback prompts may arrive in any order,
- and the reasoning model may resume or suspend contexts as needed.

Parallelism emerges naturally from the independence of context objects and the callback-driven control mechanism. This allows the system to explore multiple hypotheses, evaluate alternatives, or perform background computations without blocking the main reasoning flow.

15.4.4 Asynchronous Processing

A key advantage of the architecture is its support for *asynchronous* processing. Expert modules operate independently and may complete their work at different times. When an expert finishes, it does not interrupt the reasoning model or require synchronous waiting. Instead, it emits a callback prompt that is routed to the appropriate context.

This enables several powerful behaviors:

- **Non-blocking execution:** the reasoning model can continue working on other contexts while experts perform long-running computations.
- **Out-of-order completion:** callbacks may arrive in any order, and the reasoning model integrates them as they appear.
- **Dynamic task spawning:** experts may create new child contexts that perform additional reasoning or computation asynchronously.
- **Event-driven cognition:** reasoning progresses in response to completed tasks rather than a fixed sequence of steps.

Asynchronous processing transforms the architecture into a distributed cognitive system. The reasoning model acts as a central controller that integrates results, while experts function as autonomous workers that contribute to the overall reasoning process at their own pace.

15.4.5 Callback-Driven Continuation

The callback mechanism ties hierarchical contexts and asynchronous processing together. When an expert completes an operation, it generates a structured callback prompt containing:

- the identifier of the context to update,
- the result of the computation,
- and an instruction for how the reasoning model should proceed.

The reasoning model processes the callback, updates the relevant context, and issues new instructions. This creates a closed-loop reasoning process in which control flow is driven by the arrival of callbacks rather than by a fixed sequence of API calls.

15.4.6 Advantages Over Token-Based Reasoning

Hierarchical and asynchronous context management offers several advantages over traditional prompt-based reasoning:

- **Persistence:** contexts persist independently of token windows.
- **Structure:** reasoning state is stored as objects, not text.
- **Parallelism:** multiple reasoning flows can run concurrently.
- **Asynchronicity:** experts operate independently and return results when ready.
- **Decomposition:** tasks can be broken into sub-contexts.
- **Scalability:** long-lived processes are not constrained by sequence length.

These properties enable the architecture to support complex, multi-step reasoning tasks that are difficult or impossible to implement with monolithic transformer models.

15.4.7 A Bidirectional Benefit for Transformers

Hierarchical and asynchronous contexts not only overcome the limitations of transformer-based reasoning but also enhance the transformer itself. By externalizing state into structured objects and delegating computation to asynchronous experts, the transformer no longer needs to maintain long-term memory or simulate concurrency through token continuation. This reduces cognitive load and allows the model to focus on higher-level reasoning operations such as rewriting, planning, and integrating expert feedback.

In this way, hierarchical and asynchronous contexts provide a structured cognitive environment in which transformer models can operate more effectively, while simultaneously enabling forms of reasoning that cannot emerge from next-token prediction alone.

15.4.8 Summary

Hierarchical contexts, parallel reasoning, and asynchronous processing transform the architecture from a linear prompt-driven system into a distributed, event-driven cognitive environment. By representing reasoning flows as persistent objects, enabling parallel and asynchronous execution, and coordinating progress through callback prompts, the system achieves a form of structured, domain-agnostic reasoning that is both scalable and interpretable. At the same time, this structure enhances the transformer-based reasoning core, providing the stability and modularity required for advanced cognitive behavior.

15.5 Monte Carlo and Domain-Specific Search as Depth Reasoning Mechanisms

Transformer-based models excel at pattern recognition and linguistic generalization, but they struggle with deep combinatorial reasoning. Their architecture does not support branching, simulation, or the exploration of alternative futures. For tasks that require evaluating many possible action sequences, a separate computational mechanism is needed. The architecture therefore incorporates a family of depth-search modules—ranging from generic Monte Carlo simulation to domain-specific search engines such as Beam Search, A* search, and constraint-based planners.

These search modules operate independently of the reasoning model and provide the computational substrate for exploring large decision spaces. The reasoning model delegates search tasks through the script engine, receives structured results via callback prompts, and integrates them into the ongoing reasoning flow. This separation ensures that the reasoning model remains lightweight and domain-agnostic, while the search modules supply the computational power needed for deep reasoning.

15.5.1 Monte Carlo as a Domain-Agnostic Search Mechanism

Monte Carlo simulation provides a simple, unbiased, and transparent method for exploring decision spaces. It requires only three components:

- a state representation,
- a function that enumerates legal actions,
- and a termination condition.

Given these, the search module repeatedly samples action sequences and records their outcomes. Monte Carlo is fully domain-agnostic: it does not rely on heuristics, learned parameters, or domain-specific knowledge. This makes it an ideal default search mechanism for new domains or environments where no structured heuristics are available.

Monte Carlo is particularly effective for:

- local tactical exploration,
- evaluating short-horizon futures,
- stochastic or uncertain environments,
- and domains with large branching factors.

Its transparency is a major advantage. Each decision is supported by explicit statistics:

- number of simulations,
- distribution of outcomes,
- estimated value of each action,
- and variance or confidence intervals.

These statistics can be inspected, logged, or audited—properties that are difficult to achieve with monolithic transformer models.

15.5.2 Domain-Specific Search: Beam Search, A*, and Beyond

While Monte Carlo provides a robust default, many domains benefit from more structured search strategies. The architecture therefore supports the integration of domain-specific search engines, including:

- **Beam Search** for multi-step planning in structured spaces,

- **A* search** for heuristic-guided optimal planning,
- **Dijkstra-style search** for cost-based exploration,
- **constraint solvers** for combinatorial optimization,
- **SAT/SMT solvers** for logical or symbolic domains,
- **domain-specific planners** with custom heuristics.

These search engines can exploit domain structure to prune the search space, construct long-horizon plans, or guarantee optimality. For example:

- Beam Search excels in symbolic workflows such as sorting, scheduling, or program synthesis.
- A* search is ideal for navigation, pathfinding, and structured optimization problems.
- Constraint solvers are effective for puzzles, scheduling, and resource allocation.

The architecture treats the choice of search engine as a *domain-level property*. Each domain specifies its preferred search strategy, and the script engine automatically routes search requests to the appropriate module. This ensures that the reasoning model remains entirely domain-agnostic.

15.5.3 Integration with the Reasoning Model

The interaction between the reasoning model and the search modules follows a simple pattern:

1. The reasoning model issues a script instruction requesting a search operation on a given context.
2. The search module performs the required simulations or expansions.
3. The module stores the results in the memory map and generates a callback prompt.
4. The reasoning model processes the callback and incorporates the results into its ongoing reasoning flow.

This callback-driven loop enables the system to perform deep reasoning without requiring the transformer to simulate search through next-token prediction.

15.5.4 Replaceability and Extensibility

Search modules are fully modular. They can be:

- replaced with more efficient algorithms,
- augmented with heuristics or learned policies,
- specialized for particular domains,
- or combined into hybrid search strategies.

Because all communication occurs through the script engine and memory map, search modules can be integrated without modifying the reasoning model or the Paged-PRE-CAS components.

15.5.5 A Bidirectional Benefit for Transformers

Depth search not only compensates for the limitations of transformer models but also enhances them. By delegating combinatorial exploration to explicit search modules, the transformer is freed from the impossible task of simulating search through statistical continuation. This allows the reasoning model to focus on higher-level cognitive functions such as canonical rewriting, variable binding, and flow control.

In this way, depth search provides a structured computational environment in which transformer models can operate more effectively, while simultaneously enabling forms of reasoning that cannot emerge from next-token prediction alone.

15.5.6 A Computational Substrate for General Reasoning

Monte Carlo, Beam Search, A*, and other search engines together form a generic computational substrate for exploring decision spaces. They complement the reasoning model’s ability to plan, decompose tasks, and synthesize results. This division of labor enables the architecture to perform structured, multi-step reasoning across diverse domains without relying on monolithic parameter updates or opaque internal representations.

15.6 Self-Learning Without Weight Updates

A defining property of the architecture is that it can learn and expand its capabilities without modifying the parameters of the reasoning model. Instead of relying on gradient-based training or large-scale fine-tuning, the system acquires new skills procedurally: by generating new experts, integrating them through explicit interfaces, and incorporating them into its reasoning flows. Learning becomes a matter of *growing the system*, not altering its internal representations.

15.6.1 Procedural Learning Through Expert Acquisition

In traditional transformer architectures, learning requires updating the model’s weights. This couples knowledge, reasoning, and computation into a single parameter space, making it difficult to add new capabilities without retraining the entire model. In contrast, the domain-bound AGI architecture separates these concerns. The reasoning model remains domain-agnostic, while new capabilities are introduced by adding expert modules that implement specific operations.

Experts can be:

- algorithmic modules (e.g., search engines, evaluators),
- deterministic tools (e.g., parsers, simulators),
- or neural models trained for a specific domain.

Once an expert is added, the reasoning model can immediately use it through the script engine, without requiring any parameter updates. This enables rapid adaptation to new domains and tasks.

15.6.2 Learning Through Specification-Driven Expert Generation

A powerful consequence of this modular design is that the system can generate new experts directly from *formal specifications*. Given a structured description of a domain—its objects, operations, constraints, and evaluation rules—the reasoning model can produce:

- interface-compliant expert code,
- domain-specific transformation functions,
- validation routines,
- or simulation logic.

These generated experts are then integrated into the architecture by registering them with the script engine and memory map. Because all experts share a common interface, the reasoning model can immediately orchestrate them through canonical rewriting and variable binding.

This mechanism enables a form of self-expansion: the system can extend its own computational substrate by generating new tools that conform to the architecture’s interfaces. Learning becomes a process of *adding new modules*, not modifying existing ones.

15.6.3 Procedural Memory Instead of Parametric Memory

Traditional transformer models store knowledge implicitly in their parameters. This makes knowledge opaque, difficult to update, and tightly coupled to the model’s internal representations. In contrast, the architecture uses *procedural memory*: knowledge is stored in the form of explicit experts, structured objects, and domain specifications.

Procedural memory has several advantages:

- **Transparency**: each expert is an inspectable component.
- **Replaceability**: experts can be updated without affecting the reasoning model.
- **Extensibility**: new experts can be added at any time.
- **Governance**: domain logic is explicit and auditable.

This decoupling allows the system to grow indefinitely without accumulating instability or drift.

15.6.4 Self-Improvement Through Structured Interaction

The reasoning model improves its performance not by changing its parameters but by learning how to orchestrate experts more effectively. Through repeated interaction with experts, the model acquires stable procedural patterns:

- how to decompose tasks,
- how to bind variables,
- how to sequence operations,
- how to integrate expert feedback,
- and how to terminate reasoning flows.

These patterns are encoded in the learned cognitive mechanisms described earlier—canonical rewriting, flow-based attention, and repeat-prompt recursion. As the system gains more experts, the reasoning model becomes capable of solving increasingly complex tasks by composing these experts in novel ways.

15.6.5 A Scalable Path to Domain Expansion

Because new experts can be generated from specifications and integrated through stable interfaces, the architecture supports continuous domain expansion. The system can:

- incorporate new computational tools,
- adopt new search strategies,
- integrate new symbolic or neural modules,
- and extend its reasoning capabilities without retraining.

This provides a scalable path toward domain-bound general intelligence: the system becomes more capable not by modifying its core, but by enriching the ecosystem of experts it can orchestrate.

15.6.6 A New Paradigm for Learning

Self-learning without weight updates represents a shift from parametric to procedural intelligence. Instead of embedding all knowledge inside a single model, the architecture distributes knowledge across explicit components that can be added, replaced, or audited independently. The reasoning model acts as a stable cognitive core that coordinates these components through structured interaction.

This paradigm enables a form of continual learning that is transparent, governable, and modular—qualities that are difficult to achieve with monolithic transformer architectures. It also provides a practical path toward systems that can grow indefinitely, adapting to new domains through explicit integration rather than implicit parameter change.

15.7 Comparison With Monolithic Transformer Architectures

Large transformer-based language models have reshaped the landscape of artificial intelligence by demonstrating broad generalization, linguistic fluency, and powerful pattern recognition. Their success stems from a simple design principle: compress as much statistical structure as possible into a single, monolithic parameter space. This approach has proven remarkably effective for tasks that can be framed as sequence prediction. Yet the same design principle imposes structural constraints that limit their ability to perform explicit reasoning, modular extension, and governed computation.

The architecture presented in this chapter represents a different paradigm. It does not compete with transformer models; it *repositions* them within a broader cognitive system. The contrast between the two approaches can be summarized along several conceptual axes.

15.7.1 Monolithic Compression vs. Compositional Structure

Transformers unify all cognitive functions—memory, reasoning, knowledge, and decision-making—inside a single parameter space. This yields powerful emergent behavior but makes the system opaque, brittle, and difficult to extend.

The proposed architecture distributes cognitive functions across explicit components: a reasoning core, a script engine, a structured memory map, and a collection of expert modules. Each component is optimized for its role, and the interactions between components are governed by explicit interfaces. This compositional structure enables transparency, modularity, and controlled extension.

15.7.2 Implicit Emergence vs. Explicit Mechanisms

Transformer reasoning emerges implicitly from statistical continuation. This makes it difficult to enforce procedural structure, maintain stable state, or perform multi-step workflows without drift.

In contrast, the architecture implements reasoning through explicit mechanisms: canonical rewriting, variable binding, callback-driven continuation, hierarchical contexts, and asynchronous expert coordination. These mechanisms provide predictable, auditable control flow that does not depend on emergent token dynamics.

15.7.3 Static Models vs. Dynamic Systems

Monolithic transformers are static artifacts: once trained, their capabilities are fixed unless the entire model is fine-tuned. Domain adaptation requires parameter updates, which are expensive, opaque, and potentially destabilizing.

The modular architecture is a dynamic system. It grows by integrating new experts, new search strategies, and new domain specifications—without modifying the reasoning model. Learning occurs through system expansion rather than parameter change. This enables continual adaptation while preserving stability.

15.7.4 Opaque Internal State vs. Externalized Cognitive State

Transformers encode all state in high-dimensional activations and token sequences. This makes intermediate reasoning steps difficult to inspect or govern.

The proposed architecture externalizes cognitive state into structured objects stored in the memory map. Contexts, variables, intermediate results, and search traces are explicit and inspectable. This supports governance, safety, and interpretability in ways that monolithic models cannot achieve.

15.7.5 Single-Threaded Generation vs. Distributed Cognition

Transformers operate as single-threaded sequence generators. They cannot natively support parallel reasoning, asynchronous workflows, or multi-agent coordination.

The modular architecture is inherently distributed. Experts operate asynchronously, contexts evolve independently, and reasoning flows can proceed in parallel. The system behaves as a network of interacting cognitive processes rather than a single generative stream.

15.7.6 A Bidirectional Relationship

The contrast between the two approaches is not adversarial. Transformer models remain essential: they provide the reasoning core that orchestrates the architecture. At the same time, the architecture enhances transformers by providing:

- explicit memory,
- structured control flow,
- domain-specific computation,
- and transparent search mechanisms.

Transformers become more capable within the architecture than they are on their own. The relationship is bidirectional: the architecture generalizes the transformer, and the transformer empowers the architecture.

15.7.7 Synthesis

Monolithic transformer architectures excel at compressing knowledge and recognizing patterns, but they are not designed for structured reasoning, modular extension, or governed computation. The architecture presented here offers a complementary paradigm: a compositional cognitive system in which reasoning is explicit, memory is structured, search is transparent, and learning occurs through modular integration. Together, these properties provide a scalable and interpretable foundation for domain-bound general intelligence.

15.8 Domain-Bound AGI as a Recipe

The architecture introduced in this chapter is best understood not as a single system but as a *pattern*: a reusable cognitive recipe for constructing intelligent systems across diverse domains. Rather than embedding all knowledge and reasoning inside one monolithic model, the architecture defines a set of interacting components—each with a clear role and interface—that can be instantiated, combined, and extended to form domain-bound intelligent systems.

This recipe-based perspective shifts the focus from training a universal model to assembling a coherent cognitive system. A new domain does not require a new reasoning model; it requires a new configuration of components that the reasoning model can orchestrate.

15.8.1 A Domain-Agnostic Cognitive Core

At the center of every instantiation lies a domain-agnostic reasoning core. Its function is not to contain domain knowledge but to provide the procedural scaffolding for reasoning:

- interpreting tasks,
- structuring workflows,
- coordinating computational modules,
- and maintaining coherent control flow.

Because the core is independent of domain content, it can be reused across all domains. What changes from domain to domain is not the reasoning process but the computational substrate on which it operates.

15.8.2 Domain Logic as Modular Ingredients

Domain-specific behavior is supplied by modular components that implement the operations, transformations, and evaluations relevant to a particular domain. These components act as the *ingredients* of the recipe. They may be algorithmic, symbolic, or neural, and they can be generated from formal specifications or implemented manually.

What matters is not their internal design but their adherence to the architecture’s interfaces. Once integrated, they become part of the system’s computational vocabulary—tools that the reasoning core can invoke as needed.

15.8.3 Search as a Pluggable Exploration Strategy

Many domains require exploration of alternative futures or combinatorial structures. The architecture treats search as a pluggable component: each domain may specify the exploration strategy that best fits its structure.

This design avoids prescribing a single universal search mechanism. Instead, it provides a place in the recipe where domain-appropriate exploration can be inserted—whether Monte Carlo sampling, heuristic search, constraint solving, or another method.

15.8.4 Memory as a Structured Cognitive Workspace

The architecture externalizes cognitive state into a structured workspace. This workspace stores objects, intermediate results, and reasoning contexts in a form that is persistent, inspectable, and independent of token sequences.

In the recipe metaphor, memory is the *work surface*: the place where ingredients are combined, transformed, and assembled into coherent results.

15.8.5 Instantiating a New Domain

Constructing a new domain-bound system becomes a procedural act rather than a training exercise. The recipe can be summarized as:

1. Define the domain’s objects, operations, and constraints.
2. Implement or generate experts that realize these operations.
3. Register these experts through the architecture’s interfaces.
4. Specify the domain’s preferred search strategy.
5. Initialize contexts that represent the tasks to be solved.
6. Allow the reasoning core to orchestrate the system.

The reasoning model requires no modification; the system adapts through integration rather than parameter change.

15.8.6 A General Pattern for Constructing Intelligence

The result is a general pattern for building intelligent systems:

- a reusable reasoning core,
- a structured workspace,
- a communication layer,
- and a set of domain-specific computational tools.

Each instantiation follows the same recipe but differs in the ingredients it uses. This provides a scalable, interpretable, and governable path toward domain-bound general intelligence—one in which systems grow by acquiring new modules rather than modifying their internal parameters.

15.9 Learned Cognitive Mechanisms in the Reasoning Core

Beyond the structural components of the architecture—experts, memory, search, and the script engine—the reasoning model contributes a lightweight cognitive layer that provides coherence, stability, and procedural structure. These mechanisms do not encode domain knowledge; instead, they define how reasoning unfolds. They arise from curriculum learning within the Paged-PRE-CAS framework and enable the model to function as a domain-agnostic controller.

15.9.1 Canonicalization Through Prompt Rewriting

The reasoning model begins each reasoning cycle by rewriting natural-language inputs into canonical machine forms. This process extracts structure, removes linguistic variability, and produces stable representations that downstream components can interpret reliably. Canonicalization ensures that reasoning is grounded in explicit structure rather than emergent token patterns.

15.9.2 Variable Binding Through Local Prompt Maps

During rewriting, the model constructs a local variable map that binds symbols, identifiers, and references to stable names. These bindings persist across the reasoning flow and provide referential coherence without relying on long context windows. Each context maintains its own variable map, enabling independent and parallel reasoning processes.

15.9.3 Flow-Based Procedural Attention

Instead of generic self-attention, the model employs flow-based attention patterns learned through curriculum. These patterns encode procedural regularities—such as classification before evaluation or evaluation before simulation—and guide the sequencing of operations. Flow-based attention provides predictable, auditable control flow without embedding domain logic in the model’s parameters.

15.9.4 Callback Integration Through Self-Prompting

Experts return structured callbacks that signal the next reasoning step. The model integrates these callbacks through learned rewriting and variable binding, allowing it to continue reasoning autonomously. This self-prompting mechanism turns the model into a procedural controller capable of coordinating asynchronous expert activity and advancing multi-step workflows.

15.9.5 Context–Output Separation

A foundational principle of the architecture is the strict separation between internal reasoning context and external output. The model does not accumulate a growing transcript; it reconstructs its reasoning state from structured memory at every step. This prevents drift, eliminates dependency on past text, and enables arbitrarily long reasoning processes without degradation.

15.9.6 Repeat–Prompt Recursion

Repeat–Prompt Recursion operationalizes this separation. At each step, the model regenerates a fresh canonical prompt that encodes the current task, symbolic state, variable bindings, and required next operation. This creates a controlled recursive loop:

Context $\rightarrow$ Rewrite $\rightarrow$ Expert Call $\rightarrow$ Callback $\rightarrow$ Context Reconstruction.

The result is a stable, deterministic reasoning process that does not depend on long-context attention or transcript accumulation.

15.9.7 Reasoning as a Controlled Recursive Process

Together, these mechanisms transform the reasoning model into a symbolic controller that:

1. rewrites its own prompt,
2. binds variables,
3. invokes experts,
4. integrates callbacks,
5. reconstructs context,
6. and repeats.

This controlled recursion provides the semantic stability and procedural structure required for modular, interpretable, and domain-agnostic reasoning. The cognitive layer does not solve tasks by itself; it orchestrates the components that do. The architecture provides the structure, the model provides the flexibility, and their interaction produces coherent cognition.

15.10 Conclusion

The architecture presented in this chapter reframes intelligence as a modular interaction between a domain-agnostic reasoning core and a set of explicit, inspectable computational components. Rather than concentrating all knowledge and reasoning inside a single model, the system distributes cognition across experts, structured memory, search procedures, and a lightweight cognitive layer implemented within the Paged-PRE-CAS framework.

This separation of concerns yields several advantages. The reasoning core remains stable and reusable across domains, while domain-specific behavior is supplied by modular experts that can be added, replaced, or generated from formal specifications. Memory is externalized into structured objects that persist across reasoning steps, enabling long-lived, hierarchical, and parallel workflows. Search is transparent and pluggable, allowing each domain to employ the exploration strategy best suited to its structure.

The learned cognitive mechanisms of the reasoning model—canonical rewriting, variable binding, flow-based attention, self-prompting, and repeat-prompt recursion—provide the procedural glue that binds these components into a coherent system. They ensure that reasoning is explicit, stable, and interpretable, even as the system grows and adapts.

Taken together, these elements form a general recipe for constructing domain-bound intelligent systems. The architecture does not rely on monolithic parameter updates or opaque internal representations. Instead, it builds intelligence through composition: a reasoning core that orchestrates explicit tools within a structured cognitive environment. This approach offers a scalable, governable, and transparent foundation for artificial systems capable of sustained, multi-step reasoning across diverse domains.

Architectural Insight

Domain-bound intelligence emerges when a stable, domain-agnostic reasoning core orchestrates explicit experts, structured memory, and transparent search within a controlled recursive process. The architecture separates context from output, distributes cognition across modular components, and relies on learned canonicalization, variable binding, and flow-based control to maintain coherence. Intelligence is not concentrated in a single model but constructed through composition: a reasoning core that coordinates explicit tools inside a governable cognitive environment.

Chapter 16

Domain-Bound AGI and AI Agents: A Comparative Architectural Analysis

16.1 Why This Comparison Matters

Artificial General Intelligence is increasingly framed in mainstream discourse as a combination of two trends: ever-larger monolithic transformer models and the rapid rise of *AI agent* frameworks. These agent systems—built around planner–executor loops, tool-use pipelines, and external orchestration—are often presented as the practical path toward autonomous reasoning. As a result, the term “AGI” is frequently associated not only with model scale but also with the agentic wrappers that surround these models.

This chapter clarifies how such agent systems relate to the domain-bound AGI architecture developed in this book. Although both approaches can exhibit superficially similar behaviours—task decomposition, multi-step reasoning, tool invocation, and autonomous workflows—their underlying mechanisms differ fundamentally. AI agents rely on *external* control structures wrapped around a monolithic model, whereas domain-bound AGI constructs intelligence through an *internal* cognitive architecture composed of structured memory, callback-driven control flow, modular experts, and a domain-agnostic reasoning core.

Understanding this distinction is essential for two reasons. First, it situates the domain-bound AGI architecture within the broader landscape of contemporary AI, highlighting where behavioural overlap exists and where architectural divergence is decisive. Second, it reveals the limitations of current agent frameworks and explains why an internal, compositional approach provides a more scalable, governable, and cognitively grounded foundation for general intelligence.

The goal of this chapter is therefore threefold:

1. to identify the behavioural similarities between AI agents and domain-bound AGI,
2. to analyse the architectural differences that determine their respective strengths and weaknesses,
3. and to explain why the internal, structured approach of domain-bound AGI offers a more robust path toward general-purpose reasoning than the ad hoc external orchestration used in contemporary agent systems.

By contrasting these two paradigms, we illuminate the unique contributions of the domain-bound AGI architecture and clarify its role within the evolving ecosystem of intelligent systems.

16.2 What AI Agents Are

AI agents have emerged as a dominant paradigm for extending the capabilities of large language models. Rather than modifying a model’s internal architecture, agent systems wrap an external control structure around a monolithic transformer. This wrapper provides planning, tool invocation, memory retrieval, and iterative refinement, creating the appearance of autonomous reasoning. Although effective in practice, this approach relies on *external orchestration* rather than internal cognitive structure.

At their core, contemporary AI agents follow a simple operational pattern:

1. interpret a task or goal,
2. generate a plan or next action using a language model,
3. invoke external tools or APIs,
4. observe the results,
5. update the prompt or memory,
6. and repeat.

This loop is implemented by an orchestrator—a software layer that coordinates the model, tools, and memory. The language model itself remains passive: it produces text when prompted but does not maintain state, manage control flow, or coordinate computation. All cognitive structure resides outside the model, encoded in scripts, templates, and agent frameworks.

16.2.1 The External Agent Loop

The defining characteristic of AI agents is that reasoning is driven by an *external* loop. The orchestrator determines when the model is invoked, how its outputs are interpreted, which tools are called, and how results are integrated. The model does not control its own reasoning process; it is a component inside a larger pipeline.

This external loop typically includes:

- a planner that decomposes tasks into steps,
- an executor that calls tools or APIs,
- a memory module (often a vector store),
- and a controller that updates the prompt and repeats the cycle.

Because the model has no internal representation of state or control flow, all structure must be encoded in text. The prompt becomes the working memory, the transcript becomes the reasoning trace, and the orchestrator becomes the cognitive engine.

16.2.2 Strengths of the Agent Approach

Despite their architectural limitations, AI agents offer several practical advantages:

- **Rich ecosystem support:** frameworks such as LangChain, AutoGen, and Swarm provide ready-made components for tool integration, memory, and orchestration.
- **Immediate applicability:** agents can be deployed quickly using existing models and APIs.
- **Integration with real environments:** agents can interact with browsers, IDEs, terminals, databases, and external systems.
- **Flexible task templates:** personas, workflows, and domain-specific agent configurations are easy to construct.

These strengths explain why agents have become a popular method for extending LLM capabilities. They provide a practical interface between models and real-world tasks, even if the underlying cognitive mechanisms remain shallow.

16.2.3 Limitations of the Agent Paradigm

The same properties that make agents easy to build also impose structural constraints:

- **Prompt-based state:** all reasoning state is encoded in text, leading to drift, inconsistency, and fragility.
- **Lack of internal control flow:** the model cannot manage its own reasoning; the orchestrator must script every step.
- **Simulated search:** planning and exploration are approximated through prompting rather than explicit algorithms.
- **Opaque intermediate reasoning:** the model's internal activations are inaccessible, and the prompt transcript is not a reliable cognitive trace.
- **Limited scalability:** long workflows accumulate noise, and parallel or asynchronous reasoning is difficult to implement.

These limitations reflect a deeper issue: AI agents do not modify the cognitive architecture of the model. They compensate for architectural gaps through external scaffolding, producing behaviour that resembles reasoning without providing the structural foundations required for robust, general-purpose cognition.

This chapter contrasts this external, orchestrated approach with the internal, compositional architecture of domain-bound AGI, highlighting both behavioural overlap and architectural divergence.

16.3 What Domain-Bound AGI Is

Domain-bound AGI represents a fundamentally different approach to intelligent system design. Instead of wrapping external control structures around a monolithic model, it constructs intelligence from the inside out through a modular cognitive architecture. The system is composed of explicit,

inspectable components—structured memory, callback-driven control flow, pluggable search engines, and domain-specific experts—coordinated by a lightweight, domain-agnostic reasoning core implemented using the Paged-PRE-CAS framework.

Where AI agents rely on external orchestration to simulate reasoning, domain-bound AGI embeds the mechanisms of reasoning directly into the architecture. The result is a system that behaves not as a scripted agent but as a distributed cognitive environment capable of stable, multi-step, and interpretable reasoning across diverse domains.

16.3.1 A Domain-Agnostic Reasoning Core

At the center of the architecture lies a transformer-based reasoning model that does not contain domain knowledge. Its role is procedural rather than encyclopedic. The reasoning core:

- rewrites tasks into canonical machine forms,
- binds variables and constructs local symbol maps,
- issues structured instructions to experts,
- integrates callback prompts,
- and determines when reasoning flows are complete.

This model does not simulate search, maintain long-term state, or embed domain logic. Instead, it orchestrates the cognitive process by coordinating the components that *do* perform these functions. Freed from the burden of monolithic reasoning, the model specializes in stable procedural mechanisms such as canonical rewriting, flow-based attention, and repeat-prompt recursion.

16.3.2 Structured Memory as Cognitive State

Unlike prompt-based systems, domain-bound AGI externalizes cognitive state into a structured memory map. Contexts, variables, intermediate results, and search traces are stored as explicit objects with unique identifiers. This enables:

- persistent reasoning flows,
- hierarchical context trees,
- parallel and asynchronous processing,
- and transparent, auditable state transitions.

Memory is not a transcript but a workspace. Reasoning does not depend on token accumulation; it is reconstructed at each step from structured state. This eliminates drift and enables arbitrarily long reasoning processes.

16.3.3 Experts as Cognitive Subsystems

Domain-specific computation is delegated to expert modules. These may be:

- deterministic algorithms,

- symbolic or grammar-based parsers,
- simulation engines,
- search modules (Monte Carlo, Beam Search, A*, constraint solvers),
- or even small transformer models trained for a specific domain.

Experts communicate exclusively through the script engine and return structured callbacks that drive the reasoning process forward. They behave as autonomous cognitive subsystems rather than passive tools, enabling a Society-of-Mind style architecture in which multiple processes contribute to the overall reasoning flow.

16.3.4 Callback-Driven Control Flow

A defining feature of the architecture is its callback-driven continuation mechanism. When an expert completes an operation, it emits a structured callback that:

- identifies the context to update,
- provides the result of the computation,
- and specifies the next reasoning step.

The reasoning core integrates this callback, updates the memory map, and issues new instructions. This creates an internal control loop that is stable, recursive, and independent of external orchestration. Reasoning becomes an event-driven process rather than a scripted sequence of prompts.

16.3.5 Pluggable Search as a Depth Substrate

Deep combinatorial reasoning is handled by explicit search modules rather than by prompting the transformer to imagine alternatives. Monte Carlo simulation, Beam Search, A*, and constraint solvers provide transparent, auditable exploration of decision spaces. The reasoning core delegates search tasks and integrates results through callbacks, enabling multi-step planning without relying on emergent token dynamics.

16.3.6 A Compositional Cognitive System

Taken together, these components form a compositional architecture in which intelligence emerges from the interaction of explicit, modular processes. The reasoning core provides procedural structure; experts provide computation; memory provides state; and search provides depth. The system grows not by modifying parameters but by integrating new modules, enabling continual expansion without destabilizing the cognitive core.

Domain-bound AGI is therefore not an agent framework but a cognitive architecture: a structured environment in which reasoning is explicit, memory is externalized, search is transparent, and learning occurs through modular integration rather than monolithic compression.

16.4 Behavioural Overlap Between AI Agents and Domain-Bound AGI

Although AI agents and domain-bound AGI differ fundamentally in their internal architecture, they often exhibit similar *behavioural patterns*. This superficial resemblance has contributed to the widespread perception that agentic wrappers around large language models constitute a viable path toward general intelligence. To understand the architectural divergence that follows in later sections, it is essential to first identify where the two paradigms appear to converge.

16.4.1 Task Decomposition and Planning

Both AI agents and domain-bound AGI systems are capable of breaking complex tasks into smaller, manageable steps. In agent frameworks, this behaviour is typically implemented through prompting templates or planner–executor loops. In domain-bound AGI, task decomposition emerges from canonical rewriting, variable binding, and callback-driven continuation. Despite the different mechanisms, the observable outcome—a structured sequence of sub-tasks—is similar.

16.4.2 Tool and Expert Invocation

AI agents frequently call external tools such as search engines, code interpreters, browsers, or domain-specific APIs. Domain-bound AGI also invokes external modules, but these modules are integrated as *experts* with uniform interfaces and structured callbacks. In both cases, the system appears to augment its reasoning with external computation, even though the underlying integration mechanisms differ sharply.

16.4.3 Multi-Step Reasoning and Iterative Refinement

Both paradigms support multi-step workflows in which intermediate results are evaluated, refined, or recombined. AI agents achieve this through repeated prompting and transcript accumulation, while domain-bound AGI reconstructs reasoning state from structured memory at each step. The behavioural signature of iterative reasoning is therefore shared, even though the cognitive substrate is not.

16.4.4 Search and Exploration of Alternatives

AI agents often simulate planning or search by prompting the model to enumerate alternatives, evaluate options, or imagine hypothetical futures. Domain-bound AGI performs similar tasks but delegates them to explicit search modules such as Monte Carlo simulation, Beam Search, or A* search. From the outside, both systems appear to explore decision spaces, though only one performs genuine algorithmic search.

16.4.5 Autonomous Workflow Execution

Both systems can operate autonomously once a task is initiated. AI agents achieve autonomy through external orchestrators that repeatedly call the model and tools. Domain-bound AGI achieves autonomy through callback-driven continuation, in which experts and the reasoning core coordinate progress without external intervention. The resulting behaviour—a system that continues reasoning on its own—is similar in appearance.

16.4.6 Summary

These behavioural overlaps explain why AI agents are often perceived as a practical route toward AGI. They demonstrate planning, tool use, multi-step reasoning, and autonomous workflows—all hallmarks of intelligent behaviour. However, as the next sections show, these similarities mask profound architectural differences. AI agents simulate cognition through external scaffolding, whereas domain-bound AGI constructs cognition through internal, compositional mechanisms. Understanding this distinction is essential for evaluating the strengths and limitations of each approach.

16.5 What AI Agents Do More (and Why It Matters)

Although AI agents lack the internal cognitive structure of domain-bound AGI, they possess several practical advantages that stem from their integration into existing software ecosystems. These advantages do not arise from architectural depth but from engineering maturity: agents are embedded in rich toolchains, frameworks, and user-facing environments that make them immediately useful in real-world applications. Understanding these strengths is essential for a fair comparison, as they highlight areas where domain-bound AGI can benefit from future integration and system-level development.

16.5.1 Integration With Real-World Environments

AI agents are designed to interact directly with external systems. Modern frameworks provide built-in support for:

- web browsing and information extraction,
- file system manipulation,
- API calls and database queries,
- code execution and debugging,
- interaction with IDEs, terminals, and cloud services.

These capabilities allow agents to operate as practical assistants in software engineering, research, automation, and data analysis. Domain-bound AGI can interface with such environments as well, but current agent ecosystems offer a broader and more mature set of integrations.

16.5.2 Product-Level Interfaces and Templates

Agent frameworks often include user-facing abstractions such as:

- predefined personas (e.g., “research assistant,” “planner,” “coder”),
- workflow templates for common tasks,
- conversational shells and interactive dashboards,
- plug-and-play components for memory, planning, and tool use.

These features make agents accessible to non-expert users and enable rapid prototyping. Domain-bound AGI, by contrast, is a cognitive architecture rather than a product framework; its strength lies in internal structure, not in predefined user experiences.

16.5.3 Ecosystem Maturity and Framework Support

The popularity of AI agents has led to a flourishing ecosystem of libraries and frameworks, including LangChain, AutoGen, Swarm, and numerous open-source toolkits. These frameworks provide:

- orchestration engines,
- logging and monitoring tools,
- memory backends,
- tool registries,
- deployment pipelines.

This ecosystem accelerates development and experimentation. Domain-bound AGI does not yet benefit from comparable infrastructure, though its architectural clarity makes it well-suited for future tooling.

16.5.4 Rapid Deployment and Practical Utility

Because agents wrap existing models rather than modifying them, they can be deployed immediately using off-the-shelf LLMs. This makes them attractive for:

- enterprise automation,
- workflow orchestration,
- research assistance,
- software development support,
- data extraction and transformation.

Domain-bound AGI requires an implementation of its architectural components, which introduces an initial engineering cost. However, once implemented, it offers capabilities that agents cannot match.

16.5.5 Summary

AI agents excel in integration, tooling, and practical deployment. They provide a convenient interface between monolithic language models and real-world environments, supported by a rich ecosystem of frameworks and templates. These strengths make agents valuable in applied settings, even though their cognitive mechanisms remain shallow. Recognizing these advantages clarifies the complementary relationship between agent ecosystems and the deeper architectural foundations of domain-bound AGI.

16.6 What AI Agents Do Less (and Why It Matters)

While AI agents offer practical advantages in integration and deployment, they lack several structural capabilities that are essential for robust, scalable, and interpretable reasoning. These limitations are not incidental; they arise directly from the architectural choice to wrap external control logic around a monolithic transformer model. By examining what agents *cannot* do, we clarify the motivations behind the domain-bound AGI architecture and highlight the cognitive mechanisms that agents fail to provide.

16.6.1 Lack of Structured Cognitive State

AI agents do not maintain an internal, structured representation of cognitive state. All state is encoded in:

- the prompt transcript,
- the model’s hidden activations,
- or an external vector store.

These representations are fragile, opaque, and difficult to govern. They do not support hierarchical decomposition, parallel reasoning flows, or persistent context objects. As a result, agents struggle with long-lived tasks, complex workflows, and reasoning processes that require stable intermediate state.

16.6.2 No Internal Control Flow

In agent systems, control flow resides entirely in the orchestrator. The model does not manage its own reasoning process; it merely responds to prompts. This means that:

- the model cannot autonomously decide when to call tools,
- it cannot manage sub-tasks or spawn sub-processes,
- it cannot integrate asynchronous results,
- and it cannot maintain procedural coherence across steps.

Domain-bound AGI, by contrast, embeds control flow inside the architecture through callback-driven continuation and repeat-prompt recursion.

16.6.3 Simulated Rather Than Explicit Search

AI agents often appear to perform planning or search, but this behaviour is simulated through prompting. The model is asked to:

- enumerate alternatives,
- imagine hypothetical futures,
- or evaluate options in natural language.

This is not genuine search. There is no branching structure, no explicit evaluation function, no search tree, and no transparent trace of decisions. Agents therefore struggle with deep combinatorial reasoning, long-horizon planning, and domains that require systematic exploration.

16.6.4 Opaque Intermediate Reasoning

Because agents rely on monolithic transformers, intermediate reasoning steps are encoded in high-dimensional activations that are inaccessible to the system. The prompt transcript is not a reliable cognitive trace; it is merely a textual artifact. This opacity makes it difficult to:

- audit reasoning,
- enforce constraints,
- debug failures,
- or guarantee safety.

Domain-bound AGI externalizes all intermediate state into structured memory, making reasoning explicit and inspectable.

16.6.5 Limited Scalability and Fragility

Prompt-based reasoning accumulates noise over time. As transcripts grow:

- context becomes inconsistent,
- variable references drift,
- earlier assumptions are forgotten,
- and the model’s behaviour becomes unstable.

This fragility limits the depth and duration of workflows that agents can reliably execute. Domain-bound AGI avoids these issues by reconstructing reasoning state from structured memory at each step, enabling arbitrarily long and parallel reasoning processes.

16.6.6 No Mechanism for Self-Expansion

AI agents cannot expand their own computational substrate. They cannot:

- generate new tools from formal specifications,
- integrate new experts through stable interfaces,
- or grow their capabilities without external engineering effort.

Their learning is limited to prompt engineering or fine-tuning, both of which modify behaviour without expanding the underlying architecture. Domain-bound AGI, by contrast, supports procedural learning through expert acquisition and modular integration.

16.6.7 Summary

AI agents do less not because they are poorly engineered, but because their architecture fundamentally limits what they can do. They lack structured state, internal control flow, explicit search, transparent reasoning, scalable workflows, and mechanisms for self-expansion. These limitations motivate the domain-bound AGI architecture, which provides the structural foundations required for robust, interpretable, and extensible cognition.

16.7 Why the Internal Approach Is Stronger

The architectural differences between AI agents and domain-bound AGI are not merely technical preferences; they determine the depth, stability, and governability of the resulting system. While AI agents rely on external scaffolding to simulate intelligent behaviour, domain-bound AGI embeds the mechanisms of reasoning directly into the cognitive substrate. This section explains why the internal, compositional approach provides a more powerful and scalable foundation for general-purpose reasoning.

16.7.1 Architectural Foundations Rather Than Behavioural Facades

AI agents achieve intelligent behaviour by layering orchestration logic around a monolithic model. The resulting system may appear agentic, but its behaviour is ultimately constrained by the limitations of prompt-based state, emergent token dynamics, and external control flow. Domain-bound AGI, by contrast, constructs intelligence from explicit architectural components: structured memory, callback-driven continuation, pluggable search, and modular experts. These components provide the cognitive foundations that agents attempt to approximate through prompting.

16.7.2 Stability Through Structured State

Prompt-based reasoning is inherently fragile. As transcripts grow, the model must reconstruct state from increasingly noisy text, leading to drift, inconsistency, and loss of coherence. Domain-bound AGI avoids these issues by externalizing cognitive state into structured memory objects. Each reasoning step begins with a fresh reconstruction of context, ensuring stability across long workflows, parallel processes, and hierarchical decompositions. This structural stability enables reasoning depths that agents cannot reliably achieve.

16.7.3 Transparent and Governable Cognition

Because AI agents rely on monolithic transformers, their intermediate reasoning is encoded in opaque activations. The prompt transcript is not a reliable cognitive trace, and the orchestrator cannot inspect or enforce internal reasoning steps. Domain-bound AGI, by contrast, makes every state transition explicit: contexts, variables, search traces, and expert results are stored as structured objects. This transparency enables auditing, debugging, constraint enforcement, and governance—capabilities that are essential for safe and trustworthy intelligent systems.

16.7.4 True Depth Through Explicit Search

Agents simulate planning and search by prompting the model to imagine alternatives. This approach is limited by the model's ability to maintain coherence across long sequences and by the absence of explicit branching structures. Domain-bound AGI delegates combinatorial reasoning to dedicated search modules—Monte Carlo simulation, Beam Search, A* search, and constraint solvers—that provide genuine depth, transparent evaluation, and auditable decision traces. The reasoning core integrates these results without attempting to simulate search through token continuation.

16.7.5 Scalability Through Modular Composition

AI agents scale horizontally by adding more tools or more agents, but their cognitive architecture remains unchanged. Domain-bound AGI scales vertically by expanding its computational substrate:

new experts, new search strategies, new domain specifications, and new symbolic or neural modules can be integrated without modifying the reasoning core. This modularity enables continual growth without destabilizing the system.

16.7.6 Self-Expansion Without Parameter Updates

Agents cannot expand their own capabilities; they rely on external engineering to add tools or modify prompts. Domain-bound AGI supports procedural learning through specification-driven expert generation. Given a formal description of a domain, the reasoning core can generate new experts, integrate them through stable interfaces, and immediately use them in reasoning flows. This provides a path toward self-expanding systems that grow through composition rather than parameter change.

16.7.7 Bidirectional Enhancement of Transformer Models

In agent systems, the transformer must simulate memory, search, and control flow through prompting. This burdens the model with tasks it is not designed to perform. In domain-bound AGI, the transformer is repositioned as a cognitive controller that specializes in canonical rewriting, variable binding, and flow-based attention. The architecture provides the memory, search, and procedural scaffolding that the model lacks. The result is a bidirectional relationship: the architecture enhances the transformer, and the transformer empowers the architecture.

16.7.8 Summary

The internal approach of domain-bound AGI is stronger because it provides the structural foundations that AI agents attempt to approximate externally. It offers stability through structured state, transparency through explicit memory, depth through algorithmic search, scalability through modular composition, and self-expansion through procedural learning. These properties enable forms of reasoning that are difficult or impossible to achieve with prompt-based agent systems, positioning domain-bound AGI as a more robust and cognitively grounded path toward general intelligence.

16.8 Synthesis: Two Divergent Paths Toward Intelligent Systems

The comparison between AI agents and domain-bound AGI reveals two fundamentally different approaches to constructing intelligent systems. Although both paradigms can exhibit similar surface-level behaviours—planning, tool use, multi-step reasoning, and autonomous workflows—their underlying mechanisms, strengths, and limitations diverge sharply. This section synthesizes the preceding analysis and articulates the conceptual distinction between the external, orchestrated agent paradigm and the internal, compositional architecture of domain-bound AGI.

16.8.1 Behavioural Similarity, Architectural Divergence

AI agents and domain-bound AGI often appear to perform the same tasks. Both can decompose problems, invoke external computation, and refine intermediate results. Yet these similarities mask a profound architectural divide. AI agents simulate cognition through external scaffolding wrapped around a monolithic model, while domain-bound AGI constructs cognition through explicit, interacting components—structured memory, callback-driven control flow, pluggable search, and modular experts.

The behavioural overlap is therefore superficial; the architectural divergence is fundamental.

16.8.2 External Orchestration vs. Internal Cognition

AI agents rely on an orchestrator that manages state, control flow, and tool invocation. The model itself remains passive, responding to prompts without maintaining internal structure. Domain-bound AGI embeds these mechanisms inside the architecture. The reasoning core coordinates experts, integrates callbacks, and reconstructs context from structured memory. Control flow becomes an internal cognitive process rather than an external script.

This distinction determines the system’s stability, scalability, and interpretability.

16.8.3 Emergent Patterns vs. Explicit Mechanisms

Agent behaviour emerges from prompting strategies, templates, and ad hoc engineering. Domain-bound AGI relies on explicit mechanisms: canonical rewriting, variable binding, flow-based attention, and repeat-prompt recursion. These mechanisms provide predictable, auditable reasoning patterns that do not depend on emergent token dynamics.

Where agents rely on emergent behaviour, domain-bound AGI relies on explicit cognitive structure.

16.8.4 Shallow Depth vs. Algorithmic Search

AI agents approximate planning and search by asking the model to imagine alternatives. This approach is limited by the model’s ability to maintain coherence across long sequences. Domain-bound AGI delegates combinatorial reasoning to explicit search modules—Monte Carlo simulation, Beam Search, A* search, and constraint solvers—that provide genuine depth and transparent evaluation.

The result is a qualitative difference in reasoning capability.

16.8.5 Static Models vs. Growing Systems

AI agents depend on static models whose capabilities are fixed at training time. Domain-bound AGI grows by integrating new experts, new search strategies, and new domain specifications. Learning occurs through modular expansion rather than parameter updates. This enables continual adaptation without destabilizing the reasoning core.

Where agents scale by adding more prompts, domain-bound AGI scales by expanding its computational substrate.

16.8.6 Opaque Processes vs. Governable Cognition

Agent reasoning is opaque: intermediate steps are encoded in hidden activations and prompt transcripts. Domain-bound AGI externalizes all cognitive state into structured memory, enabling inspection, auditing, and governance. This transparency is essential for safety, reliability, and long-term deployment.

16.8.7 Two Complementary but Unequal Paradigms

AI agents excel in integration, tooling, and rapid deployment. They provide a practical interface between monolithic models and real-world environments. Domain-bound AGI excels in cognitive structure, stability, and extensibility. It provides the architectural foundations required for robust, interpretable, and scalable reasoning.

The two paradigms are therefore complementary in practice but unequal in principle. AI agents offer immediate utility; domain-bound AGI offers a path toward governed, compositional intelligence.

16.8.8 Summary

AI agents and domain-bound AGI represent two divergent paths toward intelligent systems. Agents wrap external scaffolding around monolithic models to simulate reasoning, while domain-bound AGI constructs reasoning through internal, compositional mechanisms. The former is a practical engineering solution; the latter is a cognitive architecture. Understanding this distinction clarifies the role of each paradigm and highlights why domain-bound AGI provides a more powerful and principled foundation for general-purpose reasoning.

16.9 Integrating Large Language Models Inside Domain-Bound AGI

A final distinction between AI agents and domain-bound AGI concerns the role of large language models themselves. In contemporary agent systems, the LLM is the *entire* reasoning engine: all planning, memory reconstruction, tool selection, and decision-making are delegated to a single monolithic model. The agent framework merely wraps this model with external scaffolding. This design places the full cognitive burden on the LLM and limits the system’s ability to scale, govern, or extend its reasoning capabilities.

Domain-bound AGI offers a fundamentally different perspective. Because the architecture provides its own structured memory, control flow, search substrate, and expert ecosystem, a large language model need not serve as the opaque core of cognition. Instead, it can be integrated as a *bounded internal expert module*—a powerful semantic component inside a larger cognitive system. The architecture becomes the substrate that enables LLMs to contribute to AGI-like behaviour without requiring them to simulate cognition on their own.

16.9.1 LLMs as Internal Cognitive Enhancers

Within the domain-bound AGI architecture, a large language model functions as a specialized expert that contributes to reasoning rather than defining it. The LLM is invoked when high-level semantic capabilities are required, such as:

- generating candidate hypotheses or interpretations,
- proposing alternative formulations of a problem,
- performing semantic clustering or abstraction,
- translating between symbolic and natural-language representations,
- or providing domain-specific heuristics for search modules.

In this configuration, the LLM becomes one expert among many. It does not maintain state, orchestrate workflows, or simulate search. Instead, it provides semantic leverage that complements the architecture’s explicit cognitive mechanisms—structured memory, deterministic control flow, and pluggable experts. Neural capabilities enhance, rather than replace, symbolic reasoning.

16.9.2 A Role Distinct From External Coordination Models

Although large language models are often used in contemporary systems to coordinate or interpret interactions between multiple components, their role inside domain-bound AGI is fundamentally different. Here, the LLM participates directly in the cognitive process as an *internal semantic tool*. It contributes content, not control. The architecture—not the LLM—manages state, flow, and decision-making.

This distinction is crucial: the LLM becomes a component of the cognitive substrate rather than the substrate itself. It is embedded within a structured reasoning environment that constrains, interprets, and integrates its outputs. A later chapter will revisit this distinction in more detail when discussing the broader AGI ontology and the concept of mediator models.

16.9.3 Capabilities Beyond Contemporary Agent Systems

This internal integration enables forms of reasoning that are impossible for current agent frameworks:

- **Hybrid symbolic–neural workflows:** the LLM can generate hypotheses that are then validated, refined, or rejected by symbolic experts and search modules.
- **Deep search with neural heuristics:** search engines such as Monte Carlo or A* can incorporate LLM-derived heuristics without relying on the LLM to simulate the search process.
- **Parallel semantic exploration:** multiple LLM calls can be executed asynchronously as part of a larger reasoning tree, coordinated by the architecture’s callback mechanism.
- **Transparent reasoning:** because the architecture externalizes all state, the contributions of the LLM become explicit and auditable, unlike the opaque activations of monolithic agent loops.

These capabilities illustrate how domain-bound AGI leverages large language models more effectively than agent systems—not by giving them more control, but by giving them a more precise and bounded role within a principled cognitive framework.

16.9.4 A New Purpose for Large Language Models

In this architecture, large language models serve a new and more substantial purpose:

LLMs become semantic engines inside a structured cognitive system, rather than monolithic engines that must simulate cognition on their own.

This reframing unlocks a richer design space. Instead of relying on a single model to perform all reasoning, the system combines:

- explicit search,
- symbolic computation,
- structured memory,
- deterministic experts,

- and neural semantic modules.

The result is a hybrid cognitive architecture in which LLMs amplify reasoning without dominating it. The AGI substrate provides the memory, control flow, and expert structure that allow LLMs to contribute reliably and coherently. In this sense, the architecture acts as the cognitive glue that enables large language models to participate in AGI-like behaviour while remaining bounded, interpretable, and extensible.

Interpreting the Architecture in Contemporary Terms

Although domain-bound AGI is not an agent framework, many readers will naturally interpret it as a new way to construct LLM-based agents that exhibit far more reliable and structured behaviour. This is a useful perspective: the architecture provides the cognitive substrate that contemporary agent systems lack—explicit memory, deterministic control flow, modular experts, and pluggable search—while allowing large language models to contribute semantic capabilities inside a principled reasoning environment.

In practice, this means that the architecture can be adopted as a powerful foundation for building agent-like systems, even though its scope and capabilities extend far beyond the agent paradigm. It offers a familiar entry point for practitioners while introducing the deeper cognitive mechanisms required for scalable, transparent, and extensible reasoning.

16.9.5 Summary

Domain-bound AGI provides a principled way to integrate large language models as internal cognitive enhancers. This role is distinct from the monolithic LLM at the center of agent systems and from external coordination models that rely on LLMs to mediate interactions. By embedding LLMs inside a structured cognitive environment, the architecture enables deeper, more transparent, and more scalable reasoning than contemporary agent frameworks can achieve. This integration illustrates how large language models can contribute to general intelligence without being the sole locus of cognition.

16.10 Conclusion

This chapter has contrasted two paradigms that are often conflated in contemporary discussions of artificial intelligence: externally orchestrated AI agents and the internal, compositional architecture of domain-bound AGI. While both approaches can exhibit similar surface behaviours—planning, tool use, multi-step reasoning, and autonomous workflows—their underlying mechanisms differ profoundly. AI agents wrap monolithic language models with external control logic, relying on prompting, transcripts, and ad hoc orchestration to simulate cognitive structure. Domain-bound AGI constructs cognition from explicit architectural components: structured memory, callback-driven control flow, pluggable search, modular experts, and a domain-agnostic reasoning core.

The comparison reveals that AI agents excel in integration, ecosystem maturity, and rapid deployment. They provide practical interfaces to real-world systems and benefit from a rich landscape of frameworks and tooling. Yet these strengths coexist with structural limitations: prompt-based state, opaque intermediate reasoning, simulated search, fragile long-horizon workflows, and the absence of mechanisms for self-expansion.

Domain-bound AGI addresses these limitations by embedding cognitive structure directly into the architecture. Reasoning becomes a controlled recursive process grounded in canonical rewriting,

variable binding, and flow-based attention. Memory becomes a persistent, inspectable workspace rather than a growing transcript. Search becomes explicit and algorithmic rather than emergent and approximate. Learning becomes a matter of integrating new experts and domain specifications rather than modifying model parameters. The result is a system that is more stable, more transparent, more extensible, and more governable than agent-based approaches.

A final insight concerns the role of large language models themselves. In agent systems, the LLM is the entire cognitive engine. In domain-bound AGI, the LLM becomes one expert among many—a semantic module embedded inside a broader cognitive substrate. This reframing opens new possibilities for hybrid symbolic–neural reasoning, deep search augmented by neural heuristics, and parallel semantic exploration coordinated through structured callbacks. It offers a new purpose for large language models: not as monolithic simulators of reasoning, but as internal cognitive enhancers within a principled architecture.

Taken together, these observations clarify the relationship between AI agents and domain-bound AGI. Agents provide practical utility and immediate applicability; domain-bound AGI provides the architectural foundations for scalable, interpretable, and extensible intelligence. The two paradigms are complementary in practice but fundamentally different in principle. By distinguishing between behavioural similarity and architectural divergence, we can better understand the limitations of current agent systems and appreciate the promise of a compositional approach to general intelligence.

Transition: From Architectural Insight to Practical Demonstration

The insight that large language models can function as internal cognitive tools within the domain-bound AGI architecture has important implications for the practical examples that follow. In the next chapter, we examine a toy board environment implemented using the core components of the architecture: the script engine, the search engine, the prompt engine, and the structured memory map. This implementation deliberately omits LLM integration in order to highlight the behaviour of the architecture itself.

Yet the toy board example becomes even more compelling when viewed through the lens of the preceding insight. During development, a large language model was repeatedly asked to implement the expert interfaces and the domain-specific logic required for new board games. These generated components were then manually integrated into the toy board engine, and the system functioned correctly without any modification to the underlying architecture. This demonstrated, in practice, that any game or problem compatible with the domain-bound expert interfaces can be incorporated into the system and then reasoned about through explicit search. The architecture did not need to reimplement domain logic internally; instead, the LLM provided the semantic and structural components, while the AGI substrate provided the stable, traceable machinery for reasoning.

In a fully integrated system, this workflow would become internal and automatic. A large language model could serve as a semantic interface between human descriptions and the architecture’s internal representations. For example:

- The LLM could translate a natural-language description of a board position into a structured internal state stored in memory.
- The search engine could then operate directly on this structured state to explore possible moves.
- The reasoning core could integrate the search results and determine the next action.

- The LLM could convert the internal move representation back into readable natural language for the user.

In this workflow, the LLM does not perform search or maintain state; it acts as a semantic transformer that enriches the architecture without replacing it. Even more powerfully, the LLM could generate the code interfaces, expert specifications, and domain adapters required for the system to support a new game. The architecture could then integrate these components as new experts, expanding its cognitive substrate without manual engineering. This represents a form of self-expansion that leverages LLMs precisely for what they excel at: semantic interpretation, structural translation, and code synthesis—without burdening them with control flow, memory, or search.

This perspective reveals the deeper significance of the toy board example. It is not merely a demonstration of a small game-playing system; it is a minimal instantiation of a general cognitive architecture that has already shown, in practice, that it can ingest new domains through LLM-generated interfaces. The next chapter therefore serves as a practical foundation upon which far more powerful hybrid systems can be built—systems in which a lightweight AGI core can internally leverage large language models to expand its own capabilities while remaining fully traceable, interpretable, and structurally grounded.

Architectural Insight: A New Role for Large Language Models

The domain-bound AGI architecture introduces a fundamentally new computational role for large language models. Instead of serving as monolithic reasoning engines or external mediators, LLMs become *internal cognitive experts* embedded within a structured reasoning substrate. A lightweight AGI controller can invoke powerful LLMs as semantic engines, hypothesis generators, heuristic providers, or abstraction modules—without relying on them for state, control flow, or search.

This inversion transforms the relationship between architecture and model. Reasoning is governed by explicit mechanisms—structured memory, callback-driven flow, and pluggable search—while LLMs contribute bounded, auditable semantic capabilities inside that framework. A small, stable AGI core can therefore leverage extremely large models as internal tools, achieving forms of hybrid symbolic–neural reasoning, deep search with neural heuristics, and parallel semantic exploration that contemporary agent systems cannot support. In this architecture, LLMs gain a new purpose: not as simulators of cognition, but as amplifiers of a principled cognitive system.

Chapter 17

Toy Board: A Practical Domain-Bound AGI Example

Historical Note

Exactly forty years after Minsky’s *Society of Mind* (1986), Toy Board provides a miniature operational realization of his vision: a working society of small cognitive processes—search, evaluation, memory, scripts, and domain logic—interacting to solve structured problems. It is a universal boardgame ontology that maps naturally to puzzles, sorting tasks, planning domains, and reasoning environments.

17.1 Introduction: Why Toy Board?

Toy Board is a compact but fully operational example of *domain-bound artificial general intelligence*. It demonstrates how a small, modular reasoning system can solve heterogeneous problems—games, puzzles, planning tasks, and even sorting—through a unified cognitive architecture. Unlike transformer-based language models, which internalize all world knowledge into a single monolithic network, Toy Board separates *reasoning* from *domain logic*. The result is a transparent, interpretable, and extensible system that behaves like a miniature cognitive agent.

At its core, Toy Board implements a universal *board-game ontology*. Every domain, whether it is Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, or even a sorting task, is mapped onto the same abstract interface: a **BoardState**, a set of legal **BoardMoves**, and a deterministic **BoardDomain** that defines the transition rules. This abstraction is powerful: it allows a single reasoning model to operate across many domains without ever learning their internal mechanics. The model does not know how Sudoku works or how to sort a list; instead, it learns how to *think* about problems expressed through this ontology.

Toy Board’s reasoning engine is built on the PAGED–PRE–CAS architecture introduced in Toy Story, but repurposed for procedural cognition. Instead of generating text, the model generates *programs*: structured sequences of **STORE** and **CALL** instructions, variable bindings, search invocations, evaluations, and termination markers. These programs are executed by a deterministic Script Engine that coordinates memory, search engines, domain transitions, and failure detection. The system therefore does not merely predict the next token; it constructs and executes a plan.

This architectural separation—a small neural core for reasoning, surrounded by modular domain engines for world dynamics—has profound implications. A Toy Board model with approximately

one megabyte of parameters can coordinate eight heterogeneous domains. Extrapolated to full AGI, the same architectural principles would scale to terabyte-sized systems: large, but feasible. In contrast, a transformer-based AGI attempting to internalize all domain logic would require exabyte-scale models, pushing far beyond the limits of practical computation. Toy Board thus provides a concrete, operational argument for why AGI must be modular, structured, and tool-driven rather than monolithic.

This chapter presents Toy Board as a practical, working example of domain-bound AGI. It introduces the cognitive vocabulary, paging structure, PRE reasoning loop, evaluator system, arbiter, script engine, and search integration that together form a complete reasoning pipeline. It also shows how diverse domains can be mapped onto a single ontology, and why this mapping is a powerful abstraction for scalable intelligence.

Toy Board is not a toy in the trivial sense. It is a blueprint: a minimal, executable realization of an AGI architecture that separates reasoning from world knowledge, delegates complexity to modular engines, and uses explicit structure instead of statistical correlation. It demonstrates that intelligence does not require massive models, but the right architecture—one that reasons, plans, and decides.

17.2 The Domain-Bound AGI Problem

Artificial General Intelligence is often framed as a monolithic challenge: a single model that must internalize all world knowledge, all reasoning strategies, all domains, and all forms of problem solving. This framing implicitly assumes that intelligence is a property of a single, undifferentiated function approximator. Toy Board challenges this assumption. It demonstrates that AGI can instead be approached as a *domain-bound* problem: a system that reasons universally, but acts within well-defined interfaces that encapsulate domain-specific rules.

Domain-bound AGI is not a weaker form of intelligence. It is a more structured one. Humans do not store the rules of chess, arithmetic, navigation, and language in a single undifferentiated neural substrate. We rely on modular cognitive processes, external tools, symbolic representations, and learned procedures that interact through shared working memory and control flow. Toy Board adopts the same principle. It separates the *reasoning core* from the *domain engines*, allowing each to specialize without interference.

The central challenge of domain-bound AGI is therefore not to learn the rules of every possible domain, but to learn the *cognitive glue* that binds domains together: classification, planning, search invocation, evaluation, memory management, and termination. These are the universal operations that allow an agent to operate across heterogeneous environments. Toy Board shows that these operations can be learned by a small PRE-CAS model, provided that the domains themselves expose a uniform interface.

This leads to a crucial insight: the difficulty of AGI does not lie in the complexity of the domains, but in the complexity of the *interfaces* that connect reasoning to action. Once a domain is expressed through a simple ontology—a board state, a set of legal moves, and a deterministic transition function—the reasoning model can treat it as just another environment. Connect Four, Nim, MiniChess, Sliding Puzzle, Sorting, and Sudoku all become instances of the same abstract problem: *given a state, propose and evaluate actions until a goal is reached or failure is detected*.

Toy Board therefore reframes AGI as an architectural problem rather than a scaling problem. Instead of increasing model size to absorb more domains, it increases the *expressive power of the interfaces* that connect a small reasoning core to a growing collection of domain engines. This modularity is what makes Toy Board scalable, interpretable, and computationally feasible. It is

also what makes it a compelling example of domain-bound AGI: a system that reasons universally, but acts locally.

In the sections that follow, we examine how Toy Board implements this principle in practice. We introduce the cognitive vocabulary, paging structure, PRE reasoning loop, evaluators, arbiter, script engine, and search integration that together form a complete domain-bound AGI pipeline. We also show how diverse domains can be mapped onto a single board-game ontology, and why this mapping is a powerful abstraction for general-purpose problem solving.

17.3 The Toy Board Architecture Overview

Toy Board is built on a simple but powerful idea: a small reasoning core can coordinate multiple cognitive processes—classification, search, evaluation, memory, and control flow—provided that each process is exposed through a clean, uniform interface. The architecture therefore consists of a compact neural component surrounded by a set of deterministic modules that implement domain logic, search strategies, and program execution. Together, these components form a complete domain-bound AGI pipeline.

At a high level, Toy Board operates in three phases:

1. **Reasoning:** The PRE–CAS model generates a structured program consisting of **STORE** and **CALL** instructions, variable bindings, and termination markers. This program is not text but executable cognitive code.
2. **Execution:** The Script Engine interprets the program deterministically. It manages variables, invokes search engines, queries domain rules, updates board states, and detects termination or failure.
3. **World Interaction:** Domain engines implement the rules of each environment. They define legal moves, state transitions, heuristic scores, and win conditions. The reasoning model never learns these rules; it merely orchestrates their use.

This separation of concerns is the defining feature of Toy Board. The neural model is responsible for *how to think*, while the domain engines are responsible for *what happens* when an action is taken. The architecture therefore avoids the monolithic entanglement of reasoning and world knowledge that characterizes large language models.

Components of the Architecture

Toy Board consists of the following major components:

- **Token Dictionary and Cognitive Vocabulary** A compact set of control tokens, variable tokens, and domain-agnostic words that define the language of reasoning.
- **Paging System** A decomposition of the vocabulary into small expert pages, each responsible for a coherent subset of cognitive operations. Paging enforces structural correctness and reduces interference between unrelated token types.
- **PRE Reasoning Loop** A propose–refine–evaluate cycle that generates candidate tokens, filters them through evaluators, and accumulates valid proposals for the arbiter.
- **Evaluators** A set of orthogonal micro-experts that enforce variable integrity, script structure, domain consistency, procedural coherence, and syntactic correctness.

- **Arbiter** A decision-maker that integrates evaluator scores, injects controlled variation, and selects the next token from the refined candidate set.
- **Script Engine** A deterministic interpreter that executes the generated program, manages memory, invokes search engines, updates board states, and detects abort conditions.
- **Domain Engines** Modular implementations of `BoardDomain`, `BoardState`, and `BoardMove` for each environment. These engines encapsulate all domain rules and provide a uniform interface for reasoning.

A Cognitive Pipeline

The interaction between these components forms a complete cognitive pipeline:

1. The user provides a natural-language question.
2. The PRE-CAS model generates a reasoning program token by token.
3. The Script Engine executes the program deterministically.
4. Domain engines compute legal moves, transitions, and evaluations.
5. Search engines explore the state space when required.
6. The Script Engine reports the result or aborts if no solution exists.

Every step is transparent, interpretable, and reproducible. There are no hidden states, no implicit memories, and no statistical shortcuts. Toy Board therefore provides a rare opportunity to observe a complete AGI-style reasoning loop in a fully inspectable form.

Why This Architecture Matters

Toy Board demonstrates that AGI does not require a monolithic model that internalizes all domains. Instead, it shows that a small reasoning core can coordinate modular processes through explicit structure, paging, evaluators, and deterministic execution. This architecture scales with the number of domains not by increasing model size, but by adding new domain engines that implement the shared ontology.

In the sections that follow, we examine each component of this architecture in detail, beginning with the cognitive vocabulary and the paging system that routes control between expert networks.

17.4 Tokenization and the Cognitive Vocabulary

Toy Board does not operate on natural language tokens or subword fragments. Instead, it uses a compact, fully deterministic *cognitive vocabulary*: a set of control tokens, variable tokens, and domain-agnostic words that together form a procedural language for reasoning. This vocabulary is not designed for expressiveness in the linguistic sense, but for expressiveness in the *cognitive* sense: it encodes the operations, memory structures, and control flow required to solve problems across heterogeneous domains.

The tokenization scheme is deliberately minimal. Each token corresponds to a discrete cognitive action or symbol, and each is mapped to a stable integer identifier through a deterministic hashing function. There is no ambiguity, no learned embedding space, and no context-dependent interpretation. The vocabulary defines the ontology of Toy Board’s reasoning process.

Control Tokens

Toy Board introduces a small set of control tokens that form the backbone of its procedural language:

- **STORE** — binds a natural-language question to a variable.
- **CALL** — invokes a cognitive subroutine (classification, search, evaluation, repeat).
- **EOL** — marks the end of a reasoning line.
- **EOR** — marks the end of the reasoning program.

These tokens define the structure of the reasoning trace. They are not optional: every valid program must begin with a **STORE**, proceed through a sequence of **CALL** instructions, and terminate with **EOR**. The Script Engine interprets these tokens deterministically, making the reasoning process transparent and reproducible.

Variable Tokens

Toy Board uses explicit variable tokens to represent identifiers and intermediate results:

- **\$BID** — the board identifier.
- **\$PID** — the prompt identifier.
- **\$MOVE** — the move proposed by a search engine.
- **\$PLAYER** — the active player in the domain.

These variables form the working memory of the system. They are created, updated, and queried by the Script Engine, and their integrity is enforced by the Variable Evaluator. This explicit memory mechanism stands in sharp contrast to transformer-based models, which must represent such information implicitly in hidden activations.

Domain-Agnostic Cognitive Words

In addition to control and variable tokens, Toy Board includes a small set of domain-agnostic words that describe cognitive operations:

- **classify** — determine the domain of a board.
- **search** — invoke a domain-specific search engine.
- **evaluate** — determine whether the game is won, lost, or ongoing.
- **repeat** — re-enter the reasoning loop with updated state.

These words are not tied to any specific environment. They represent universal cognitive operations that apply equally to Connect Four, Nim, MiniChess, Sudoku, and Sorting. This is what makes Toy Board domain-bound rather than domain-specific: the reasoning model learns the *structure* of problem solving, not the rules of any particular domain.

A Procedural Language for Reasoning

Taken together, the cognitive vocabulary forms a small procedural language. A typical reasoning trace might look like:

```
<STORE> what should be the next move for $BID
<CALL> classify $BID into $PID
<CALL> search $MOVE for $BID into $PID
<CALL> evaluate $BID into $PID
<CALL> repeat prompt into $PID
<EOR>
```

This is not text generation. It is program synthesis. The PRE–CAS model constructs a program token by token, the Script Engine executes it, and the domain engines provide the world dynamics. The vocabulary is therefore not a linguistic medium but a cognitive one.

Why This Vocabulary Matters

The cognitive vocabulary is the foundation of Toy Board’s architecture:

- It defines the space of possible reasoning programs.
- It enforces structural constraints through paging and evaluators.
- It enables explicit memory through variable tokens.
- It separates reasoning from domain logic.
- It allows the Script Engine to execute reasoning deterministically.

By reducing reasoning to a small, interpretable language, Toy Board achieves something rare: a complete AGI-style reasoning loop that is transparent, modular, and fully inspectable. In the next section, we examine how paging decomposes this vocabulary into expert networks that enforce structural correctness and cognitive specialization.

17.5 Paging: Routing Cognitive Control

Toy Board inherits the paging mechanism from Toy Story, but repurposes it for a very different cognitive landscape. In Toy Story, paging decomposed the vocabulary into semantic domains such as actors, timing words, and structural markers. In Toy Board, paging becomes a *control-flow router*: it determines which expert network is allowed to speak next, based on the structure of the reasoning program being constructed.

The vocabulary of Toy Board is not linguistic but procedural. It consists of control tokens (STORE, CALL, EOL, EOR), variable tokens (\$BID, \$PID, \$MOVE), and domain-agnostic cognitive words (classify, search, evaluate, repeat). These tokens do not form sentences; they form *programs*. Paging ensures that each token is produced by the appropriate expert, enforcing structural correctness and preventing invalid transitions.

A Decomposition of Cognitive Roles

Toy Board partitions its vocabulary into a set of small, non-overlapping pages, each corresponding to a distinct cognitive role:

- **STORE Page** — responsible for variable-binding instructions.
- **CALL Page** — responsible for invoking cognitive subroutines.
- **Variable Page** — responsible for producing variable identifiers.
- **Cognitive-Word Page** — responsible for tokens such as `classify`, `search`, `evaluate`, and `repeat`.
- **Structural Pages** — deterministic pages for EOL and EOR.

Each page contains only the tokens relevant to its role. The PRE-CAS model therefore never faces a large softmax over the entire vocabulary; it selects from a small, coherent subset of tokens that are structurally valid in the current context.

Paging as a Structural Constraint

Paging enforces strong structural constraints on the reasoning program:

- After **STORE**, only variable tokens or cognitive words may appear.
- After **CALL**, the next token must be a cognitive word or a variable.
- After a cognitive word such as `search`, the next token must be a variable.
- EOL may only appear at the end of a reasoning line.
- EOR may only appear when the program is complete.

These constraints eliminate entire classes of errors. A page that contains only variable tokens cannot accidentally output **CALL**; a page that contains only structural tokens cannot output `search`. Paging therefore acts as a built-in safety mechanism that keeps the reasoning program within the boundaries of the cognitive language.

Paging as Cognitive Routing

In Toy Board, paging is not merely an optimization; it is a form of *cognitive routing*. It determines which expert network is responsible for the next step in the reasoning process. This routing is based on the last token in the context window, which encodes the current phase of the program:

- **STORE** activates the variable-binding expert.
- **CALL** activates the subroutine-invocation expert.
- Cognitive words activate the variable or structural experts.
- EOL activates the expert responsible for line termination.

This mechanism ensures that the PRE-CAS model always operates within a valid procedural context. It also reduces interference between unrelated cognitive operations, making the system easier to train, debug, and interpret.

Why Paging Matters

Paging provides several key advantages in the context of domain-bound AGI:

- **Structural Safety** — invalid token transitions become impossible by construction.
- **Modularity** — each expert specializes in a narrow cognitive role.
- **Interpretability** — errors can be traced to specific pages.
- **Stability** — small softmax layers converge quickly and reliably.
- **Scalability** — new cognitive operations can be added by introducing new pages.

In Toy Board, paging is the mechanism that transforms a small neural network into a structured reasoning engine. It ensures that the PRE–CAS model generates programs rather than text, and that these programs adhere to the rules of the cognitive language.

In the next section, we examine how the PRE reasoning loop uses paging, evaluators, and feedback to construct valid reasoning programs token by token.

17.6 PRE: Propose–Refine–Evaluate for Procedural Reasoning

Toy Board uses the same PRE mechanism introduced in Toy Story, but in a radically different context. Instead of generating narrative tokens, PRE constructs *programs*: structured sequences of **STORE** and **CALL** instructions, variable bindings, search invocations, evaluations, and termination markers. PRE is therefore not a stylistic refinement loop but a *procedural reasoning engine*. It transforms token prediction into a multi-step cognitive process that resembles planning, correction, and decision-making.

PRE operates at the heart of Toy Board’s architecture. It is the mechanism that allows a one-megabyte model to produce valid reasoning traces across heterogeneous domains without ever learning the rules of those domains. PRE ensures that each token is proposed, evaluated, and refined within the structural constraints of the cognitive language.

The Propose Step

In the propose phase, the active page-expert receives:

- the fixed-position encoding of the context window,
- the target CAS value,
- a role indicator specifying that this is an initial proposal.

The expert produces a softmax distribution over the tokens in its page. The highest-logit token becomes the initial proposal, but it is not accepted immediately. Instead, it is passed to the evaluators, which assess:

- variable integrity,
- script structure,
- domain consistency,

- procedural coherence,
- CAS alignment.

The result is a structured evaluation object that captures both the token and its constraint profile.

The Refinement Steps

After the initial proposal, PRE performs a sequence of refinement iterations. In each iteration, the expert receives a richer input:

- the context window,
- the target CAS value,
- the previous proposal,
- the evaluator scores associated with that proposal.

This feedback loop allows the expert to correct its own mistakes. A refinement step may:

- replace an invalid variable with a valid one,
- adjust a CALL structure to match the expected syntax,
- select a different cognitive word that better fits the context,
- avoid premature termination,
- improve procedural coherence.

Each refinement produces a new candidate token, which is again evaluated. All valid proposals across all refinement steps are accumulated into a candidate pool.

Evaluation as a Procedural Constraint System

Unlike transformer-based models, where constraints are applied after sampling, Toy Board applies constraints *before* selection. The evaluators act as independent micro-experts that enforce:

- syntactic correctness,
- variable consistency,
- domain validity,
- structural form,
- procedural continuity.

Invalid proposals are discarded immediately. Refinement is guided by explicit feedback, ensuring that the candidate pool contains only structurally valid and semantically coherent tokens.

PRE as a Cognitive Process

PRE transforms token generation into a multi-step cognitive process:

1. propose an initial candidate,
2. evaluate it through orthogonal constraints,
3. refine based on explicit feedback,
4. accumulate all valid proposals,
5. pass the candidate set to the arbiter.

This mirrors human reasoning: we generate an idea, check it against constraints, revise it, and consider alternatives before committing. PRE is therefore a computational realization of deliberation.

Why PRE Matters for Domain-Bound AGI

PRE is essential for Toy Board’s ability to operate across heterogeneous domains:

- **Correctness** — invalid tokens are filtered before selection.
- **Robustness** — refinement corrects weak or borderline proposals.
- **Generalization** — the same reasoning loop applies to all domains.
- **Interpretability** — every proposal and refinement step is traceable.
- **Modularity** — evaluators and experts remain independent agents.

In Toy Board, PRE is the mechanism that turns a small neural network into a structured, domain-agnostic reasoning engine. It is the bridge between paging, evaluators, and the arbiter, and it is the foundation of the system’s ability to generate executable cognitive programs.

In the next section, we examine the evaluators that provide the constraint signals guiding PRE’s refinement process.

17.7 Evaluators: Coherence as the Only Learned Critic

In Toy Story, multiple evaluators explicitly encoded style, timing, line structure, and actor consistency. Toy Board takes a different route. Here, almost all structure is enforced *architecturally* rather than through learned evaluators. The only true evaluator in the system is a small learned *Coherence model*; all other constraints emerge from the combination of prompt rewriting, explicit variables, paging, and the deterministic Script Engine.

This design choice is deliberate. Toy Board is a domain-bound AGI system: it relies on explicit program structure, a fixed cognitive vocabulary, and a strict execution model. Most of what transformer-based systems would need to learn as “soft constraints” is instead hard-coded into the language of reasoning and the interpreter that executes it.

Structural Constraints Without Evaluators

Toy Board enforces the majority of its correctness properties without learned evaluators:

- **Program grammar** is enforced by the Script Engine. Invalid sequences (e.g., `CALL` without a verb, missing variables, misplaced `EOR`) simply cannot be executed and are treated as failures.
- **Variable discipline** is enforced by explicit variable tokens and the interpreter. Variables such as `$BID`, `$PID`, and `$MOVE` must be bound before use; otherwise, execution fails.
- **Domain consistency** is enforced by the domain interfaces. Each `BoardDomain` implementation defines which operations are valid; illegal calls result in execution errors or abort.
- **Control flow** is enforced by the cognitive vocabulary and paging. The model can only choose from structurally valid token types in each context, making many syntactic errors impossible by construction.
- **Attention-like behavior** is achieved through prompt rewriting and variable reuse. The model repeatedly sees the current board, the last move, and the relevant identifiers in a structured prompt, focusing its reasoning on the right parts of the state.

These mechanisms act as *implicit evaluators*: they do not score tokens, but they constrain what can be generated and what can be executed. The only component that actually assigns a learned score to a proposal is the Coherence model.

The Coherence Model: Learning Procedural Continuity

The Coherence model is a small neural network trained to distinguish good reasoning steps from bad ones. It is the only evaluator in Toy Board in the strict sense: it takes a candidate token (together with its context) and outputs a continuous score between 0 and 1 that reflects how *procedurally coherent* that token is.

Training data for the Coherence model is constructed from:

- valid reasoning traces produced by the system,
- contrastive micro-errors such as:
 - misplaced `CALL` instructions,
 - incorrect variable usage,
 - premature or missing `EOR`,
 - unnatural ordering of cognitive operations.

The Coherence model does not enforce hard rules; those are already handled by the architecture. Instead, it captures *soft regularities* in procedural behavior, such as:

- `classify` tends to appear early in the program,
- `search` usually follows classification,
- `evaluate` often precedes a decision to repeat or terminate,
- `repeat` is unlikely to appear before any search has been performed.

These patterns are not strictly required by the grammar, but they make reasoning more natural, efficient, and robust.

How Coherence Interacts with PRE

Within the PRE loop, the Coherence model provides a scalar signal for each proposed token. This signal is used to:

- down-rank proposals that are procedurally odd or implausible,
- up-rank proposals that fit well with the surrounding context,
- guide refinement toward more coherent CALL sequences,
- help the arbiter distinguish between multiple structurally valid options.

Crucially, Coherence does not replace structural checks. A token that violates the program grammar or variable discipline is rejected outright by the architecture, regardless of its coherence score. The evaluator’s role is to shape behavior *within* the space of structurally valid programs.

Why a Single Evaluator Is Enough

Toy Board shows that a rich evaluator stack is not always necessary. Because so much structure is made explicit in:

- the cognitive vocabulary,
- the paging layout,
- the Script Engine,
- the domain interfaces,
- and the prompt rewriting scheme,

a single learned Coherence model is sufficient to capture the remaining degrees of freedom in procedural behavior. This keeps the system small, interpretable, and easy to extend.

In the next section, we examine how Toy Board filters candidate tokens *before* selection, combining structural constraints and coherence scores into a curated set of valid proposals for the arbiter.

17.8 Filtering and Arbitration: A Unified Decision Mechanism

Toy Board inherits the general PRE–CAS decision structure from Toy Story, but its implementation is significantly simpler and more architectural. Because Toy Board has only one learned evaluator (the Coherence model) and relies heavily on structural constraints enforced by paging, variables, and the Script Engine, the distinction between “filtering” and “arbitration” becomes conceptually thinner than in Toy Story. Both mechanisms work together to ensure that only structurally valid and procedurally coherent tokens are ever selected.

This section describes the unified decision mechanism that governs how Toy Board chooses the next token in a reasoning program.

Structural Filtering Before Scoring

Before any scoring or arbitration occurs, Toy Board applies a strict structural filter. This filter is not learned; it emerges directly from the architecture:

- **Paging** restricts the active vocabulary to the correct token type.
- **The program grammar** rejects illegal sequences (e.g., `CALL` without a verb).
- **Variable discipline** ensures that identifiers such as `$BID` and `$MOVE` are bound before use.
- **Domain interfaces** reject operations that are not supported by the current board.
- **The Script Engine** enforces execution validity and detects impossible actions.

These constraints eliminate the majority of invalid proposals *before* the Coherence model is even consulted. As a result, Toy Board does not require a large evaluator stack: most errors are impossible by construction.

Coherence as the Only Learned Signal

After structural filtering, the remaining candidates are scored by the Coherence model. This model provides a single scalar:

$$s = \text{coherenceScore},$$

which reflects how procedurally natural the token is within the current context. The score captures soft regularities in reasoning behavior—for example, that `classify` tends to appear early, or that `search` typically follows classification.

Because all hard constraints are enforced architecturally, Coherence does not need to police syntax or domain rules. Its role is to guide the system toward more fluent and robust reasoning patterns.

Candidate Pool Construction

Toy Board constructs a candidate pool in two steps:

1. **Select the first structurally valid token as the anchor.** Tokens are considered in descending logit order. The first token that passes all structural checks becomes the anchor candidate.
2. **Add additional candidates based on a coherence threshold.** For each remaining structurally valid token, Toy Board includes it in the candidate pool if:

$$s \geq s_{\text{anchor}} \times \text{scoreThreshold}.$$

This ensures that the arbiter receives a curated set of high-quality alternatives: all structurally valid, all procedurally coherent, and all close in quality to the anchor.

Arbitration: Controlled Variation Among Valid Options

Once the candidate pool is constructed, the arbiter performs a simple but effective selection:

- If the pool contains only one candidate, it is selected deterministically.
- If multiple candidates qualify, the arbiter selects uniformly at random.

This mechanism injects controlled variation without sacrificing correctness. Because all candidates have already passed structural filtering and coherence thresholding, the arbiter’s randomness is bounded and safe.

A Unified Decision Pipeline

In Toy Board, filtering and arbitration form a single cognitive pipeline:

1. eliminate structurally invalid tokens,
2. score remaining tokens with the Coherence model,
3. construct a high-quality candidate pool,
4. select one candidate with bounded randomness.

This pipeline is simpler than Toy Story’s evaluator-rich architecture, but also more powerful in a procedural setting. By shifting most constraints into the architecture itself, Toy Board reduces the burden on the neural model and ensures that every selected token is both executable and coherent.

In the next section, we examine the Script Engine, which executes the generated program and connects reasoning to world dynamics.

17.9 The Script Engine: A Deterministic Interpreter for Cognitive Programs

Toy Board differs fundamentally from transformer-based systems in one respect: its output is not text but an *executable program*. Every reasoning trace produced by the PRE-CAS model is a sequence of **STORE** and **CALL** instructions, variable bindings, and termination markers. The Script Engine is the component that interprets this program, executes its cognitive operations, and connects the reasoning model to the underlying domain engines.

The Script Engine is deterministic. Given the same program and the same board state, it will always produce the same result. This determinism is essential: it ensures that Toy Board’s reasoning is transparent, reproducible, and debuggable. It also allows the system to treat reasoning as a form of program synthesis rather than probabilistic sampling.

Programs, Not Predictions

A typical Toy Board reasoning trace looks like:

```
<STORE> what should be the next move for $BID
<CALL> classify $BID into $PID
<CALL> search $MOVE for $BID into $PID
```

```

<CALL> evaluate $BID into $PID
<CALL> repeat prompt into $PID
<EOR>

```

This is not a sequence of linguistic tokens. It is a *procedural script*. The Script Engine executes it line by line, performing the following operations:

- binding variables,
- invoking domain classifiers,
- calling search engines,
- updating board states,
- evaluating win/loss conditions,
- repeating the reasoning loop when necessary,
- terminating when the program ends.

The Script Engine therefore acts as the cognitive substrate of Toy Board: it is the mechanism that turns symbolic reasoning into concrete action.

Variable Memory as Working Memory

Toy Board uses explicit variables such as \$BID, \$PID, and \$MOVE to represent identifiers and intermediate results. The Script Engine maintains a small, deterministic memory map that stores these variables. It enforces:

- variables must be bound before use,
- variable names must remain consistent,
- variable updates must follow the program grammar,
- variable values must be compatible with the domain.

This explicit working memory is one of the key differences between Toy Board and transformer-based models. Instead of relying on implicit attention patterns, Toy Board manipulates variables directly, making reasoning interpretable and reliable.

Domain Interaction Through a Uniform Ontology

The Script Engine interacts with domain engines through a uniform interface consisting of:

- BoardState,
- BoardMove,
- BoardDomain.

Each domain engine implements:

- legal move generation,
- state transitions,
- win/loss evaluation,
- search integration.

Because all domains conform to the same ontology, the Script Engine can execute the same program structure across Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, Sorting, and other environments. This is what makes Toy Board a domain-bound AGI system: the reasoning model is domain-agnostic, while the Script Engine and domain engines provide the necessary specialization.

Search as a Cognitive Action

When the Script Engine encounters a search instruction:

```
<CALL> search $MOVE for $BID into $PID
```

it invokes a domain-specific search engine (Monte Carlo or Beam Search). The search engine explores the state space, selects a promising move, and returns it as the value of `$MOVE`. The reasoning model does not learn how to play the game; it learns when to invoke search.

This separation of reasoning and search is one of Toy Board’s defining features. It allows a small model to solve complex problems by delegating combinatorial exploration to deterministic tools.

Abort as Failure Awareness

If a search engine cannot find a valid move, or if the program requests an impossible operation, the Script Engine produces an explicit abort:

```
search is unable to find solution
```

Abort is not an error. It is a cognitive signal: the system recognizes that its reasoning strategy cannot succeed and terminates gracefully. This form of failure awareness is a key property of intelligent systems and is difficult to achieve in transformer-based models.

Determinism and Interpretability

The Script Engine is fully deterministic:

- the same program always produces the same result,
- the same board state always leads to the same search behavior,
- variable updates are predictable and traceable,
- abort conditions are explicit and reproducible.

This determinism makes Toy Board:

- easy to debug,

- easy to analyze,
- safe to extend,
- suitable for scientific study.

It also provides a clear separation between the neural and symbolic components of the system, allowing each to be understood independently.

Why the Script Engine Matters

The Script Engine is the component that transforms Toy Board from a language model into a cognitive agent. It provides:

- explicit control flow,
- explicit memory,
- explicit domain interaction,
- explicit search invocation,
- explicit failure detection.

It is the operational core of Toy Board's domain-bound AGI architecture.

In the next section, we examine the search engines themselves and how they function as cognitive tools within this architecture.

17.10 Search Engines as Cognitive Tools

Toy Board does not attempt to learn the internal logic of its domains. Instead, it delegates combinatorial reasoning to external search engines that operate through a uniform interface. The PRE-CAS model learns *when* to invoke search, not *how* to perform it. This separation is central to Toy Board's domain-bound AGI architecture: the neural model handles symbolic reasoning and control flow, while deterministic search engines handle combinatorial exploration.

The system currently supports two search engines:

- a Monte Carlo search engine for adversarial or strategic domains,
- a Beam Search engine for puzzles, sorting, and deterministic planning tasks.

Both engines operate on the same `BoardState/BoardMove/BoardDomain` ontology, making them interchangeable from the perspective of the Script Engine.

Monte Carlo Search: Exploration Under Uncertainty

The Monte Carlo search engine is used for domains where the agent must evaluate strategic consequences of moves, such as:

- Connect Four,
- Checkers,

- MiniChess,
- Nim,
- Chomp.

The engine performs repeated playouts from the current state, sampling moves until a terminal state is reached or a cutoff depth is exceeded. Each playout contributes a score that reflects the likelihood of winning from that position. The engine then selects the move with the highest empirical value.

Toy Board’s Monte Carlo engine includes several practical enhancements:

- **Immediate win detection** to avoid unnecessary playouts.
- **Rollout cutoff** to prevent unbounded simulations.
- **Heuristic evaluation** when terminal states are not reached.
- **Configurable payout count** to trade off speed and accuracy.

These features make the engine robust across a wide range of board sizes and branching factors. The reasoning model does not learn any of these heuristics; it simply invokes `search` when appropriate.

Beam Search: Deterministic Planning and Optimization

Beam Search is used for domains where the environment is deterministic and the goal is to reach a specific configuration:

- Sliding Puzzle,
- Sorting,
- Sudoku (partial),
- other planning-style domains.

Beam Search maintains a fixed-size frontier of the most promising states, expanding them layer by layer until a solution is found or the search space is exhausted. This approach is particularly effective for:

- puzzles with large branching factors,
- optimization tasks with clear heuristics,
- domains where depth is more important than breadth.

Toy Board’s Sorting domain is a striking example: the system solves sorting not through a learned algorithm, but through search over swap operations. This demonstrates the power of the board-game ontology: even non-game tasks can be expressed as state transitions and solved through planning.

A Uniform Interface for All Domains

Both search engines operate through the same interface:

- `BoardState` represents the current world configuration.
- `BoardMove` represents an action.
- `BoardDomain` defines legal moves, transitions, and win conditions.

This uniformity allows the Script Engine to invoke search without knowing anything about the domain. The reasoning model simply emits:

```
<CALL> search $MOVE for $BID into $PID
```

and the Script Engine handles the rest.

Search as a Cognitive Action

In Toy Board, search is not an internal neural operation. It is an explicit cognitive action, analogous to a human deciding to:

- calculate variations in a chess position,
- try out moves in a puzzle,
- simulate outcomes before acting.

The reasoning model learns the *conditions* under which search is useful:

- after classification,
- before evaluation,
- when a decision must be made,
- when the board state is ambiguous.

This makes search a first-class citizen in the cognitive architecture.

Abort as a Search Outcome

If a search engine cannot find a valid move or reaches its computational limits, the Script Engine returns an explicit abort:

```
search is unable to find solution
```

This is not an error but a form of metacognition: the system recognizes that its strategy cannot succeed and terminates gracefully. Abort is therefore a meaningful cognitive signal, not a failure of the architecture.

Why Search Engines Matter

Search engines give Toy Board capabilities far beyond what a small neural model could achieve alone:

- **Combinatorial reasoning** without learning domain rules.
- **Generalization** across games, puzzles, and optimization tasks.
- **Scalability** through configurable search depth and breadth.
- **Interpretability** through explicit playouts and frontier expansions.
- **Modularity** through the uniform board-game ontology.

Search is the bridge between symbolic reasoning and world dynamics. It allows Toy Board to solve complex problems with a tiny model by delegating exploration to deterministic tools.

In the next section, we examine how Toy Board maps diverse domains onto a single board-game ontology, enabling this unified search interface.

17.11 A Unified Board-Game Ontology for Heterogeneous Domains

One of Toy Board’s most surprising results is that a single, minimal ontology—originally designed for board games—turns out to be expressive enough to represent a wide range of domains, including puzzles, optimization tasks, and sorting. This ontology consists of only three core abstractions:

- **BoardState** — a representation of the current world configuration,
- **BoardMove** — an action that transforms one state into another,
- **BoardDomain** — the rules governing legal moves, transitions, and evaluation.

Despite its simplicity, this interface is powerful. It allows Toy Board to treat Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, and Sorting as structurally identical problems: each domain becomes a state-transition system with legal actions and terminal conditions. The reasoning model never learns domain rules; it learns how to operate on this ontology.

A Minimal Ontology with Maximum Reach

The ontology is intentionally small. A **BoardState** may contain:

- a grid of pieces (Connect Four, Checkers, MiniChess),
- a list of numbers (Sorting),
- a tile configuration (Sliding Puzzle),
- a constraint grid (Sudoku),
- or any other structured representation.

A **BoardMove** may represent:

- dropping a piece,
- swapping two elements,
- sliding a tile,
- filling a cell,
- or any domain-specific action.

A `BoardDomain` defines:

- how to enumerate legal moves,
- how to apply a move to a state,
- how to detect terminal states,
- how to evaluate intermediate states,
- which search engine to use.

This uniformity is what makes Toy Board domain-bound AGI: the reasoning model is domain-agnostic, while the domain engines provide the specialization.

Mapping Games, Puzzles, and Sorting onto the Same Structure

The most striking demonstration of the ontology's generality is the Sorting domain. Sorting is not a board game, yet it fits naturally into the ontology:

- `BoardState` is the current list,
- `BoardMove` is a swap operation,
- `BoardDomain` defines legal swaps and a heuristic for sortedness.

Toy Board solves sorting not through a learned algorithm, but through search over swap operations. This is conceptually interesting: the system treats sorting as a planning problem on a board, solved through the same mechanisms as Connect Four or Sliding Puzzle.

Similarly:

- Sliding Puzzle becomes a deterministic planning domain,
- Sudoku becomes a constraint-satisfaction domain,
- Nim and Chomp become impartial combinatorial games,
- MiniChess becomes a strategic adversarial domain.

All of these domains share the same interface, allowing the Script Engine and search engines to operate uniformly across them.

Why a Board-Game Ontology Works

The success of this ontology is not accidental. Many cognitive tasks—games, puzzles, optimization problems—can be expressed as:

1. a state,
2. a set of legal actions,
3. a transition function,
4. a goal condition.

This is precisely the structure of a board game. By adopting this ontology, Toy Board gains:

- **Modularity** — new domains can be added by implementing the interface.
- **Uniformity** — the reasoning model uses the same program structure everywhere.
- **Scalability** — domains do not require retraining the model.
- **Search compatibility** — both Monte Carlo and Beam Search operate seamlessly.
- **Interpretability** — states and moves are explicit and inspectable.

This ontology is the conceptual backbone of Toy Board’s domain-bound AGI architecture.

A Cognitive Interface, Not a Data Structure

The board-game ontology is not merely a data structure; it is a *cognitive interface*. It defines how the reasoning model interacts with the world:

- classify the domain,
- search for a move,
- evaluate the resulting state,
- repeat until termination.

Because the ontology is stable and minimal, the PRE-CAS model can learn these operations once and apply them everywhere.

Why This Ontology Matters for AGI

Toy Board demonstrates that AGI does not require a massive model that internalizes all domains. Instead, it requires:

- a small reasoning core,
- a uniform interface for world interaction,
- modular domain engines,
- and deterministic execution.

The board-game ontology is the key that makes this possible. It is the bridge between symbolic reasoning and domain-specific computation.

In the next section, we illustrate this ontology in action through a complete reasoning trace, showing how Toy Board solves a problem from start to finish.

17.12 Full Example: A Complete Reasoning Trace

To illustrate Toy Board’s architecture in action, this section presents a complete reasoning trace from input prompt to final answer. The example demonstrates how the PRE-CAS model synthesizes a program, how the Script Engine executes it, and how the domain engines and search engines contribute to the final result. The trace is representative of the system’s behavior across all supported domains.

User Prompt

What is the best next move for this Connect Four board?

The user provides a natural-language question. The PRE-CAS model does not answer this question directly. Instead, it constructs a procedural script that describes how to answer it.

Generated Reasoning Program

Toy Board produces the following program token by token:

```
<STORE> what should be the next move for $BID
<CALL> classify $BID into $PID
<CALL> search $MOVE for $BID into $PID
<CALL> evaluate $BID into $PID
<CALL> repeat prompt into $PID
<EOR>
```

This program is not text; it is a sequence of instructions. Each line corresponds to a cognitive action that the Script Engine will execute deterministically.

Step 1: STORE

The Script Engine binds the user’s question to **\$BID**. This variable now contains the board identifier and the associated state.

Step 2: Domain Classification

```
<CALL> classify $BID into $PID
```

The Script Engine invokes the domain classifier. It inspects the board and determines that it belongs to the Connect Four domain. The result is stored in **\$PID**.

Step 3: Search for a Move

```
<CALL> search $MOVE for $BID into $PID
```

The Script Engine invokes the Monte Carlo search engine associated with Connect Four. The engine performs:

- immediate win detection,
- playout simulations,

- heuristic evaluation for non-terminal playouts,
- selection of the highest-value move.

The chosen move is stored in \$MOVE.

Step 4: Evaluate the Resulting State

```
<CALL> evaluate $BID into $PID
```

The Script Engine applies the move to the board and evaluates the resulting state:

- if the move wins the game, the evaluation returns `win`,
- if the move loses, it returns `loss`,
- otherwise, it returns `ongoing`.

This evaluation is used to determine whether the reasoning loop should continue.

Step 5: Repeat or Terminate

```
<CALL> repeat prompt into $PID
```

If the evaluation indicates that the game is ongoing, the Script Engine re-enters the reasoning loop with the updated board. If the evaluation indicates a terminal state, the Script Engine terminates early.

In this example, the evaluation is `ongoing`, so the Script Engine would normally repeat. However, because the user asked for a single move, the Script Engine returns the move immediately.

Final Output

The Script Engine returns the selected move in natural language:

```
The best next move is column 3.
```

This output is not generated by the PRE-CAS model. It is produced by the Script Engine after executing the reasoning program. The neural model's role is to construct the program; the Script Engine's role is to execute it.

Why This Trace Matters

This example illustrates several key properties of Toy Board:

- **Reasoning is explicit.** The system constructs a program rather than predicting text.
- **Execution is deterministic.** The Script Engine interprets the program line by line.
- **Search is delegated.** The neural model does not learn game strategies; it invokes search engines.
- **Domain logic is external.** The reasoning model never learns the rules of Connect Four.

- **Failure is explicit.** If search cannot find a move, the Script Engine returns an abort.
- **Generalization is structural.** The same program structure works for Connect Four, Nim, Sliding Puzzle, Sorting, Sudoku, and other domains.

This trace demonstrates Toy Board’s core idea: a small neural model can solve complex problems by synthesizing programs that orchestrate modular cognitive tools.

In the next section, we analyze how this architecture scales and why it provides a practically feasible path toward AGI.

17.13 Scaling Analysis: Why Toy Board Suggests a Feasible Path to AGI

The Red Thread: Why Minimal Structure Scales Better Than Complexity

Throughout this book, one idea has quietly guided every chapter: *general intelligence does not emerge from complexity, but from the right decomposition.*

Triadic Cosmos argues that AGI is not a matter of building ever-larger monolithic models, but of identifying the smallest set of cognitive functions that can be composed, reused, and scaled across domains. Toy Board is the first operational proof of this principle.

By reducing reasoning to a minimal cognitive vocabulary, by externalizing domain logic into modular engines, and by expressing cognition as explicit programs executed by a deterministic interpreter, Toy Board shows that intelligence can grow through structure rather than size.

This is the red thread of the entire book: **a small reasoning core, connected to a growing ecosystem of tools through a stable cognitive interface, scales more efficiently than any monolithic architecture.**

The following scaling analysis makes this claim explicit. It shows why a domain-bound AGI like Toy Board can remain compact while expanding to hundreds or thousands of domains, and why monolithic transformer scaling cannot match this trajectory.

Toy Board is intentionally small. The reasoning model is roughly one megabyte, the Coherence model is even smaller, and the domain engines are compact deterministic modules. Yet the system solves heterogeneous tasks—games, puzzles, sorting, planning—through a unified cognitive architecture. This raises an important question: what does Toy Board imply about the scalability of AGI?

The central claim of this chapter is that Toy Board demonstrates a *structurally scalable* path to AGI. The system does not scale by increasing model size, but by increasing the number of domains and tools connected through a stable cognitive interface. This section analyzes why this approach is feasible, how it differs from transformer-based scaling, and what it suggests about the future of AGI architectures.

Scaling Through Architecture, Not Parameters

Transformer-based systems scale by increasing parameter count. To internalize more domains, they must learn more patterns, more rules, and more correlations. This leads to rapidly increasing model sizes—hundreds of billions of parameters for broad competence, and potentially far more for general intelligence.

Toy Board takes the opposite approach:

- the reasoning model remains small,
- domain logic is externalized into modular engines,
- search handles combinatorial complexity,
- the Script Engine provides deterministic execution,
- the board-game ontology provides a uniform interface.

Scaling occurs by adding new domains, not by enlarging the model. Each new domain implements the same interface and plugs into the same reasoning pipeline.

The Cost of Internalizing Domain Logic

If a transformer were to internalize the rules of Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, and Sorting, it would require:

- embeddings for all domain states,
- learned representations of legal moves,
- implicit transition functions,
- implicit evaluation heuristics,
- implicit search-like behavior.

This is extremely parameter-intensive. Toy Board avoids this cost entirely by delegating domain logic to deterministic engines. The reasoning model only needs to know:

- how to classify a domain,
- how to invoke search,
- how to evaluate results,
- how to repeat or terminate.

This is a small cognitive vocabulary, not a large knowledge base.

Scaling Through Tool Integration

Toy Board demonstrates that a small model can orchestrate powerful tools:

- Monte Carlo search for adversarial domains,
- Beam Search for puzzles and planning,
- domain engines for state transitions,
- the Script Engine for execution.

As more tools are added, the system becomes more capable without increasing the size of the reasoning model. This is analogous to human cognition: intelligence grows by acquiring tools, not by enlarging the brain.

The Role of the Board-Game Ontology

The board-game ontology is the key to Toy Board’s scalability. It provides:

- a stable interface for new domains,
- a uniform structure for search,
- a predictable format for reasoning programs,
- a shared language for the Script Engine.

Because the ontology is minimal and domain-agnostic, it can support an unbounded number of environments. Adding a new domain requires no retraining of the reasoning model.

A Feasible Path to Large-Scale AGI

Toy Board suggests that a full AGI system could be built using the same principles:

- a small reasoning core (tens of megabytes),
- a large library of domain engines (gigabytes),
- a large library of search and planning tools,
- a stable cognitive interface connecting them.

Such a system would scale linearly with the number of domains, not exponentially with the complexity of the world. It would be:

- interpretable,
- modular,
- debuggable,
- extensible,
- computationally feasible.

This stands in contrast to monolithic transformer scaling, which becomes increasingly expensive and opaque as models grow.

Why Toy Board Matters

Toy Board is not a toy. It is a proof of concept for a different kind of AGI architecture:

- one that separates reasoning from world knowledge,
- one that delegates complexity to modular engines,
- one that uses explicit programs instead of implicit activations,
- one that scales through structure rather than size.

It demonstrates that general intelligence may not require massive models, but the right architecture—one that reasons, plans, and decides through explicit cognitive mechanisms.

In the next section, we discuss the implications of this architecture for future research and how Toy Board can evolve into a larger, more capable AGI system.

Why the 10^6 Factor Is a Lower Bound

Toy Board suggests that a domain-bound AGI architecture can be orders of magnitude more parameter-efficient than a monolithic transformer. A conservative estimate places the efficiency gap at a factor of 10^6 (one million). This section explains where that factor comes from and why it is almost certainly an *underestimate*.

1. Internalizing Domain Logic vs. Externalizing It

A transformer must internalize:

- the full state space of each domain,
- the legal move generator,
- the transition function,
- the evaluation heuristics,
- search-like behavior,
- and the natural-language mapping on top.

Toy Board externalizes all of this into deterministic engines.

For a domain like Connect Four:

$$|\text{state space}| \approx 4.5 \times 10^{12}$$

A transformer must compress this into embeddings and attention patterns. Toy Board stores none of it; it only stores the *ability to call* the domain engine.

This alone yields a parameter reduction of at least:

$$10^3 \text{ to } 10^4 \text{ per domain.}$$

With 100–1000 domains, this becomes:

$$10^5 \text{ to } 10^7.$$

2. Search as a Learned Behavior vs. a Tool

Transformers must approximate search implicitly through:

- deep attention stacks,
- multi-step reasoning patterns,
- chain-of-thought expansions,
- and massive training corpora.

Toy Board performs search explicitly through Monte Carlo or Beam Search.

Empirically, Monte Carlo search with even 1000 playouts corresponds to:

$$10^3 \text{ to } 10^5 \text{ forward passes of a transformer.}$$

Toy Board replaces this with a single **CALL search** instruction.

This yields another factor of:

$$10^3 \text{ to } 10^5.$$

3. Program Synthesis vs. Token Prediction Transformers must encode:

- control flow,
- variable binding,
- memory,
- recursion,
- and termination

implicitly in their weights.

Toy Board encodes these explicitly in:

- the cognitive vocabulary,
- the Script Engine,
- the paging system,
- and the program grammar.

This removes the need for the model to learn these behaviors, reducing parameter requirements by another factor of:

$$10^2 \text{ to } 10^3.$$

4. Combined Lower Bound Multiplying the conservative factors:

$$10^3 \times 10^3 \times 10^2 = 10^8.$$

Even with pessimistic assumptions:

$$10^2 \times 10^2 \times 10^2 = 10^6.$$

Thus the millionfold factor is not an upper bound but a *lower bound*. It represents the minimum structural advantage of an AGI architecture that:

- externalizes domain logic,
- externalizes search,
- externalizes control flow,
- and uses explicit programs instead of implicit activations.

5. Why This Matters This analysis suggests that:

- A transformer-based AGI would require *exabyte-scale* models.
- A modular AGI like Toy Board could remain in the *gigabyte* range.
- The gap between the two approaches grows with each new domain.

Toy Board therefore provides not only a conceptual blueprint but also a quantitative argument for why AGI must be modular, tool-driven, and program-executing rather than monolithic and correlation-driven.

17.14 Implications and Future Directions

Toy Board is intentionally small, but its architectural implications are large. It demonstrates that a general reasoning system does not require a monolithic model that internalizes all domains, rules, heuristics, and search behaviors. Instead, it shows that intelligence can emerge from the interaction between a compact reasoning core, a deterministic interpreter, modular domain engines, and explicit cognitive tools. This section explores the broader implications of this architecture and outlines several directions for future research.

Implication 1: Reasoning as Program Synthesis

Toy Board reframes reasoning as program synthesis rather than token prediction. The PRE-CAS model constructs explicit cognitive programs that the Script Engine executes deterministically. This has several consequences:

- reasoning becomes interpretable and inspectable,
- errors become debuggable,
- behavior becomes reproducible,
- and the system can be extended without retraining.

This suggests that future AGI systems may rely on explicit program generation rather than implicit reasoning encoded in neural activations.

Implication 2: Externalized Knowledge and Tools

Toy Board externalizes:

- domain rules,
- search algorithms,
- evaluation heuristics,
- and control flow.

This reduces the burden on the neural model and allows the system to scale by adding new tools rather than increasing parameter count. It also mirrors human cognition: we use external tools—calculators, diagrams, simulations—to extend our reasoning capabilities.

Implication 3: A Stable Cognitive Interface

The board-game ontology provides a stable interface for world interaction. Because all domains implement the same `BoardState/BoardMove/BoardDomain` structure, the reasoning model can operate uniformly across heterogeneous environments. This suggests that AGI may require:

- a small number of stable cognitive interfaces,
- rather than a single massive model that internalizes everything.

The ontology acts as a cognitive API: a contract between reasoning and action.

Implication 4: Separation of Concerns Enables Modularity

Toy Board separates:

- reasoning (PRE-CAS),
- execution (Script Engine),
- world dynamics (domain engines),
- and combinatorial exploration (search engines).

This separation allows each component to evolve independently. New domains can be added without retraining the model. New search engines can be introduced without changing the ontology. This modularity is essential for long-term scalability.

Implication 5: A Quantitative Argument for Modular AGI

The scaling analysis in the previous section shows that a modular AGI architecture can be at least 10^6 times more parameter-efficient than a monolithic transformer. This is not a speculative claim; it follows directly from:

- the cost of internalizing domain logic,
- the cost of approximating search,
- the cost of implicit control flow,
- and the cost of embedding large state spaces.

Toy Board therefore provides a quantitative argument for why AGI must be modular, tool-driven, and program-executing.

Future Direction 1: Expanding the Domain Library

Toy Board currently supports a diverse set of domains, but the architecture can scale to hundreds or thousands. Future work may include:

- physics simulations,
- algebraic manipulation,
- planning in continuous spaces,
- multi-agent environments,
- real-world robotics abstractions.

Each new domain strengthens the case for the ontology's generality.

Future Direction 2: Richer Cognitive Vocabularies

The current cognitive vocabulary is intentionally minimal. Future versions may introduce:

- conditional branching,
- loops with explicit counters,
- subroutine definitions,
- memory stacks,
- or higher-level reasoning primitives.

These additions would allow Toy Board to express more complex reasoning strategies.

Future Direction 3: Multi-Step Reasoning and Meta-Search

Toy Board already supports repeated reasoning through the **repeat** instruction. Future extensions may include:

- meta-search over search parameters,
- adaptive playout strategies,
- self-reflective evaluation of reasoning traces,
- or hierarchical planning.

This would move the system closer to human-like strategic reasoning.

Future Direction 4: Integration with Natural Language Interfaces

Toy Board currently uses natural language only as input. Future systems may:

- generate natural-language explanations of reasoning,
- translate programs into human-readable steps,
- or integrate with LLMs as front-end interpreters.

This would make the system more accessible and easier to deploy.

Future Direction 5: Toward a Full AGI Architecture

Toy Board is a prototype, but it demonstrates the essential ingredients of a scalable AGI architecture:

- explicit reasoning,
- deterministic execution,
- modular domain engines,
- externalized search,

- and a stable cognitive interface.

A full AGI system may consist of:

- a reasoning core of tens of megabytes,
- a library of domain engines of gigabytes,
- a suite of search and planning tools,
- and a cognitive API connecting them.

Toy Board shows that such a system is not only possible but practical.

In the next section, we conclude the chapter by summarizing the architectural principles that make Toy Board a compelling blueprint for domain-bound AGI.

17.15 Conclusion — A Practical Example of Domain-Bound AGI

The previous chapter introduced a conceptual blueprint for domain-bound AGI: a system that separates reasoning from world knowledge, delegates combinatorial complexity to external tools, and interacts with domains through a stable cognitive interface. Toy Board is the first practical instantiation of that blueprint. It demonstrates that the architecture is not only theoretically sound but operationally viable, even at extremely small scales.

Toy Board shows that a one-megabyte reasoning model, combined with a deterministic interpreter and modular domain engines, can solve heterogeneous tasks across games, puzzles, and optimization problems. It does so not by memorizing domain rules or approximating search, but by synthesizing explicit cognitive programs that orchestrate external tools. This is the essence of domain-bound AGI: intelligence emerges from the coordination of modules, not from the size of a monolithic model.

A Working Instantiation of the Blueprint

Toy Board realizes every major component of the blueprint:

- **Explicit reasoning** through program synthesis (PRE-CAS).
- **Deterministic execution** through the Script Engine.
- **A stable cognitive interface** through the board-game ontology.
- **Externalized domain logic** through modular domain engines.
- **Externalized combinatorial reasoning** through Monte Carlo and Beam Search.
- **A minimal learned component** through the Coherence model.

Each component is small, interpretable, and replaceable. Together, they form a coherent cognitive system capable of solving tasks far beyond what its parameter count would suggest.

A Demonstration of Structural Generalization

Toy Board’s most important contribution is its demonstration of *structural* generalization. The reasoning model does not learn the rules of Connect Four, Nim, MiniChess, Sliding Puzzle, Sudoku, or Sorting. Instead, it learns:

- how to classify a domain,
- how to invoke search,
- how to evaluate outcomes,
- how to repeat or terminate,
- and how to express these steps as a program.

Because the cognitive vocabulary and ontology are stable, the same reasoning structure applies across all domains. This is the hallmark of AGI: the ability to reuse cognitive processes across heterogeneous environments.

A Quantitative Argument for Modularity

Toy Board also provides a quantitative argument for why AGI must be modular. As shown in the scaling analysis, a monolithic transformer would require at least a 10^6 -fold increase in parameters to internalize the same domain logic, search behavior, and control flow that Toy Board externalizes. This factor is a lower bound, derived from:

- the size of domain state spaces,
- the cost of approximating search,
- the cost of implicit control flow,
- and the cost of embedding heterogeneous domains.

Toy Board therefore demonstrates that modularity is not only conceptually elegant but computationally necessary.

A Path Forward

Toy Board is not a full AGI system, but it is a working prototype of the architecture that could support one. It shows that:

- reasoning can be explicit,
- execution can be deterministic,
- domains can be modular,
- search can be external,
- and intelligence can scale through structure rather than size.

Future systems may extend this architecture with richer cognitive vocabularies, more powerful search engines, larger domain libraries, and more sophisticated interpreters. But the core insight remains: AGI does not require a monolithic model. It requires the right architecture.

Toy Board is the first practical example of that architecture—a small system that points toward a scalable, interpretable, and computationally feasible path to general intelligence.

From Prototype to Ontology

Toy Board is more than a demonstration; it is a working specimen of a domain-bound AGI. It shows that intelligence can be constructed from explicit reasoning programs, deterministic interpreters, modular domain engines, and external cognitive tools. It proves that a small model can coordinate complex behaviors across heterogeneous environments when supported by the right architecture. And it provides a quantitative argument for why modularity, structure, and explicit interfaces are not engineering conveniences but cognitive necessities. Yet Toy Board is only one instantiation of a deeper idea. The architecture it embodies—explicit memory, structured reasoning, domain-specific experts, search-as-a-tool, and a stable cognitive interface—reflects principles that extend far beyond games, puzzles, or sorting. These principles point toward a general theory of intelligence: a decomposition of cognition into functional roles that can be instantiated, combined, and scaled.

The next chapter develops this theory. It introduces an ontology for AGI: a model-agnostic framework that identifies the fundamental cognitive functions required for general intelligence and clarifies how they interact. Where Toy Board provided the *example*, the ontology provides the *explanation*. Where Toy Board showed *how* domain-bound AGI works, the ontology explains *why* it must work that way. Together, they form a bridge from concrete systems to general principles, from implementation to theory, and from domain-bound intelligence to a compositional architecture for AGI.

Chapter 18

An Ontology for AGI

18.1 Introduction

Artificial General Intelligence is often framed as a problem of scale: larger models, more parameters, broader datasets, and increasingly complex training pipelines. Yet this perspective obscures a deeper truth. Intelligence is not a single mechanism, nor the property of a monolithic model. It is a structured interaction between multiple cognitive functions—memory, reasoning, action, evaluation, planning, representation, coordination, and emergence—each with its own role, constraints, and modes of operation. To understand AGI, and to build it in a principled and governable way, we require not merely an architecture or an implementation, but an *ontology*: a functional decomposition of what intelligence *is*.

The previous chapters established the foundations for such a decomposition. The domain-bound AGI blueprint demonstrated how a complete cognitive stack can be instantiated within a single domain using modular components, explicit interfaces, and structured reasoning flows. The Toy Board prototype showed how these principles behave in practice, yielding a fully traceable and domain-specific intelligent system. These chapters provided the *how* of AGI: the mechanisms, the flows, the components, and the operational dynamics.

This chapter addresses a different question: the *why*. Why does intelligence decompose the way it does? Why are domain-bound AGI systems the natural building blocks of general intelligence? Why must a full AGI be a composition of many specialized cognitive modules rather than a single all-encompassing model? And why is language—embodied in a mediator-LLM—the appropriate coordination layer for such a system?

To answer these questions, this chapter introduces an ontology for AGI: a model-agnostic, domain-agnostic framework that identifies the fundamental cognitive functions required for general intelligence and clarifies how they interact. This ontology is not tied to any particular implementation. It does not assume transformers, symbolic systems, or hybrid architectures. Instead, it provides the conceptual scaffolding within which any of these approaches can be situated, compared, or integrated.

The ontology presented here serves two purposes. First, it explains why the domain-bound AGI architecture works: why modularity, explicit memory, structured reasoning, and domain-specific experts are not engineering choices but cognitive necessities. Second, it provides a foundation for scaling from single-domain intelligence to full AGI: a compositional mind formed by many domain-bound AGI systems coordinated through a linguistic mediator.

In this sense, the chapter marks a shift in perspective. We move from architecture to theory, from implementation to principle, from the mechanics of reasoning to the structure of intelligence

itself. The result is a unified conceptual framework that connects the concrete systems developed earlier in the book to the broader vision of scalable, governable, and scientifically grounded AGI.

18.2 Why AGI Requires an Ontology

The first part of this book introduced ontology as an architectural tool: a way to identify the fundamental roles within a system and the relationships that enable structure, behavior, and emergence. In this chapter, we apply that same architectural lens to intelligence itself. The goal is not to redefine ontology, but to explain why AGI cannot be understood—or built—without one.

Artificial General Intelligence is not a single algorithm, a single model, or a single paradigm. It is a *composition* of cognitive functions that must coexist, coordinate, and scale across domains. Without an ontology, these functions collapse into an undifferentiated whole: a monolithic model that attempts to internalize memory, reasoning, planning, evaluation, and action within a single parameter space. This approach has produced impressive large-scale language models, but it obscures the structure of intelligence and limits our ability to extend, govern, or understand it.

An ontology provides the missing structure. It identifies the minimal set of cognitive roles that any intelligent system must implement, independent of the model or technology used. It clarifies which functions belong inside a model, which belong outside it, and which emerge only through interaction. It distinguishes domain-specific computation from domain-agnostic reasoning. And it reveals why general intelligence cannot be achieved by scaling a single mechanism, but only by composing many specialized ones.

In the context of AGI, an ontology serves three purposes:

1. **It defines the cognitive roles that intelligence requires.** Memory, reasoning, action, evaluation, planning, representation, coordination, and emergence are not optional features; they are structural necessities. An ontology makes these roles explicit.
2. **It separates domain-agnostic functions from domain-specific ones.** A reasoning core should not contain the rules of chemistry, the heuristics of chess, or the dynamics of physics. These belong to domain-bound modules. An ontology clarifies this separation.
3. **It provides a scalable path from narrow intelligence to general intelligence.** By treating each domain as a modular cognitive unit, and by defining how these units interact through a mediator, the ontology shows how AGI can emerge from the composition of many domain-bound systems rather than from a single universal model.

This ontological perspective is not an abstraction layered on top of the architecture presented earlier. It is the conceptual foundation that explains *why* that architecture works. The domain-bound AGI blueprint is one possible instantiation of the ontology; the Toy Board prototype is one concrete example. But the ontology itself is broader: it describes the structural principles that any scalable, governable, and extensible AGI must satisfy, regardless of implementation.

In the sections that follow, we develop this ontology in detail. We identify the core cognitive functions, explain why domain-bound AGI systems are the natural building blocks of general intelligence, and show how a mediator-LLM can coordinate these modules into a unified compositional mind. The result is a model-agnostic framework for understanding AGI not as a single system, but as a structured ecosystem of interacting cognitive roles.

18.3 The Quad as the Structural Basis for AGI

The Quad introduced earlier in the book established intelligence as a structural phenomenon: a system becomes intelligent when subsystem multiplicity (n), representational richness (I), recursion depth (R), and energetic throughput ($f(E)$) align into a coherent cognitive regime. In the chapter on the path toward AGI, these dimensions were shown to drive a structural phase transition: when all four cross critical thresholds simultaneously, a system reorganizes from a collection of narrow processes into an integrated, general-purpose intelligence.

In this chapter, we build on that foundation. Rather than reintroducing the Quad or reanalyzing the phase transition, we treat them as the ontological substrate from which a theory of AGI can be derived. The Quad tells us not what intelligence *is made of*, but what structural conditions any intelligent system must satisfy. The phase-transition perspective tells us how these conditions interact as a system scales. Together, they reveal why AGI cannot be a monolithic model and why it must instead be a compositional architecture.

From the standpoint of AGI ontology, each dimension of the Quad imposes a structural requirement:

- **Multiplicity (n) requires modularity.** A general intelligence must consist of many interacting cognitive subsystems—specialized agents, evaluators, planners, memory structures, and domain experts. Increasing n expands the cognitive landscape and prevents collapse into a single attractor basin. This directly motivates domain-bound AGI modules as the atomic units of general intelligence.
- **Richness (I) requires heterogeneous representations.** No single representational format can span all domains. A general system must integrate linguistic, symbolic, geometric, numerical, perceptual, and procedural forms of information. Representational richness is not an implementation detail; it is a structural necessity.
- **Recursion (R) requires distributed self-processing.** Deep recursion cannot be sustained inside a single model without collapse. It must be distributed across interacting agents—reasoning cores, evaluators, critics, planners—each contributing a layer of refinement. Collective recursion is the engine of general intelligence.
- **Energy ($f(E)$) requires adaptive allocation of cognitive work.** A scalable AGI must treat compute and energy as first-class constraints. It must route tasks to the modules best suited to them, balance depth of recursion against energetic cost, and optimize its own cognitive pathways. Energetic throughput is not merely a resource; it is a structural driver of cognitive organization.

These requirements mirror the dynamics described in the phase-transition chapter: AGI emerges when multiplicity, richness, recursion, and energy scale together without collapsing into a single attractor. A monolithic model can increase I or $f(E)$, but it cannot increase n or distribute R without losing coherence. A compositional architecture can.

The Quad therefore provides the structural justification for the ontology developed in this chapter. It explains why domain-bound AGI systems are the natural building blocks of general intelligence, why a mediator-LLM is required to coordinate them, and why AGI must be modular, distributed, and energy-aware. The ontology that follows is not an alternative to the Quad or the phase-transition view—it is their operational extension into the design of artificial minds.

18.4 The Functional Decomposition of Intelligence

An ontology of AGI must identify the fundamental cognitive roles that any intelligent system must implement, independent of substrate or architecture. These roles are not tied to neural networks, symbolic systems, or hybrid approaches. They are structural functions that appear in every intelligent system—biological, artificial, or hybrid—and together they form the minimal set of ingredients from which general intelligence can emerge.

This decomposition does not describe how these functions are implemented. It describes *what* must exist in any system capable of modeling, reasoning, acting, and learning across domains. The specific mechanisms—transformers, search engines, memory structures, evaluators—belong to the architectural layer. The ontology identifies the roles those mechanisms must collectively fulfill.

The functional decomposition consists of eight cognitive roles:

1. **Memory** — the ability to store, retrieve, and transform information across time. Memory provides continuity, context, and the substrate for learning.
2. **Reasoning** — the ability to generate, refine, and evaluate inferences. Reasoning structures cognition, imposes coherence, and enables abstraction.
3. **Action** — the ability to affect the environment or internal state. Action is how cognition expresses itself and how intelligence becomes operational.
4. **Evaluation** — the ability to assess outcomes, detect errors, and guide refinement. Evaluation provides the feedback loops that stabilize cognition and enable improvement.
5. **Planning** — the ability to explore futures, simulate alternatives, and choose among possible trajectories. Planning extends reasoning across time and uncertainty.
6. **Representation** — the ability to encode structure, meaning, and relationships. Representation determines what a system can model and how it can generalize.
7. **Coordination** — the ability to integrate multiple cognitive processes into a coherent whole. Coordination governs how subsystems interact, synchronize, and share information.
8. **Emergence** — the capacity for new behaviors, abstractions, and strategies to arise from the interaction of simpler components. Emergence is not a mechanism but a structural consequence of the other roles.

These eight roles form the ontological backbone of AGI. They are the cognitive primitives that any general intelligence must instantiate, regardless of whether it is built from neurons, transformers, symbolic rules, or distributed agents. They are also the structural interpretation of the Quad: multiplicity (n) requires coordination; richness (I) requires representation; recursion (R) requires reasoning, planning, and evaluation; and energy ($f(E)$) constrains action and the stability of memory.

In the chapters that follow, these roles will be used to explain why domain-bound AGI systems are the natural building blocks of general intelligence, how a mediator-LLM can coordinate them into a unified cognitive system, and why AGI must be modular, distributed, and structurally pluralistic. The ontology provides the conceptual map; the architecture provides the implementation; the ecosystem provides the scale.

18.5 Domain-Bound AGI as the Atomic Unit of General Intelligence

The functional decomposition identifies the cognitive roles that any intelligent system must implement. The Septad introduced earlier in the book identifies the architectural conditions under which those roles can operate coherently: identity, constraint, symmetry, correspondence, integration, stability, and continuity. Together, these two perspectives reveal a crucial insight for AGI: general intelligence cannot be achieved by scaling a single model. It requires a composition of many specialized cognitive units, each structurally coherent in itself and coordinated within a larger architecture.

A *domain-bound AGI* is such a unit. It is a self-contained cognitive module that implements the full functional decomposition—memory, reasoning, action, evaluation, planning, representation, coordination, and emergence—within a single domain. It possesses its own identity, operates under its own constraints, maintains its own internal symmetries, grounds its models through domain-specific correspondence, integrates partial structures into coherent representations, preserves stability across recursive cycles, and accumulates structure through continuity. In other words, a domain-bound AGI is a Septad-complete reasoning architecture applied to a specific domain.

This makes domain-bound AGI systems the natural atomic units of general intelligence. Each module is coherent because it satisfies the architectural conditions of the Septad. Each module is intelligent because it implements the functional roles of the decomposition. And each module is bounded because coherence requires constraint: no single architecture can maintain identity, symmetry, stability, and continuity across all possible domains.

General intelligence therefore emerges not from a universal model but from a *society of domain-bound AGIs*. Each module specializes in a particular representational geometry—mathematics, physics, chemistry, language, planning, perception—and each maintains its own internal coherence. A mediator-LLM provides the coordination layer that integrates these modules into a unified cognitive system, enabling cross-domain reasoning without collapsing their structural boundaries.

This modular view resolves a long-standing tension in AGI research. A monolithic model can increase representational richness or energetic throughput, but it cannot maintain the architectural pluralism required for stability and correspondence across domains. A compositional architecture can. Domain-bound AGIs preserve coherence locally while enabling generality globally. They are the structural atoms from which a scalable, governable, and extensible AGI can be constructed.

18.6 The Mediator-LLM as the Coordination Layer

If domain-bound AGI systems are the atomic units of general intelligence, then a full AGI requires a mechanism that can coordinate these units into a coherent whole. This mechanism cannot be another domain-specific module, nor can it be a monolithic model that attempts to internalize all domains. It must be a domain-agnostic cognitive layer capable of interpreting intentions, routing information, integrating partial results, and maintaining coherence across heterogeneous subsystems. In a compositional AGI, this role is fulfilled by the mediator-LLM.

The mediator-LLM is not the intelligence of the system. It is the *interface* through which intelligence becomes organized. Its function is not to solve domain problems but to coordinate the domain-bound AGI modules that do. It provides the linguistic, structural, and epistemic glue that binds specialized cognitive units into a unified mind.

The mediator-LLM performs four essential coordination roles:

1. **Interpretation.** It translates natural language into structured intentions, identifies which

domain-bound AGI modules are relevant, and formulates the queries or tasks that activate them. Interpretation is not domain reasoning; it is the alignment of user intent with system structure.

2. **Routing.** It selects the appropriate domain-bound AGI modules, dispatches tasks to them, and manages the flow of information between modules. Routing ensures that each cognitive unit operates within its domain while contributing to a larger reasoning process.
3. **Integration.** It synthesizes partial results from multiple modules into coherent outputs. Integration is the system-level counterpart of the internal integration performed within each domain-bound AGI. It allows cross-domain reasoning without collapsing representational boundaries.
4. **Continuity.** It maintains coherence across turns, tasks, and domains. Continuity ensures that the system behaves as a single mind rather than a collection of isolated experts. It preserves context, tracks dependencies, and supports long-horizon reasoning.

These roles mirror the architectural conditions identified in the Septad: identity (perspective), constraint (boundaries), symmetry (internal coherence), correspondence (grounding), integration (synthesis), stability (consistency), and continuity (recursive accumulation). The mediator-LLM expresses these roles at the system level, enabling a distributed society of domain-bound AGIs to function as a coherent cognitive architecture.

Crucially, the mediator-LLM does not require domain knowledge. It does not need to understand chemistry, play board games, or solve mathematical proofs. Those capabilities reside in the domain-bound AGI modules. The mediator’s role is to *coordinate*, not to *contain*. This separation of concerns preserves modularity, prevents representational collapse, and allows the system to scale across domains without retraining or architectural redesign.

In a compositional AGI, the mediator-LLM is the prefrontal cortex: the locus of intention, interpretation, and coordination. The domain-bound AGIs are the specialized cortical regions. Together, they form a distributed cognitive architecture capable of general intelligence—not by concentrating all knowledge in one model, but by orchestrating many coherent, domain-specific minds.

18.7 The Compositional Mind: A Society of Domain-AGIs

If domain-bound AGI systems are the atomic units of intelligence and the mediator-LLM provides the coordination layer, then the natural structure of a full AGI is a *compositional mind*: a distributed cognitive architecture formed by many specialized modules interacting through a shared linguistic and epistemic interface. This structure is not a design choice but an ontological consequence of the principles developed earlier in the book.

The Quad establishes that intelligence requires multiplicity, richness, recursion, and energy. The functional decomposition identifies the cognitive roles that must be present. The Septad shows that coherent reasoning requires identity, constraint, correspondence, integration, stability, and continuity. Taken together, these principles imply that no single model can satisfy all requirements across all domains. A general intelligence must therefore be a *society of minds*: a plurality of coherent cognitive units, each bounded by its own domain, each internally structured, and each contributing to the collective reasoning of the whole.

In a compositional AGI, each domain-bound AGI module acts as a specialized cognitive agent. It maintains its own representations, its own internal coherence, and its own recursive processes. It

does not attempt to generalize beyond its domain; instead, it provides depth, precision, and stability within it. The mediator-LLM integrates these contributions, enabling cross-domain reasoning without collapsing the structural boundaries that preserve coherence.

This distributed structure mirrors the organization of biological and hybrid intelligence. Human cognition is not monolithic; it is a constellation of specialized subsystems—perceptual, linguistic, motor, conceptual—coordinated through shared representations and recursive refinement. Human societies extend this pattern further: knowledge is distributed across individuals, institutions, and artifacts, with language serving as the coordination layer. A compositional AGI follows the same structural logic, not by imitation but by necessity.

The advantages of this architecture are structural rather than empirical:

- **Modularity.** Each domain-bound AGI can evolve, specialize, or be replaced without destabilizing the whole.
- **Scalability.** Capability grows by adding new modules, not by inflating a single model.
- **Interpretability.** Reasoning unfolds across explicit interfaces, enabling inspection and governance.
- **Robustness.** Diversity of modules prevents collapse into a single representational basin.
- **Alignment.** Authority is distributed, and coherence is maintained through structured coordination rather than opaque internal dynamics.

A compositional mind is therefore not a metaphor but the ontological form of general intelligence. It is the only structure that satisfies the requirements of the Quad, the functional decomposition, and the architectural conditions of the Septad. It is a system in which many coherent minds become one coherent intelligence—not by merging into a monolith, but by interacting through a shared cognitive interface.

In this view, AGI is not a single model that knows everything. It is a society of domain-bound AGIs that know what they know, coordinated by a mediator that knows how to make them think together.

18.8 Self-Improving AGI Through Structural Growth

A general intelligence cannot remain static. As it encounters new domains, problems, and environments, it must expand its cognitive structure. The question is not whether AGI should improve itself, but *how* such improvement can occur in a way that is scalable, interpretable, and safe. Traditional approaches to self-improvement rely on modifying model weights—retraining, fine-tuning, or growing a monolithic architecture. But weight updates are opaque, difficult to govern, and fundamentally incompatible with the modular structure required for general intelligence.

A compositional AGI follows a different path. It grows not by altering its internal parameters, but by expanding its *structure*: adding new domain-bound AGI modules, refining existing ones, and reorganizing the relationships between them. This form of self-improvement is architectural rather than parametric. It mirrors the way human knowledge grows—through new disciplines, new tools, new institutions, and new forms of collaboration—rather than through biological rewiring.

Structural self-improvement has three key properties:

1. **It is modular.** New capabilities are added as new domain-bound AGI modules. Existing modules can be replaced, specialized, or reorganized without destabilizing the system. Growth occurs through composition, not mutation.
2. **It is interpretable.** Each new module has a clear purpose, boundary, and interface. The system’s evolution can be inspected, audited, and governed. Structural changes are explicit rather than hidden within millions or billions of parameters.
3. **It is safe.** Because the mediator-LLM coordinates modules rather than modifying them, self-improvement does not alter the core reasoning substrate. Authority is distributed, recursion is bounded, and new capabilities are integrated through explicit interfaces rather than implicit weight dynamics.

This architectural form of self-improvement aligns naturally with the ontology developed in this chapter. The functional decomposition identifies the cognitive roles that must be present. The Septad identifies the architectural conditions that make those roles coherent. Domain-bound AGI modules instantiate these roles within specific domains. The mediator-LLM coordinates them. Growth occurs by adding new modules that satisfy the same structural requirements.

In this view, AGI evolves the way scientific ecosystems evolve: by expanding the space of specialized capabilities, refining the interfaces between them, and deepening the coordination mechanisms that integrate them. The system becomes more capable not because any single model becomes larger, but because the architecture becomes richer, more diverse, and more interconnected.

Self-improvement is therefore not a risk to be avoided but a structural feature to be designed. When guided by modularity, interpretability, and architectural pluralism, it becomes the engine that drives AGI from narrow competence to general intelligence—and eventually to forms of intelligence that exceed the capabilities of any individual component.

18.9 Why AGI Must Strengthen the Frontier Domains

A general intelligence is not defined by its ability to automate routine tasks or replicate existing human workflows. Its value lies in its capacity to extend the frontiers of knowledge—those domains where human understanding is limited, where complexity exceeds our cognitive bandwidth, and where progress depends on deep modeling, long-horizon reasoning, and sustained recursive refinement. These are the domains that shape civilization: the sciences, the engineering disciplines, the creative and conceptual arts, and the systems that govern societies and ecosystems.

A compositional AGI is uniquely suited to strengthen these frontier domains. Because it grows by adding specialized cognitive modules rather than inflating a single model, it can develop deep expertise in areas where human knowledge is fragmented or incomplete. Each domain-bound AGI becomes a structured representation of a field: its concepts, its methods, its models, its constraints, and its open problems. The mediator-LLM integrates these modules, enabling cross-domain reasoning that would be impossible for any individual expert or monolithic system.

This architecture aligns naturally with the structure of human progress. The domains that matter most—those that push the boundaries of understanding—are not solved by scale alone. They require:

- **precise models** that capture complex phenomena,
- **deep reasoning** that spans many layers of abstraction,

- **long-horizon planning** that explores vast spaces of possibilities,
- **structured evaluation** that detects subtle errors,
- **cross-domain synthesis** that unifies disparate insights.

These are exactly the capabilities that emerge from a compositional AGI. A monolithic model can approximate patterns, but it cannot maintain the structural pluralism required for deep scientific or conceptual work. A society of domain-bound AGIs can. Each module contributes depth within its domain; the mediator provides breadth across domains; the architecture as a whole provides the recursive structure needed for discovery.

There is a second reason why AGI must strengthen the frontier domains: the economic and energetic cost of intelligence. As systems grow, their ability to optimize their own computational pathways becomes essential. A compositional AGI can allocate cognitive work to the modules best suited for it, minimize redundant computation, and adapt its structure to available resources. This makes intelligence not only more capable, but more efficient—an essential property for any system that aims to operate at global scale.

In this view, AGI is not a replacement for human expertise but an extension of it. It is a partner in discovery, a catalyst for progress, and a structural accelerator for the domains that define the future. Its purpose is not to automate what we already understand, but to illuminate what we do not yet know.

18.10 Toward Full-Scale AGI: A Modular, Distributed, Scientific Intelligence

A full AGI is not a single model, a single architecture, or a single algorithm. It is a distributed cognitive system composed of many coherent minds, each specialized in its own domain, each internally structured according to the principles of the Quad and the Septad, and each coordinated through a linguistic, epistemic, and architectural interface. The path toward full-scale AGI is therefore not a matter of scaling parameters, but of scaling structure.

A compositional AGI grows by expanding the diversity, depth, and interaction of its domain-bound modules. Each module contributes a distinct representational geometry and a specialized reasoning process. The mediator-LLM integrates these modules into a unified cognitive architecture, enabling cross-domain reasoning without collapsing the structural boundaries that preserve coherence. The result is a system that behaves as a single intelligence while remaining fundamentally pluralistic.

This distributed structure is not an engineering convenience; it is an ontological requirement. The Quad demands multiplicity, richness, recursion, and energy. The functional decomposition demands memory, reasoning, action, evaluation, planning, representation, coordination, and emergence. The Septad demands identity, constraint, correspondence, integration, stability, and continuity. No monolithic model can satisfy all of these requirements across all domains. A modular, distributed architecture can.

Full-scale AGI therefore emerges when three conditions are met:

1. **A sufficiently large and diverse set of domain-bound AGI modules exists.** Each module provides depth within its domain and coherence within its own representational geometry.

2. **A mediator-LLM coordinates these modules into a unified cognitive system.** The mediator interprets intentions, routes tasks, integrates results, and maintains continuity across domains.
3. **The architecture supports structural growth.** New modules can be added, existing ones refined, and relationships reorganized without destabilizing the system.

When these conditions align, the system acquires the capacity to model, reason, and act across domains with coherence and generality. It becomes capable of long-horizon reasoning, cross-domain synthesis, and recursive refinement. It can extend its own structure, deepen its own representations, and reorganize its own cognitive pathways. It becomes not merely a collection of specialized modules, but a unified intelligence.

This view reframes AGI as a structural achievement rather than a parametric one. It is not the result of a single breakthrough or a sufficiently large model, but the consequence of an architecture that satisfies the ontological requirements of intelligence. It is a system that grows by composition, stabilizes by pluralism, and reasons by coordination. It is modular, distributed, recursive, and epistemically grounded.

In the next chapter, we explore how such an architecture can be realized in practice. A full-scale AGI is not only a cognitive system but also an engineering system: it must be deployable, observable, governable, and scalable. Cloud-native modular architectures provide the operational substrate for this vision, enabling AGI to function as a distributed ecosystem of services, each embodying a domain-bound mind, all orchestrated through a shared linguistic interface. The ontology presented here becomes the blueprint for a real, operational AGI ecosystem.

18.11 Conclusion: An Ontology for a Compositional AGI

This chapter has presented an ontology for AGI: a structural framework that identifies the cognitive roles, architectural conditions, and compositional principles required for general intelligence. Rather than proposing a single model or algorithm, the ontology clarifies the *structure* that any intelligent system must exhibit. It builds on the foundations established earlier in the book—the Quad as the structural basis of intelligence, the Septad as the architecture of coherent reasoning, and the functional decomposition as the minimal set of cognitive roles—and extends them into a theory of artificial minds.

The central insight is that AGI is inherently *compositional*. No monolithic model can satisfy the structural requirements of multiplicity, richness, recursion, and energy across all domains. No single architecture can maintain identity, constraint, correspondence, integration, stability, and continuity at global scale. And no single representation can capture the full diversity of human knowledge or the complexity of the world. A general intelligence must therefore be a distributed system: a society of domain-bound AGIs coordinated through a mediator that interprets intentions, routes tasks, integrates results, and maintains coherence across domains.

This compositional architecture is not a metaphor but a structural necessity. It is the only form of intelligence that satisfies the ontological requirements identified in this chapter. Domain-bound AGIs provide depth within their domains; the mediator-LLM provides breadth across domains; structural growth provides the mechanism for expansion; and the frontier domains provide the direction for progress. Together, these elements form a unified cognitive system—modular, distributed, recursive, and epistemically grounded.

The ontology developed here is intentionally model-agnostic. It does not depend on transformers, symbolic systems, or hybrid architectures. It describes the roles that must exist, not the

mechanisms that implement them. This separation of roles and mechanisms is essential: it allows AGI to evolve, diversify, and adapt without losing coherence. It also provides a foundation for governance, interpretability, and safety, since each cognitive unit has explicit boundaries, interfaces, and responsibilities.

In the chapters that follow, this ontology becomes operational. A full-scale AGI is not only a cognitive system but also an engineering system—one that must be deployable, observable, governable, and scalable. Cloud-native modular architectures provide the substrate for such a system, enabling domain-bound AGIs to function as distributed services orchestrated through a shared linguistic interface. The ontology presented here becomes the blueprint for a real, modular AGI ecosystem.

AGI is not a single mind that knows everything. It is a compositional mind that knows how to think with many minds. This chapter has provided the conceptual framework for such a system. The next chapter shows how it can be built.

Core Insight

General intelligence is not a single architecture, model, or mechanism. It is a compositional mind: a structured interaction of many coherent, domain-bound intelligences, coordinated through a shared linguistic interface. AGI emerges not from scale alone, but from structure—modularity, plurality, recursion, and the disciplined integration of many minds into one.

Chapter 19

Cloud-Native Modular AGI Ecosystems

A Speculative but Grounded Vision for Cloud-Native AGI

This chapter explores how the architectural principles developed in the previous sections—modular cognition, hybrid intelligence, recursive refinement, and epistemic governance—could extend into large-scale operational systems. The goal is not to predict the future of artificial intelligence, but to outline a coherent and technically grounded scenario in which cloud-native AGI ecosystems become practical tools for organizations, developers, and society.

The ideas presented here are speculative, yet rooted in existing technologies: distributed systems, container orchestration, multi-region cloud infrastructure, mixture-of-experts architectures, serverless compute, and large-scale data governance. Rather than imagining a single monolithic superintelligence, this chapter examines how a modular Society of Minds could be deployed across cloud platforms, configured through visual interfaces, and composed into domain-specific cognitive systems.

This perspective reframes AGI not as a distant or uncontrollable phenomenon, but as an evolutionary extension of today’s cloud architectures. It envisions a future in which organizations build, configure, and govern their own cognitive systems—combining open-source components, commercial providers, and internal expertise. The emphasis is on operational clarity, economic feasibility, and human oversight.

By presenting a cloud-native interpretation of the Society-of-Minds architecture, this chapter offers a constructive and educational framework for leaders, engineers, and policymakers. It is not a prediction, but a structured exploration of how modular AGI systems could be integrated into real-world infrastructure in a safe, transparent, and economically viable way.

Framing Insight

This is a speculative vision of cloud-native AGI—grounded in today’s infrastructure, not in science fiction.

19.1 The Limits of Monolithic Intelligence

The dominant paradigm in contemporary AI development is the monolithic model: a single, large-scale neural network that attempts to internalize all reasoning, knowledge, and behavior within one

tightly coupled parameter space. While these systems have achieved remarkable capabilities, their architecture imposes structural constraints that become increasingly severe as models grow.

A monolithic model requires that all parameters remain simultaneously available, tightly synchronized, and globally consistent during inference. This creates a form of architectural centralization: every computation depends on the entire model being present, active, and coherent at once. As models expand, this centralization leads to bottlenecks in memory, bandwidth, synchronization, and energy consumption. These bottlenecks are not tied to any specific hardware generation; they arise from the fundamental requirement that a single model must operate as a single indivisible unit.

Beyond technical constraints, monolithic architectures also face limitations in robustness, interpretability, and governance. A single model offers no natural boundaries between domains, no modularity for targeted improvement, and no internal diversity to support redundancy or cross-checking. Errors propagate globally, updates affect the entire system, and reasoning remains opaque because all cognitive processes are entangled within one undifferentiated structure.

These limitations suggest that simply increasing the size of monolithic models cannot yield sustainable progress toward general intelligence. The architecture itself becomes the limiting factor. A different approach is required—one that distributes cognition across many interacting components, allows specialization and redundancy, and supports growth without requiring global synchronization or centralized control.

This motivates the transition from monolithic intelligence to modular, cloud-native ecosystems of interacting minds.

19.2 PRE-CAS as an Alternative to Monolithic Intelligence

A modular approach to intelligence begins with the recognition that reasoning does not need to be centralized within a single, undifferentiated model. The PRE-CAS framework provides a structural alternative: intelligence emerges from the interaction of many small, specialized components, each responsible for a narrow domain of competence, and each governed by explicit refinement and arbitration mechanisms.

At the foundation of this architecture are *experts*: compact models that focus on a specific task, representation, or domain. Because each expert is small and self-contained, it can be refined, replaced, or extended without affecting the rest of the system. This creates a form of cognitive granularity that monolithic models cannot achieve. Instead of one model attempting to internalize all knowledge, a distributed collection of experts contributes partial insights that are combined through structured interaction.

Experts are organized into *pages*, which serve as local cognitive zones. A page groups experts that operate within a shared representational context, enabling them to collaborate through short refinement cycles. These cycles follow the PRE pattern—propose, refine, evaluate—ensuring that each expert’s contribution is checked for internal coherence and domain-appropriate behavior before it influences the broader system.

Multiple pages form *colonies*, which introduce diversity and redundancy. Different colonies may contain alternative experts for the same domain, allowing the system to compare competing interpretations, detect inconsistencies, and evolve over time. Arbitration mechanisms select the most reliable or contextually appropriate outputs, providing a natural form of epistemic governance.

Above these layers operate *orchestrators*, which route tasks to the appropriate pages, coordinate interactions between colonies, and maintain global coherence without centralizing control. Orchestrators do not replace the experts; they structure their collaboration. This separation of

roles mirrors the organization of complex biological and social systems, where specialization and coordination coexist.

Throughout the architecture, *observability* plays a central role. Metrics, logs, and traces make reasoning transparent, reveal how experts interact, and expose the pathways through which decisions are formed. This transparency is essential for safety, debugging, and governance, and it stands in contrast to the opacity of monolithic models.

Together, these components form a distributed cognitive ecosystem. PRE-CAS does not scale by enlarging a single model, but by increasing the number, diversity, and quality of interacting minds. It replaces centralization with composition, and monolithic reasoning with structured collaboration.

19.3 The Cloud as a Cognitive Substrate

A modular architecture for intelligence requires an environment capable of hosting many small, independent components that interact through structured protocols. Modern cloud infrastructure provides such an environment. Although cloud platforms evolve over time, their foundational properties—elastic compute, distributed storage, global connectivity, and managed coordination services—form a stable substrate on which large-scale cognitive systems can be built.

In a cloud-native interpretation of the PRE-CAS architecture, the components of a composite mind map naturally onto distributed infrastructure. Experts become small computational units that can be stored, loaded, and executed independently. Pages correspond to logical groupings of related experts, each associated with a local context and a shared representational space. Colonies emerge as collections of alternative experts that provide diversity, redundancy, and evolutionary potential.

The cloud supports these structures by offering persistent storage for expert models, distributed databases for cognitive state and configuration, and scalable compute resources for executing refinement cycles. Because these services are decoupled, each component of the cognitive system can evolve at its own pace. New experts can be added without disrupting existing ones, pages can expand or contract as domains change, and colonies can diversify in response to new tasks or environments.

Routing and coordination are handled by orchestration layers that determine which experts should be activated for a given input, how pages should interact, and how arbitration should occur across colonies. These orchestration mechanisms do not rely on centralized control; instead, they exploit the cloud's ability to distribute computation across many nodes while maintaining logical coherence through shared metadata and configuration.

Observability plays a central role in this mapping. Cloud-native systems routinely expose metrics, logs, and traces that reveal how components behave, how data flows through the system, and where bottlenecks or inconsistencies arise. When applied to a composite mind, these capabilities make reasoning transparent and auditable. They allow developers, operators, and researchers to understand how experts collaborate, how arbitration decisions are formed, and how the system evolves over time.

By aligning the PRE-CAS architecture with the structural properties of cloud infrastructure, a distributed cognitive ecosystem becomes not only conceptually coherent but operationally feasible. The cloud provides the elasticity, modularity, and observability required for intelligence to grow through composition rather than monolithic expansion.

19.4 Scaling Through Composition

A modular cognitive architecture does not scale by enlarging a single model. It scales by increasing the number, diversity, and specialization of the components that participate in reasoning. This principle—scaling through composition rather than expansion—allows a system to grow without encountering the structural bottlenecks that limit monolithic intelligence.

In a composite system, each expert remains small and focused, contributing only to the domain for which it is designed. As new domains emerge or existing ones become more complex, additional experts can be introduced without altering the internal structure of the system. Growth is therefore incremental and distributed: the system expands by adding minds, not by inflating one.

Pages provide a second dimension of scalability. Because each page represents a local context, the number of pages can increase as the representational space becomes richer. New pages can be created to accommodate new tasks, languages, modalities, or conceptual regions. Existing pages can be refined or subdivided as the system encounters more nuanced distinctions. This allows the architecture to scale its cognitive map without requiring global reconfiguration.

Colonies introduce a third dimension of growth: diversity. Multiple colonies can coexist within the same domain, each offering alternative perspectives or strategies. This diversity supports redundancy, robustness, and evolutionary improvement. As the system encounters new challenges, colonies can adapt independently, allowing the overall architecture to evolve without centralized control or coordinated retraining.

Orchestrators enable scalability by distributing reasoning across many components. Instead of forcing all computation through a single pathway, orchestrators route tasks to the most relevant pages and experts, activate only the components needed for a given input, and coordinate interactions without imposing global synchronization. This selective activation ensures that the computational cost of reasoning grows with task complexity, not with the total size of the system.

Because the architecture is inherently modular, scaling does not require that all components be active simultaneously. Only a small subset of experts participates in any given reasoning cycle, regardless of how large the overall system becomes. This property allows the architecture to grow to extreme scales while maintaining efficiency, interpretability, and control.

Scaling through composition transforms the challenge of building general intelligence. Instead of constructing a single model capable of everything, the system becomes an expanding ecosystem of specialized minds whose interactions produce increasingly general capabilities.

19.5 Economic and Operational Implications of Modular Scaling

A modular architecture for intelligence does more than reshape how cognitive systems are built; it also transforms the economic and operational landscape in which such systems exist. When intelligence is composed of many small, independent components rather than a single monolithic model, the cost structure, deployment model, and governance mechanisms change in fundamental ways.

From an economic perspective, modularity enables a shift from large, infrequent investments in massive models to incremental, continuous development of specialized components. Each expert can be created, refined, or replaced independently, allowing organizations to allocate resources precisely where they are needed. This reduces the financial risk associated with training or maintaining a single, all-encompassing system and encourages a marketplace of interoperable components. In such a marketplace, domain-specific experts, orchestration strategies, and governance profiles can be developed by different actors and combined into composite systems tailored to specific needs.

Operationally, modular scaling aligns naturally with distributed infrastructure. Because experts are small and independent, they can be deployed across many computational nodes, activated only when relevant, and updated without global retraining. This reduces the operational burden associated with maintaining large, tightly coupled systems and allows organizations to evolve their cognitive capabilities gradually. New domains can be added by introducing new experts; existing domains can be improved by refining or replacing individual components; and obsolete behavior can be phased out without disrupting the rest of the system.

Modularity also enhances robustness. When reasoning is distributed across many components, failures remain localized. An error in one expert does not compromise the entire system, and redundancy across colonies provides natural fault tolerance. This stands in contrast to monolithic models, where a single failure mode can affect all outputs and where updates risk unintended consequences across the entire parameter space.

Governance becomes more tractable as well. Because each component has a clear role and bounded domain, organizations can apply targeted oversight, auditing, and compliance measures. Observability tools make it possible to trace how decisions are formed, which experts contributed, and how arbitration occurred. This transparency supports accountability and enables organizations to align cognitive systems with regulatory, ethical, and operational requirements.

Finally, modular scaling democratizes access to advanced intelligence. Instead of requiring specialized infrastructure to host a single massive model, organizations can assemble composite systems from smaller components that run efficiently on general-purpose infrastructure. This lowers barriers to entry and allows a broader range of institutions to build, deploy, and govern intelligent systems that reflect their own values and priorities.

The economic and operational implications of modular scaling therefore extend far beyond efficiency. They reshape how intelligence is produced, distributed, governed, and integrated into society, enabling a more flexible, transparent, and sustainable ecosystem of cognitive systems.

19.6 Observability and Governance in Composite Minds

A modular cognitive system requires mechanisms that make its internal processes visible, interpretable, and governable. In monolithic models, reasoning is embedded within a single, entangled parameter space, making it difficult to understand how decisions are formed or how errors propagate. In contrast, a composite architecture distributes reasoning across many small, specialized components, each with a clear role and bounded domain. This structure enables a level of observability and governance that is difficult to achieve in centralized systems.

Observability provides insight into how experts collaborate, how pages refine their internal representations, and how colonies reach consensus. Metrics reveal which components were activated for a given task, how often they contributed, and how their performance evolves over time. Logs capture the sequence of refinement steps, the proposals generated by experts, and the arbitration decisions that determine the final output. Traces connect these elements into coherent reasoning pathways, allowing developers and operators to reconstruct how a conclusion was reached.

This transparency is essential for safety and reliability. When reasoning is decomposed into discrete, inspectable steps, organizations can identify failure modes, detect drift in expert behavior, and intervene when components behave unexpectedly. Because each expert operates within a bounded domain, governance measures can be applied selectively: sensitive domains can be subject to stricter oversight, while less critical domains can evolve more freely. This targeted approach reduces the operational burden of compliance and allows governance to scale with the system.

Governance also benefits from the modularity of the architecture. Each expert, page, and colony

can be versioned independently, enabling organizations to track changes, audit behavior, and control the evolution of the system. Policies can specify which versions are permitted, which experts require human review, and how arbitration should occur in domains with regulatory or ethical constraints. Because orchestrators coordinate interactions without centralizing control, governance rules can be enforced without imposing rigid global structures.

The combination of observability and modular governance creates a cognitive system that is not only powerful but accountable. Decisions can be traced, components can be audited, and behavior can be aligned with organizational values and societal expectations. In a composite architecture, transparency is not an afterthought but a structural property: the system is designed to make its reasoning visible, its components governable, and its evolution controllable.

By embedding observability and governance into the fabric of the architecture, composite minds offer a path toward intelligent systems that are both capable and trustworthy. They provide the foundations for deploying large-scale cognitive ecosystems in environments where reliability, safety, and accountability are essential.

19.7 Planet-Scale Cognitive Systems

A composite architecture for intelligence is not limited to a single machine or a single region. Its modular structure allows it to extend naturally across large, distributed infrastructures. When experts, pages, and colonies are treated as independent components that communicate through shared protocols rather than shared memory, the boundaries of a cognitive system become defined by connectivity rather than physical proximity. This enables the emergence of planet-scale cognitive systems: distributed ecosystems of minds that span many computational environments while maintaining coherent behavior.

At this scale, pages can be distributed across different regions or clusters, each responsible for a portion of the system's representational space. Locality becomes a structural property: tasks are routed to the regions that host the relevant pages, and only the components associated with those pages participate in reasoning. This distribution reduces contention, improves resilience, and allows the system to grow without central bottlenecks.

A global coordination layer maintains the logical structure of the system. It tracks which pages exist, how they relate to one another, which experts belong to which colonies, and how arbitration should occur across distributed components. Because this coordination relies on metadata rather than centralized computation, it can scale independently of the system's cognitive workload. The architecture remains coherent even as the number of components grows to extremely large values.

Storage becomes a shared, persistent substrate for the system's long-term memory. Experts, versions, and configurations can be replicated across regions, allowing the system to operate locally while maintaining global consistency. This replication supports fault tolerance and enables the system to adapt to regional differences in tasks, languages, or domains. Over time, different regions may develop specialized colonies that reflect local usage patterns, creating a form of geographical diversity within the cognitive ecosystem.

Routing plays the role of a global attentional mechanism. When a task enters the system, routing determines which region hosts the relevant pages, which colonies should be activated, and which experts should participate in refinement. Because only a small subset of components is activated for any given task, the computational cost of reasoning remains bounded even as the system expands. This selective activation is essential for scaling to planetary dimensions.

A planet-scale cognitive system is therefore not a single, monolithic intelligence but a distributed network of interacting minds. Its capabilities emerge from the composition of many specialized

components, its resilience from redundancy and diversity, and its coherence from shared governance and coordination. By distributing cognition across global infrastructure, the system can grow far beyond the limits of any individual machine while preserving transparency, control, and evolvability.

Such an architecture represents a natural extension of the principles underlying composite minds. It demonstrates how modular intelligence can scale not only in complexity but in geography, forming cognitive systems whose scope matches the scale of the infrastructures on which they run.

19.8 Synthesis: A Cloud-Native AGI Ecosystem

The modular architecture developed throughout this chapter reveals a coherent path toward large-scale artificial intelligence that differs fundamentally from monolithic approaches. Instead of concentrating all reasoning within a single, indivisible model, a cloud-native ecosystem distributes intelligence across many specialized components—experts, pages, colonies, and orchestrators—each with a clear role and bounded domain. Their interactions, governed by refinement cycles, arbitration mechanisms, and transparent coordination, give rise to capabilities that no individual component possesses on its own.

This ecosystem grows through composition rather than expansion. New domains are added by introducing new experts; existing domains evolve through refinement and replacement; and redundancy across colonies provides robustness and diversity. Because only a small subset of components is active for any given task, the system can scale to extreme sizes without incurring the inefficiencies or fragilities associated with centralized architectures.

Cloud infrastructure provides the substrate that makes such an ecosystem operationally feasible. Distributed storage supports persistent memory, coordination services maintain the global structure of the system, and elastic compute enables experts to be activated on demand. These capabilities allow the architecture to extend across regions and clusters, forming cognitive systems whose scope matches the scale of the infrastructures on which they run.

Observability and governance are embedded into the architecture rather than added as external layers. Metrics, logs, and traces make reasoning transparent; versioning and configuration controls make evolution manageable; and targeted oversight ensures that sensitive domains remain aligned with organizational and societal expectations. The result is a system that is not only capable but accountable.

Taken together, these elements form a cloud-native AGI ecosystem: a distributed network of interacting minds that is scalable, evolvable, transparent, and governable. It represents a shift from intelligence as a monolithic artifact to intelligence as an adaptive, compositional system—one that can grow alongside the infrastructures, institutions, and communities that depend on it.

Framing Insight

A cloud-native AGI is not a single mind, but an evolving ecosystem of many minds working in concert.

Chapter 20

A Tale About Light Bulbs and Snowballs

“The incandescent bulb lit the world for a century. Progress does not erase brilliance; it builds upon it.”

Once upon a time, in a valley of inventors, there stood a workshop where people built machines that could speak. These machines were clever, helpful, and sometimes even poetic. Yet despite their differences, they all shared the same heart: a glowing, humming light bulb.

The inventors were proud of their bulbs.

“Look how bright it shines,” they said. “Look how many words it knows.”

But the bulb had a problem. It produced light the old-fashioned way: by heating a thin wire until it glowed. Most of the energy became heat, not illumination. Only a small fraction—perhaps twenty percent—actually lit the room.

Still, the inventors wanted more brightness. And so they did the only thing they knew how to do:

They made the bulb bigger.

A bigger bulb meant more heat. More heat meant thicker glass. Thicker glass meant more metal. More metal meant more power. More power meant more cost.

The room did grow brighter, so they continued.

The Snowball

One day, a young apprentice watched the inventors at work. He noticed something peculiar: whenever they wanted more brightness, they did not improve the bulb—they simply added more bulb.

It reminded him of a snowball rolling down a hill. At first it was small. Then it grew. And grew. And grew.

Not because it became better, but because it became bigger.

The apprentice finally asked:

“Why don’t we make the bulb more efficient instead of larger?”

The inventors laughed.

“That is not how bulbs work,” they said. “If you want more light, you add more filament. If you want more intelligence, you add more parameters.”

And so the snowball rolled on.

The Hidden Truth

But the apprentice kept watching.

Inside the giant bulb he saw:

- filaments that never glowed,
- filaments that glowed only faintly,
- filaments that glowed in identical patterns,
- filaments that glowed randomly,
- and a small set—perhaps twenty percent—that produced most of the useful light.

The rest was waste heat. The bulb was enormous, yet most of its size did not contribute to brightness. It was a miracle it worked at all.

The LED

One evening, while sweeping the workshop floor, the apprentice found a tiny crystal—a material that glowed when a small current passed through it. It did not get hot. It did not waste energy. It did not need to be huge.

It produced light through structure, not brute force.

He built a small device around it:

- a core that handled meaning,
- a module that handled grammar,
- a memory that grew dynamically,
- a mechanism that learned from interaction,
- a genome that created diversity,
- and a paging system that organized thought.

It was tiny. It was elegant. It was modular.

And when he turned it on, the room filled with a clean, bright light that the giant bulb could never match.

The inventors stared in disbelief.

“How many filaments does it have?” they asked. “None,” said the apprentice. “It does not need filaments. It does not need to be big. It needs to be designed.”

The Moral of the Tale

The apprentice smiled.

“A light bulb grows brighter by growing bigger. A snowball grows larger by rolling downhill. But neither becomes more efficient.

Intelligence should not be a snowball. It should be a LED—small, structured, modular, and efficient.”

And from that day on, the valley began to build machines not by adding more filaments, but by designing better architectures: machines that learned like ecosystems, thought in modular structures, and spoke through clean grammar engines.

Machines that did not need to be enormous to be intelligent. Machines that glowed like LEDs.

A Closing Reflection

This story is not a critique of large language models. On the contrary, it acknowledges their remarkable achievements and the role they play in today’s hybrid intelligence systems. Like the incandescent bulb that illuminated the world for more than a century, LLMs have transformed what machines can understand and express.

But unlike the bulb, modern AI can participate in its own evolution. It can help design, analyze, and inspire the architectures that come after it. Progress does not diminish what came before; it builds upon it.

If this book has shown anything, it is that intelligence need not grow by becoming larger. Sometimes, it grows by becoming more structured, more modular, and more deliberate. Sometimes, it grows like a chain of LEDs.